

Ислямски празници الأذان

В раздел „Месец“ има превключвател „Часове | Празници“: всички ислямски празници и свещени нощи за годината, по месеци, със следващия празник отгоре. Датите са точно като обявените от Мюфтийството и Диянет, а напомнянето се включва за всеки празник поотделно.

Мюфтийство = Диянет

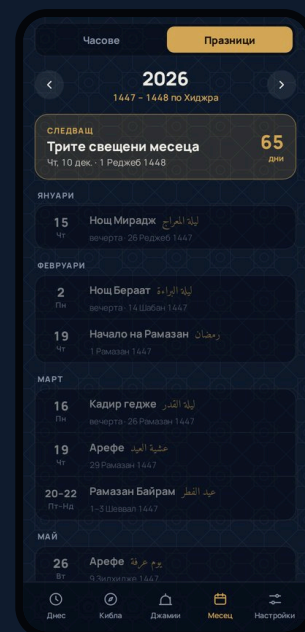
43 от 43 дати съвпадат

Умм ал-Кура

Напомняне при Магриб

Jest – 266 теста

21 файла



Какво е решено в този етап

„Месец“ → „Часове | Празници“ (без лента в „Днес“ и без шести раздел) · годината по Григорианския календар, по Хиджра отдолу · датите от таблица на месеците по Хиджра, не „горе-долу“ · България: Мюфтийството (същият календар като Диянет) · Саудитска Арабия и Залива: Умм ал-Кура – Рамазан и Байрамите като „очаквани“ · напомняне за всеки празник: свещена нощ – същата вечер при Магриб, празник – вечерта преди · датата по Хиджра в „Днес“ и „Месец“ – от същата таблица.

СЪДЪРЖАНИЕ

01 Какво прави етапът

02 Без нов build

03 Откъде са датите

04 Чии дати – Настройки

05 Напомнянията

06 Екраните и текстовете

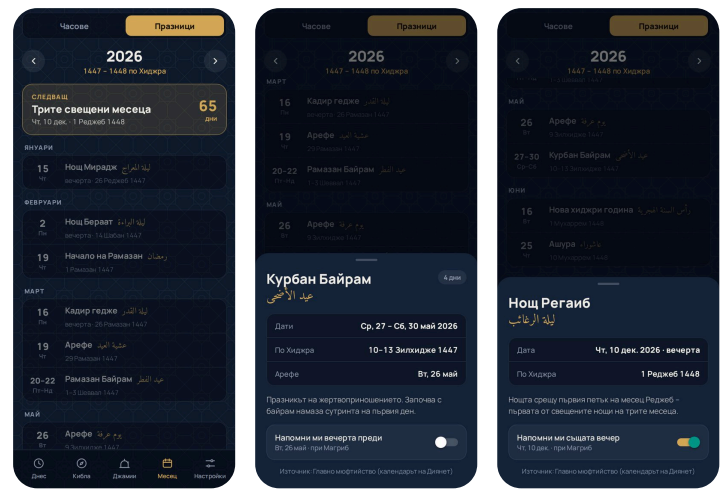
07 Тестове

08 Проверка на телефона

Стъпка 1

Какво прави етапът

„Месец“ има два изгледа: часовите за месеца (както досега) и празниците за годината.



Празниците 2026 · Курбан Байрам · Нощ Регаиб с включено напомняне

- Годината със стрелки; отдолу – годините по Хиджра („1447 – 1448 по Хиджра“).
- Следващият празник – златна карта с броя на дните. Изминалите празници са бледи.
- Всеки ред: датата и денят, името и арабското име, датата по Хиджра; при свещените нощи – „вечерта“.
- Докосване: датите по двата календара, Арефе (при Байрамите), кратко обяснение и „Напомни ми“.

ПРАЗНИКЪТ	ПО ХИДЖРА	КОГА
Трите свещени месеца	1 Реджеб	денят
Нощ Регаиб	четвъртъкът срещу първия петък на Реджеб	вечерта
Нощ Мирадж	26 Реджеб	вечерта
Нощ Бераат	14 Шабан	вечерта
Начало на Рамазан	1 Рамазан	денят
Кадир гедже	26 Рамазан	вечерта
Арефе · Рамазан Байрам	последният ден на Рамазан · 1–3 Шеввал	дните
Арефе · Курбан Байрам	9 · 10–13 Зилхидже	дните
Нова хиджри година · Ашура	1 · 10 Мухаррем	денят
Мевлид	11 Ребиул-евел	вечерта

Стъпка 2

Без нов build

Няма нов native код и нови пакети – промяната е само в JavaScript и данните.

Ако приложението е отворено с Metro – презарежда се само. Иначе – `npx expo start` и отвори приложението.

Стъпка 3

Откъде са датите

Искането беше: никаква разлика, дори с 1 ден. Затова датите са от таблица на месеците по Хиджра, а не от приблизително изчисление.

- **България:** Мюфтийството ползва календара на Диянет – например 2022 и 2026 г. съвпадат напълно; утвърдените от Министерския съвет дни за 2026 г. (Рамазан Байрам, Курбан Байрам) също.
- **Диянет** не наблюдава луната, а смята по правилата на Единния календар по Хиджра (Истанбул, 2016): месецът започва, ако някъде по света при залез преди 00:00 UTC Луната е на $\geq 8^\circ$ от Слънцето и на $\geq 5^\circ$ над хоризонта (с изключение за Америка и Нова Зеландия). Затова датите са известни години напред.
- **Таблицата** е изчислена по тези правила (с астрономическа библиотека, мрежа $1^\circ \times 1^\circ$ по цялото земно кълбо) и **проверена срещу 43 обявени от Диянет дати за 2021–2027 г. – всички съвпадат**. Диянет е обявил официално датите до януари 2028 г.; по-нататък са по същите правила (и това пише при празника).
- **Умм ал-Кура** (Саудитска Арабия): официалната таблица. Там Рамазан и Байрамите се обявяват по наблюдение на луната – затова са с „≈“ и „очаквана“.



2027 (Мюфтийство) · 2027 в Мека (Умм ал-Кура, очаквани) · „Часове“ – датата по Хиджра от същата таблица

build-hijri.mjs tools

- Пресмята таблицата (≈ 15 мин.) и я проверява срещу обявените дати на Диянет – при разлика спира с грешка.
- Пуска се само при нужда: `npm i --no-save astronomy-engine`, после `node tools/build-hijri.mjs`.

tools/build-hijri.mjs

125 реда

```
/**
 * Таблицата на месеците по Хиджра (етап 13) → src/data/hijri.json
 *
 * npm i --no-save astronomy-engine
 * node tools/build-hijri.mjs
 */
```

```

* 1) Диянет (и Главно мюфтийство в България – ползва същия календар): правилата на
* Единния календар по Хиджра (Истанбул, 2016), както ги прилага Диянет:
* - месецът започва на следващия ден, ако някъде по света при залез преди 00:00 UTC
* Луната е на  $\geq 8^\circ$  от Слънцето (елонгация, геоцентрично) и на  $\geq 5^\circ$  над хоризонта;
* - изключение: ако това е изпълнено след 00:00 UTC, но в Америка (континента),
* и новолунието е преди зората (Фаджр) в Нова Зеландия – месецът също започва.
* Проверено срещу 43 обявени от Диянет дати (2021-2027) – всички съвпадат (виж REFERENCE).
* Диянет е обявил официално датите до Шабан 1449 (януари 2028); по-нататък са по същите правила.
* 2) Умм ал-Кура (Саудитска Арабия): официалната таблица от ICU в Node (Intl, islamic-umalqura).
*
* Изчислението е бавно (~15 мин.) – мрежа  $1^\circ \times 1^\circ$  по цялото земно кълбо за всяко новолуние.
*/
import { writeFileSync } from 'node:fs';
import * as A from 'astronomy-engine';

const FROM_YEAR = 2015;
const TO_YEAR = 2046;
const DAY = 86400000;
const STEP = 1;

/** Обявени от Диянет начала на месеци (Gregorian), от „dini günler“ за съответната година. */
const REFERENCE = [
  '2021-02-13', '2021-03-14', '2021-04-13', '2021-05-13', '2021-07-11', '2021-08-09', '2021-10-07',
  '2022-02-02', '2022-03-04', '2022-04-02', '2022-05-02', '2022-06-30', '2022-07-30', '2022-09-27',
  '2023-01-23', '2023-02-21', '2023-03-23', '2023-04-21', '2023-06-19', '2023-07-19', '2023-09-16',
  '2024-03-11', '2024-04-10', '2024-06-07', '2024-07-07', '2024-09-04',
  '2025-03-01', '2025-03-30', '2025-05-28', '2025-06-26', '2025-08-24', '2025-12-21',
  '2026-01-20', '2026-02-19', '2026-03-20', '2026-05-18', '2026-06-16', '2026-08-14', '2026-12-10',
  '2027-02-08', '2027-03-09', '2027-05-07', '2027-06-06',
];

/** Котва за номерацията: 1 Мухаррем 1448 = 16.06.2026 (Диянет). */
const ANCHOR = { date: '2026-06-16', year: 1448, month: 1 };

const iso = (ms) => new Date(ms).toISOString().slice(0, 10);

function criterionMet(conjMs, localDayMs, fromMs, toMs, lon0, lon1, lat0, lat1) {
  for (let lat = lat0; lat <= lat1; lat += STEP) {
    for (let lon = lon0; lon <= lon1; lon += STEP) {
      const obs = new A.Observer(lat, lon, 0);
      // залезът в местния ден: търсим от ~06:00 местно време
      const start = localDayMs - (lon / 15) * 3600000 + 6 * 3600000;
      const sunset = A.SearchRiseSet(A.Body.Sun, obs, -1, new A.AstroTime(new Date(start)), 1);
      if (!sunset) continue;
      const t = sunset.date.getTime();
      if (t < fromMs || t >= toMs || t <= conjMs) continue;
      const moon = A.Equator(A.Body.Moon, sunset, obs, true, true);
      const altitude = A.Horizon(sunset, obs, moon.ra, moon.dec).altitude;
      if (altitude < 5) continue;
      if (A.AngleFromSun(A.Body.Moon, sunset) >= 8) return true;
    }
  }
  return false;
}

/** Първият ден на месеца след новолунието `conj` (UTC дата, ISO). */
function diyanetMonthStart(conj) {
  const c = conj.date.getTime();
  const d0 = Math.floor(c / DAY) * DAY;
  const nz = new A.Observer(-41.3, 174.8, 0);
  for (let k = 0; k < 3; k++) {
    const day = d0 + k * DAY;

```

```

    const next = day + DAY;
    if (criterionMet(c, day, day - 12 * 3600000, next, -180, 180, -60, 60)) return iso(next);
    if (criterionMet(c, day, next, next + 12 * 3600000, -125, -35, -56, 60)) {
        const fajr = A.SearchAltitude(A.Body.Sun, nz, +1, new A.AstroTime(new Date(next - 14 * 3600000)), 1,
-18);
        if (fajr && c < fajr.date.getTime()) return iso(next);
    }
}
}
throw new Error(`no month start after ${conj.date.toISOString()}`);
}

function diyanetTable() {
    const starts = [];
    let t = A.SearchMoonPhase(0, new A.AstroTime(new Date(Date.UTC(FROM_YEAR, 0, 1))), 40);
    const end = Date.UTC(TO_YEAR + 1, 0, 1);
    while (t && t.date.getTime() < end) {
        starts.push(diyanetMonthStart(t));
        process.stdout.write(`\n${starts.at(-1)}\n`);
        t = A.SearchMoonPhase(0, t.AddDays(1), 40);
    }
    process.stdout.write('\n');
    const missing = REFERENCE.filter((d) => !starts.includes(d));
    if (missing.length) throw new Error(`не съвпада с Диянет: ${missing.join(', ')}\n`);
    const a = starts.indexOf(ANCHOR.date);
    // първият месец на таблицата - началото на хиджри година (Мухаррем)
    const firstMuharram = a % 12;
    const months = starts.slice(firstMuharram);
    const firstYear = ANCHOR.year - (a - firstMuharram) / 12;
    return { firstYear, months };
}

function ummAlQuraTable() {
    const f = new Intl.DateTimeFormat('en-u-ca-islamic-umalqura', { day: 'numeric', month: 'numeric', year:
'numeric', timeZone: 'UTC' });
    const months = [];
    let firstYear = null;
    for (let d = Date.UTC(FROM_YEAR, 0, 1); d < Date.UTC(TO_YEAR + 1, 1, 1); d += DAY) {
        const p = f.formatToParts(new Date(d));
        const get = (k) => Number(p.find((x) => x.type === k).value);
        if (get('day') !== 1) continue;
        if (firstYear === null) {
            if (get('month') !== 1) continue;
            firstYear = get('year');
        }
        months.push(iso(d));
    }
    return { firstYear, months };
}

const diyanet = diyanetTable();
const ummalqura = ummAlQuraTable();
writeFileSync(
    new URL('../src/data/hijri.json', import.meta.url),
    JSON.stringify({
        source: 'tools/build-hijri.mjs - Диянет (правилата от 2016, проверени срещу обявените дати) и Умм ал-
Кура (ICU)',
        diyanet: { ...diyanet, officialUntil: [1449, 8] },
        ummalqura,
    }) + '\n',
);

```

```
console.log(`diyanet ${diyanet.firstYear}: ${diyanet.months.length} месеца · umm al-qura  
${ummAlqura.firstYear}: ${ummAlqura.months.length}`);
```

check-holidays.mjs tools

- Годишната проверка (януари, заедно с календара на Мюфтийството): сравнява таблицата с обявления календар на Диянет за текущата и следващата година (Aladhan API, метод DIYANET).
- „OK“ – нищо не се прави. Разлика – показва я; с `--fix` записва датата на Диянет, после `npm test`, `commit` и нова версия.
- `--self-test` – проверка на самия скрипт без интернет.

tools/check-holidays.mjs

102 реда

```
// Годишна проверка на датите на празниците (етап 13): node tools/check-holidays.mjs [година]
//
// Сравнява таблицата на месеците по Хиджра в src/data/hijri.json (Диянет – по нея са и
// празниците, и датата по Хиджра в приложението) с обявления календар на Диянет.
// Пуска се веднъж годишно (януари) – за текущата и следващата година, ако не е дадена година.
//
// Откъде: Aladhan API, методът DIYANET – публикува календара на Диянет
// (https://api.aladhan.com/v1/islamicCalendar/methods). За всеки месец по Хиджра
// пита коя е датата на 1-во число и я сравнява с нашата.
//
// Резултат:
// - „OK“ – всичко съвпада, нищо не се променя;
// - разлика – показва я; с `--fix` записва датата на Диянет в src/data/hijri.json
// (после: npm test, commit и нова версия на приложението);
// - месец, който Диянет още не е обявил – отбелязва се, не е грешка.
// `--self-test` – проверява самия скрипт без интернет (сравнява таблицата със себе си).
import fs from 'node:fs';
import path from 'node:path';
import { fileURLToPath } from 'node:url';

const here = path.dirname(fileURLToPath(import.meta.url));
const FILE = path.join(here, '../src/data/hijri.json');
const API = 'https://api.aladhan.com/v1/hToG';

const args = process.argv.slice(2);
const fix = args.includes('--fix');
const selfTest = args.includes('--self-test');
const years = args.filter((a) => /^d{4}$/.test(a)).map(Number);
if (!years.length) {
  const y = new Date().getFullYear();
  years.push(y, y + 1);
}

const data = JSON.parse(fs.readFileSync(FILE, 'utf8'));
const table = data.diyaneet;
const sleep = (ms) => new Promise((r) => setTimeout(r, ms));
const pad = (n) => String(n).padStart(2, '0');

/** Месеците от таблицата, чието 1-во число е в дадената година: { index, year, month, date }. */
function monthsIn(year) {
  const out = [];
  table.months.forEach((date, i) => {
    if (Number(date.slice(0, 4)) !== year) return;
    out.push({ index: i, year: table.firstYear + Math.floor(i / 12), month: (i % 12) + 1, date });
  });
  return out;
}
```

```

}

/** 1-во число на месеца по Диянет (ISO) или null - още не е обявено / извън календара. */
async function diyanetStart(hYear, hMonth) {
  if (selfTest) return table.months[(hYear - table.firstYear) * 12 + hMonth - 1];
  const url = `${API}/01-${pad(hMonth)}-${hYear}?calendarMethod=DIYANET`;
  for (let attempt = 1; attempt <= 4; attempt++) {
    try {
      const res = await fetch(url, { headers: { accept: 'application/json' } });
      const json = await res.json();
      if (json.code !== 200 || !json.data?.gregorian?.date) return null;
      const [d, m, y] = json.data.gregorian.date.split('-');
      return `${y}-${m}-${d}`;
    } catch (e) {
      if (attempt === 4) throw new Error(`${url}: ${e.message}`);
      await sleep(1500 * attempt);
    }
  }
  return null;
}

let diffs = 0;
let pending = 0;
let checked = 0;
for (const year of years) {
  console.log(`\n${year}`);
  for (const m of monthsIn(year)) {
    const theirs = await diyanetStart(m.year, m.month);
    await sleep(selfTest ? 0 : 400);
    const label = `  1.${pad(m.month)}-${m.year}`;
    if (!theirs) {
      pending++;
      console.log(`${label}  приложението ${m.date}  · Диянет още не е обявил`);
      continue;
    }
    checked++;
    if (theirs === m.date) {
      console.log(`${label}  ${m.date}  OK`);
      continue;
    }
    diffs++;
    console.log(`${label}  РАЗЛИКА: Диянет ${theirs}, приложението ${m.date}`);
    if (fix) table.months[m.index] = theirs;
  }
}

console.log(`\nПроверени ${checked}, разлики ${diffs}, още необявени ${pending}.`);
if (diffs && fix) {
  fs.writeFileSync(FILE, JSON.stringify(data) + '\n');
  console.log('src/data/hijri.json е поправен. Пусни npm test, после commit и нова версия.');
```

Веднъж годишно – трябва да завърши с „OK“

node tools/check-holidays.mjs

- Началото на всеки месец по Хиджра, 1437–1468 (2015–2046), за Диянет и Умм ал-Кура – 10 KB.

src/data/hijri.json

1 реда

```

{"source":"tools/build-hijri.mjs - Диянет (правилата от 2016, проверени срещу обявените дати) и Умм ал-Кура (ICU)","diyanet":{"firstYear":1437,"months":["2015-10-14","2015-11-13","2015-12-12","2016-01-11","2016-02-10","2016-03-10","2016-04-08","2016-05-08","2016-06-06","2016-07-05","2016-08-04","2016-09-02","2016-10-02","2016-11-01","2016-11-30","2016-12-30","2017-01-29","2017-02-28","2017-03-29","2017-04-27","2017-05-27","2017-06-25","2017-07-24","2017-08-23","2017-09-21","2017-10-21","2017-11-19","2017-12-19","2018-01-18","2018-02-17","2018-03-19","2018-04-17","2018-05-16","2018-06-15","2018-07-14","2018-08-12","2018-09-11","2018-10-10","2018-11-09","2018-12-08","2019-01-07","2019-02-06","2019-03-08","2019-04-06","2019-05-06","2019-06-04","2019-07-04","2019-08-02","2019-08-31","2019-09-30","2019-10-29","2019-11-28","2019-12-27","2020-01-26","2020-02-25","2020-03-25","2020-04-24","2020-05-24","2020-06-22","2020-07-22","2020-08-20","2020-09-18","2020-10-18","2020-11-16","2020-12-16","2021-01-14","2021-02-13","2021-03-14","2021-04-13","2021-05-13","2021-06-12","2021-07-11","2021-08-09","2021-09-08","2021-10-07","2021-11-06","2021-12-05","2022-01-04","2022-02-02","2022-03-04","2022-04-02","2022-05-02","2022-05-31","2022-06-30","2022-07-30","2022-08-28","2022-09-27","2022-10-26","2022-11-25","2022-12-24","2023-01-23","2023-02-21","2023-03-23","2023-04-21","2023-05-21","2023-06-19","2023-07-19","2023-08-17","2023-09-16","2023-10-16","2023-11-14","2023-12-14","2024-01-12","2024-02-11","2024-03-11","2024-04-10","2024-05-09","2024-06-07","2024-07-07","2024-08-05","2024-09-04","2024-10-04","2024-11-03","2024-12-02","2025-01-01","2025-01-30","2025-03-01","2025-03-30","2025-04-29","2025-05-28","2025-06-26","2025-07-26","2025-08-24","2025-09-23","2025-10-23","2025-11-21","2025-12-21","2026-01-20","2026-02-19","2026-03-20","2026-04-18","2026-05-18","2026-06-16","2026-07-15","2026-08-14","2026-09-12","2026-10-12","2026-11-10","2026-12-10","2027-01-09","2027-02-08","2027-03-09","2027-04-08","2027-05-07","2027-06-06","2027-07-05","2027-08-03","2027-09-02","2027-10-01","2027-10-30","2027-11-29","2027-12-29","2028-01-28","2028-02-27","2028-03-27","2028-04-26","2028-05-25","2028-06-24","2028-07-23","2028-08-21","2028-09-20","2028-10-19","2028-11-17","2028-12-17","2029-01-16","2029-02-14","2029-03-16","2029-04-15","2029-05-15","2029-06-13","2029-07-13","2029-08-11","2029-09-09","2029-10-09","2029-11-07","2029-12-07","2030-01-05","2030-02-04","2030-03-05","2030-04-04","2030-05-04","2030-06-02","2030-07-02","2030-07-31","2030-08-30","2030-09-28","2030-10-28","2030-11-26","2030-12-26","2031-01-24","2031-02-23","2031-03-24","2031-04-23","2031-05-22","2031-06-21","2031-07-21","2031-08-19","2031-09-18","2031-10-17","2031-11-16","2031-12-15","2032-01-14","2032-02-12","2032-03-13","2032-04-11","2032-05-11","2032-06-09","2032-07-09","2032-08-07","2032-09-06","2032-10-06","2032-11-04","2032-12-04","2033-01-02","2033-02-01","2033-03-02","2033-04-01","2033-04-30","2033-05-29","2033-06-28","2033-07-27","2033-08-26","2033-09-25","2033-10-24","2033-11-23","2033-12-23","2034-01-21","2034-02-20","2034-03-21","2034-04-20","2034-05-19","2034-06-17","2034-07-17","2034-08-15","2034-09-14","2034-10-13","2034-11-12","2034-12-12","2035-01-11","2035-02-09","2035-03-11","2035-04-09","2035-05-09","2035-06-07","2035-07-06","2035-08-05","2035-09-03","2035-10-03","2035-11-01","2035-12-01","2035-12-31","2036-01-30","2036-02-28","2036-03-29","2036-04-27","2036-05-27","2036-06-25","2036-07-24","2036-08-23","2036-09-21","2036-10-20","2036-11-19","2036-12-19","2037-01-18","2037-02-16","2037-03-18","2037-04-17","2037-05-16","2037-06-15","2037-07-14","2037-08-12","2037-09-11","2037-10-10","2037-11-08","2037-12-08","2038-01-07","2038-02-05","2038-03-07","2038-04-06","2038-05-05","2038-06-04","2038-07-04","2038-08-02","2038-08-31","2038-09-30","2038-10-29","2038-11-28","2038-12-27","2039-01-25","2039-02-24","2039-03-26","2039-04-24","2039-05-24","2039-06-23","2039-07-22","2039-08-21","2039-09-19","2039-10-19","2039-11-17","2039-12-17","2040-01-15","2040-02-13","2040-03-14","2040-04-13","2040-05-12","2040-06-11","2040-07-10","2040-08-09","2040-09-08","2040-10-07","2040-11-06","2040-12-05","2041-01-04","2041-02-02","2041-03-04","2041-04-02","2041-05-01","2041-05-31","2041-06-29","2041-07-29","2041-08-28","2041-09-26","2041-10-26","2041-11-25","2041-12-24","2042-01-23","2042-02-21","2042-03-23","2042-04-21","2042-05-20","2042-06-19","2042-07-18","2042-08-17","2042-09-15","2042-10-15","2042-11-14","2042-12-13","2043-01-12","2043-02-11","2043-03-12","2043-04-11","2043-05-10","2043-06-08","2043-07-08","2043-08-06","2043-09-05","2043-10-04","2043-11-03","2043-12-02","2044-01-01","2044-01-31","2044-03-01","2044-03-30","2044-04-29","2044-05-28","2044-06-26","2044-07-26","2044-08-24","2044-09-22","2044-10-22","2044-11-21","2044-12-20","2045-01-19","2045-02-18","2045-03-20","2045-04-18","2045-05-18","2045-06-16","2045-07-15","2045-08-14","2045-09-12","2045-10-11","2045-11-10","2045-12-09","2046-01-08","2046-02-07","2046-03-09","2046-04-07","2046-05-07","2046-06-06","2046-07-05","2046-08-03","2046-09-02","2046-10-01","2046-10-30","2046-11-29","2046-12-28"],"officialUntil":[1449,8]},"ummalqura":{"firstYear":1437,"months":["2015-10-14","2015-11-13","2015-12-12","2016-01-11","2016-02-10","2016-03-10","2016-04-08","2016-05-08","2016-06-06","2016-07-06","2016-08-04","2016-09-02","2016-10-02","2016-11-01","2016-11-30","2016-12-30","2017-01-29","2017-02-28","2017-03-29","2017-04-27","2017-05-27","2017-06-25","2017-07-24","2017-08-23","2017-09-21","2017-10-21","2017-11-19","2017-12-19","2018-01-18","2018-02-17","2018-03-18","2018-04-17","2018-05-16","2018-06-15","2018-07-14","2018-08-

```

```

12", "2018-09-11", "2018-10-10", "2018-11-09", "2018-12-08", "2019-01-07", "2019-02-06", "2019-03-08", "2019-04-
06", "2019-05-06", "2019-06-04", "2019-07-04", "2019-08-02", "2019-08-31", "2019-09-30", "2019-10-29", "2019-11-
28", "2019-12-27", "2020-01-26", "2020-02-25", "2020-03-25", "2020-04-24", "2020-05-24", "2020-06-22", "2020-07-
22", "2020-08-20", "2020-09-18", "2020-10-18", "2020-11-16", "2020-12-16", "2021-01-14", "2021-02-13", "2021-03-
14", "2021-04-13", "2021-05-13", "2021-06-11", "2021-07-11", "2021-08-09", "2021-09-08", "2021-10-07", "2021-11-
06", "2021-12-05", "2022-01-04", "2022-02-02", "2022-03-04", "2022-04-02", "2022-05-02", "2022-05-31", "2022-06-
30", "2022-07-30", "2022-08-28", "2022-09-27", "2022-10-26", "2022-11-25", "2022-12-25", "2023-01-23", "2023-02-
21", "2023-03-23", "2023-04-21", "2023-05-21", "2023-06-19", "2023-07-19", "2023-08-17", "2023-09-16", "2023-10-
16", "2023-11-15", "2023-12-14", "2024-01-13", "2024-02-11", "2024-03-11", "2024-04-10", "2024-05-09", "2024-06-
07", "2024-07-07", "2024-08-05", "2024-09-04", "2024-10-04", "2024-11-03", "2024-12-02", "2025-01-01", "2025-01-
31", "2025-03-01", "2025-03-30", "2025-04-29", "2025-05-28", "2025-06-26", "2025-07-26", "2025-08-24", "2025-09-
23", "2025-10-23", "2025-11-22", "2025-12-21", "2026-01-20", "2026-02-18", "2026-03-20", "2026-04-18", "2026-05-
18", "2026-06-16", "2026-07-15", "2026-08-14", "2026-09-12", "2026-10-12", "2026-11-11", "2026-12-10", "2027-01-
09", "2027-02-08", "2027-03-09", "2027-04-08", "2027-05-07", "2027-06-06", "2027-07-05", "2027-08-03", "2027-09-
02", "2027-10-01", "2027-10-31", "2027-11-29", "2027-12-29", "2028-01-28", "2028-02-26", "2028-03-27", "2028-04-
26", "2028-05-25", "2028-06-24", "2028-07-23", "2028-08-22", "2028-09-20", "2028-10-19", "2028-11-18", "2028-12-
17", "2029-01-16", "2029-02-14", "2029-03-16", "2029-04-15", "2029-05-14", "2029-06-13", "2029-07-13", "2029-08-
12", "2029-09-10", "2029-10-09", "2029-11-08", "2029-12-07", "2030-01-05", "2030-02-04", "2030-03-06", "2030-04-
04", "2030-05-04", "2030-06-03", "2030-07-02", "2030-08-01", "2030-08-31", "2030-09-29", "2030-10-28", "2030-11-
27", "2030-12-26", "2031-01-24", "2031-02-23", "2031-03-24", "2031-04-23", "2031-05-23", "2031-06-21", "2031-07-
21", "2031-08-20", "2031-09-18", "2031-10-18", "2031-11-16", "2031-12-16", "2032-01-14", "2032-02-12", "2032-03-
13", "2032-04-11", "2032-05-11", "2032-06-09", "2032-07-09", "2032-08-08", "2032-09-06", "2032-10-06", "2032-11-
05", "2032-12-04", "2033-01-03", "2033-02-01", "2033-03-03", "2033-04-01", "2033-04-30", "2033-05-30", "2033-06-
28", "2033-07-28", "2033-08-27", "2033-09-25", "2033-10-25", "2033-11-23", "2033-12-23", "2034-01-22", "2034-02-
20", "2034-03-22", "2034-04-20", "2034-05-19", "2034-06-18", "2034-07-17", "2034-08-16", "2034-09-14", "2034-10-
14", "2034-11-12", "2034-12-12", "2035-01-11", "2035-02-10", "2035-03-11", "2035-04-10", "2035-05-09", "2035-06-
07", "2035-07-07", "2035-08-05", "2035-09-03", "2035-10-03", "2035-11-01", "2035-12-01", "2035-12-31", "2036-01-
30", "2036-02-29", "2036-03-29", "2036-04-28", "2036-05-27", "2036-06-25", "2036-07-25", "2036-08-23", "2036-09-
21", "2036-10-21", "2036-11-19", "2036-12-19", "2037-01-18", "2037-02-17", "2037-03-18", "2037-04-17", "2037-05-
17", "2037-06-15", "2037-07-14", "2037-08-13", "2037-09-11", "2037-10-10", "2037-11-09", "2037-12-08", "2038-01-
07", "2038-02-06", "2038-03-07", "2038-04-06", "2038-05-06", "2038-06-04", "2038-07-04", "2038-08-02", "2038-09-
01", "2038-09-30", "2038-10-29", "2038-11-28", "2038-12-27", "2039-01-26", "2039-02-24", "2039-03-26", "2039-04-
25", "2039-05-24", "2039-06-23", "2039-07-22", "2039-08-21", "2039-09-19", "2039-10-19", "2039-11-18", "2039-12-
17", "2040-01-15", "2040-02-14", "2040-03-14", "2040-04-13", "2040-05-12", "2040-06-11", "2040-07-11", "2040-08-
09", "2040-09-08", "2040-10-07", "2040-11-06", "2040-12-06", "2041-01-04", "2041-02-02", "2041-03-04", "2041-04-
02", "2041-05-02", "2041-05-31", "2041-06-30", "2041-07-29", "2041-08-28", "2041-09-27", "2041-10-27", "2041-11-
25", "2041-12-25", "2042-01-23", "2042-02-22", "2042-03-23", "2042-04-21", "2042-05-21", "2042-06-19", "2042-07-
18", "2042-08-17", "2042-09-16", "2042-10-16", "2042-11-14", "2042-12-14", "2043-01-13", "2043-02-11", "2043-03-
13", "2043-04-11", "2043-05-10", "2043-06-09", "2043-07-08", "2043-08-06", "2043-09-05", "2043-10-05", "2043-11-
03", "2043-12-03", "2044-01-02", "2044-02-01", "2044-03-01", "2044-03-31", "2044-04-29", "2044-05-28", "2044-06-
26", "2044-07-26", "2044-08-24", "2044-09-23", "2044-10-23", "2044-11-21", "2044-12-21", "2045-01-20", "2045-02-
18", "2045-03-20", "2045-04-19", "2045-05-18", "2045-06-16", "2045-07-16", "2045-08-14", "2045-09-13", "2045-10-
12", "2045-11-11", "2045-12-10", "2046-01-09", "2046-02-07", "2046-03-09", "2046-04-08", "2046-05-07", "2046-06-
06", "2046-07-05", "2046-08-04", "2046-09-02", "2046-10-02", "2046-10-31", "2046-11-29", "2046-12-29", "2047-01-
27"]}]

```

hijriTable.ts src/domain

- Дата → по Хиджра и обратно, от таблицата.

src/domain/hijriTable.ts

68 реда

```

import data from '@data/hijri.json';

import type { HijriDate } from './hijri';

/**
 * Календарът по Хиджра от таблица (етап 13) - началото на всеки месец, а не изчисление
 * „горе-долу“. Таблицата е от tools/build-hijri.mjs:
 * - 'diyanet' - Диянет; същия календар ползва и Главно мюфтийство в България;

```

```

* - 'ummalqura' - Умм ал-Кура (Саудитска Арабия). Там Рамазан, Шеввал и Зилхидже
* се обявяват по наблюдение на луната - таблицата е само очакване.
*/
export type HijriCalendar = 'diyanet' | 'ummalqura';

interface Table {
  firstYear: number;
  /** Дните от 1.1.1970 (UTC) на 1-во число на всеки месец, от Мухаррем на firstYear. */
  days: number[];
}

const DAY = 86400000;
const epochDay = (iso: string) => {
  const [y, m, d] = iso.split('-').map(Number);
  return Math.round(Date.UTC(y, m - 1, d) / DAY);
};

const TABLES: Record<HijriCalendar, Table> = {
  diyanet: { firstYear: data.diyanet.firstYear, days: data.diyanet.months.map(epochDay) },
  ummalqura: { firstYear: data.ummalqura.firstYear, days: data.ummalqura.months.map(epochDay) },
};

/** До кой месец Диянет е обявил датите официално; след него - по същите правила. */
export const DIYANET_OFFICIAL_UNTIL = { year: data.diyanet.officialUntil[0], month:
data.diyanet.officialUntil[1] };

/** Денят (местна дата) като номер от 1.1.1970. */
export function localEpochDay(date: Date): number {
  return Math.round(Date.UTC(date.getFullYear(), date.getMonth(), date.getDate()) / DAY);
}

/** Датата по Хиджра от таблицата; null - извън таблицата. */
export function hijriFromDate(date: Date, calendar: HijriCalendar): HijriDate | null {
  const { firstYear, days } = TABLES[calendar];
  const day = localEpochDay(date);
  // последният месец няма известен край - не го ползваме
  if (day < days[0] || day >= days[days.length - 1]) return null;
  let lo = 0;
  let hi = days.length - 1;
  while (hi - lo > 1) {
    const mid = (lo + hi) >> 1;
    if (days[mid] <= day) lo = mid;
    else hi = mid;
  }
  return { year: firstYear + Math.floor(lo / 12), month: (lo % 12) + 1, day: day - days[lo] + 1 };
}

/** Григорианската дата (местна, 12:00) на ден по Хиджра; null - извън таблицата. */
export function gregorianFromDate(calendar: HijriCalendar, year: number, month: number, day: number): Date
| null {
  const { firstYear, days } = TABLES[calendar];
  const i = (year - firstYear) * 12 + (month - 1);
  if (i < 0 || i >= days.length - 1) return null;
  const d = new Date((days[i] + day - 1) * DAY);
  return new Date(d.getUTCFullYear(), d.getUTCMonth(), d.getUTCDate(), 12);
}

/** Годишите по Хиджра, които таблицата покрива изцяло. */
export function tableYears(calendar: HijriCalendar): { first: number; last: number } {
  const { firstYear, days } = TABLES[calendar];

```

```
    return { first: firstYear, last: firstYear + Math.floor((days.length - 1) / 12) - 1 };
  }
```

hijri.ts src/domain

- Датата по Хиджра в „Днес“ и „Месец“ – първо от таблицата (досега беше Умм ал-Кура и можеше да се различава с 1 ден от Мюфтийството). Корекцията „Дата по Хиджра“ в Настройки остава.

src/domain/hijri.ts

110 реда

```
import { hijriFromTable, type HijriCalendar } from './hijriTable';

/**
 * Дата по Хиджра.
 * 0) Етап 13: от таблицата на календара (src/domain/hijriTable.ts) – Диянет / Мюфтийството
 * или Умм ал-Кура – точно като обявените дати. Изчисленията по-долу са само резерва
 * извън годините на таблицата.
 * 1) Календарът Umm al-Qura през Intl (ако телефонът го поддържа).
 * 2) Иначе табличният („кувейтски“) алгоритъм, който е чисто аритметичен.
 * И двата могат да се разминат с 1-2 дни с датата, обявена от Мюфтийството
 * (тя зависи от наблюдението на луната). Затова има `adjustDays` –
 * в етап 2 става настройка „Корекция на датата по Хиджра“.
 * Имената на месеците са в src/i18n/strings.ts (hijriMonths).
 */

export interface HijriDate {
  day: number;
  month: number; // 1..12
  year: number;
}

function julianDayNumber(year: number, month: number, day: number): number {
  const a = Math.floor((14 - month) / 12);
  const y = year + 4800 - a;
  const m = month + 12 * a - 3;
  return (
    day +
    Math.floor((153 * m + 2) / 5) +
    365 * y +
    Math.floor(y / 4) -
    Math.floor(y / 100) +
    Math.floor(y / 400) -
    32045
  );
}

// Един форматер за цялото приложение: в Hermes създаването на Intl.DateTimeFormat е скъпо
// (за „Месец“ бяха 30 нови форматера при всяко показване). null = телефонът не го поддържа.
let formatter: Intl.DateTimeFormat | null | undefined;
function umalquraFormatter(): Intl.DateTimeFormat | null {
  if (formatter === undefined) {
    try {
      formatter = new Intl.DateTimeFormat('en-u-ca-islamic-umalqura', {
        day: 'numeric',
        month: 'numeric',
        year: 'numeric',
      });
    } catch {
      formatter = null;
    }
  }
}
```

```

    }
    return formatter;
}

function fromIntl(date: Date): HijriDate | null {
    const f = umalquraFormatter();
    if (!f) return null;
    try {
        const parts = f.formatToParts(date);
        const num = (type: string) => parseInt(parts.find((p) => p.type === type)?.value ?? '', 10);
        const day = num('day');
        const month = num('month');
        const year = num('year');
        // проверка, че наистина е върнат ислямски календар, а не григориански
        if (!(day >= 1 && day <= 30 && month >= 1 && month <= 12 && year > 1400 && year < 1600)) {
            return null;
        }
        return { day, month, year };
    } catch {
        return null;
    }
}

/** Табличният алгоритъм (експортиран за тестовите). */
export function tabularHijri(date: Date): HijriDate {
    const jd = julianDayNumber(date.getFullYear(), date.getMonth() + 1, date.getDate());

    let l = jd - 1948440 + 10632;
    const n = Math.floor((l - 1) / 10631);
    l = l - 10631 * n + 354;
    const j =
        Math.floor((10985 - l) / 5316) * Math.floor((50 * l) / 17719) +
        Math.floor(l / 5670) * Math.floor((43 * l) / 15238);
    l =
        l -
        Math.floor((30 - j) / 15) * Math.floor((17719 * j) / 50) -
        Math.floor(j / 16) * Math.floor((15238 * j) / 43) +
        29;
    const month = Math.floor((24 * l) / 709);
    const day = l - Math.floor((709 * month) / 24);
    const year = 30 * n + j - 30;

    return { day, month, year };
}

export function toHijri(date: Date, adjustDays = 0, calendar: HijriCalendar = 'diyanet'): HijriDate {
    const d = new Date(date.getFullYear(), date.getMonth(), date.getDate() + adjustDays, 12);
    return hijriFromTable(d, calendar) ?? fromIntl(d) ?? tabularHijri(d);
}

/** „17 Ребиул-ахир 1448“ или „17 Rabi al-Thani 1448“ */
export function formatHijri(
    date: Date,
    monthNames: readonly string[],
    adjustDays = 0,
    calendar: HijriCalendar = 'diyanet',
): string {
    const h = toHijri(date, adjustDays, calendar);
    return `${h.day} ${monthNames[h.month - 1]} ${h.year}`;
}

```

holidays.ts src/domain

- Празниците за година, следващият празник, „Автоматично“ по държавата, денят на напомнянето.

src/domain/holidays.ts

181 реда

```
import { gregorianFromTable, localEpochDay, type HijriCalendar } from './hijriTable';

/**
 * Ислямските празници и свещени нощи (етап 13).
 *
 * Датите са от таблицата на календара по Хиджра (src/domain/hijriTable.ts) – за България
 * това е календарът на Мюфтийството (= Диянет), обявен предварително. Свещените нощи
 * (кандили) са вечерта на посочения ден – нощта преди следващия ден по Хиджра.
 */

export type HolidayId =
  | 'threeMonths'
  | 'regaib'
  | 'miraj'
  | 'berat'
  | 'ramadan'
  | 'qadr'
  | 'fitrEve'
  | 'fitr'
  | 'arafah'
  | 'adha'
  | 'newYear'
  | 'ashura'
  | 'mawlid';

export const HOLIDAY_IDS: readonly HolidayId[] = [
  'threeMonths',
  'regaib',
  'miraj',
  'berat',
  'ramadan',
  'qadr',
  'fitrEve',
  'fitr',
  'arafah',
  'adha',
  'newYear',
  'ashura',
  'mawlid',
];

/** Чии дати: Мюфтийството и Диянет са един и същ календар; Умм ал-Кура – Саудитска Арабия. */
export type HolidaySource = 'mufti' | 'diyanet' | 'ummalqura';
export type HolidaySourceChoice = 'auto' | HolidaySource;

/** Страни, които следват обявяванията на Саудитска Арабия. */
const UMM_AL_QURA_COUNTRIES = new Set(['SA', 'AE', 'KW', 'QA', 'BH', 'YE']);

/** „Автоматично“: България → Мюфтийството; Саудитска Арабия и Залива → Умм ал-Кура; другаде → Диянет. */
export function resolveHolidaySource(choice: HolidaySourceChoice, country: string | null): HolidaySource {
  if (choice !== 'auto') return choice;
  if (country === 'BG') return 'mufti';
  if (country && UMM_AL_QURA_COUNTRIES.has(country)) return 'ummalqura';
  return 'diyanet';
}
```

```

export function calendarOf(source: HolidaySource): HijriCalendar {
    return source === 'ummalqura' ? 'ummalqura' : 'diyanet';
}

export interface HolidayMeta {
    arabic: string;
    /** Свещена нощ - вечерта на датата. */
    night: boolean;
    /** Колко дни (Байрамите). */
    days: number;
    /** Месецът по Хиджра - за Умм ал-Кура Рамазан, Шеввал и Зилхидже са „очаквани“. */
    month: number;
}

export const HOLIDAYS: Record<HolidayId, HolidayMeta> = {
    threeMonths: { arabic: 'رجب', night: false, days: 1, month: 7 },
    regaib: { arabic: 'ليلة الرغائب', night: true, days: 1, month: 7 },
    miraj: { arabic: 'ليلة المعراج', night: true, days: 1, month: 7 },
    berat: { arabic: 'ليلة البراءة', night: true, days: 1, month: 8 },
    ramadan: { arabic: 'رمضان', night: false, days: 1, month: 9 },
    qadr: { arabic: 'ليلة القدر', night: true, days: 1, month: 9 },
    fitrEve: { arabic: 'عشية العيد', night: false, days: 1, month: 9 },
    fitr: { arabic: 'عيد الفطر', night: false, days: 3, month: 10 },
    arafah: { arabic: 'يوم عرفة', night: false, days: 1, month: 12 },
    adha: { arabic: 'عيد الأضحي', night: false, days: 4, month: 12 },
    newYear: { arabic: 'رأس السنة الهجرية', night: false, days: 1, month: 1 },
    ashura: { arabic: 'عاشوراء', night: false, days: 1, month: 1 },
    mawlid: { arabic: 'المولد النبوي', night: true, days: 1, month: 3 },
};

export interface Holiday {
    id: HolidayId;
    /** Първият ден (местна дата, 12:00). При свещена нощ - денят, чиято вечер е нощта. */
    date: Date;
    /** Последният ден (= date при еднодневните). */
    end: Date;
    /** Датата по Хиджра на първия ден. */
    hijri: { day: number; month: number; year: number };
    night: boolean;
    /** Умм ал-Кура: началото на Рамазан, Шеввал и Зилхидже се обявява по наблюдение. */
    expected: boolean;
}

const addDays = (d: Date, n: number) => new Date(d.getFullYear(), d.getMonth(), d.getDate() + n, 12);

/** Празниците на една година по Хиджра. Липсва в таблицата → празен списък. */
export function holidaysOfHijriYear(source: HolidaySource, year: number): Holiday[] {
    const cal = calendarOf(source);
    const g = (month: number, day: number) => gregorianFromTable(cal, year, month, day);
    const out: Holiday[] = [];
    const add = (id: HolidayId, date: Date | null, hijri: { day: number; month: number }) => {
        if (!date) return;
        const meta = HOLIDAYS[id];
        out.push({
            id,
            date,
            end: addDays(date, meta.days - 1),
            hijri: { ...hijri, year },
            night: meta.night,
            expected: source === 'ummalqura' && [9, 10, 12].includes(meta.month),
        });
    };
}

```

```

    });
};

const rajab1 = g(7, 1);
add('threeMonths', rajab1, { month: 7, day: 1 });
if (rajab1) {
    // Регаиб: нощта срещу първия петък на Реджеб – четвъртък вечер
    const firstFriday = addDays(rajab1, (5 - rajab1.getDay() + 7) % 7);
    const regaib = addDays(firstFriday, -1);
    const day = localEpochDay(regaib) - localEpochDay(rajab1) + 1;
    // ако 1 Реджеб е петък, четвъртъкът е последният ден на Джемазиел-ахир
    const jumada1 = g(6, 1);
    add(
        'regaib',
        regaib,
        day >= 1 || !jumada1 ? { month: 7, day: Math.max(1, day) } : { month: 6, day: localEpochDay(regaib) -
localEpochDay(jumada1) + 1 },
    );
}
add('miraj', g(7, 26), { month: 7, day: 26 });
add('berat', g(8, 14), { month: 8, day: 14 });
add('ramadan', g(9, 1), { month: 9, day: 1 });
add('qadr', g(9, 26), { month: 9, day: 26 });
const shawwal1 = g(10, 1);
if (shawwal1) {
    const eve = addDays(shawwal1, -1);
    const ram1 = g(9, 1);
    add('fitrEve', eve, { month: 9, day: ram1 ? localEpochDay(eve) - localEpochDay(ram1) + 1 : 29 });
}
add('fitr', shawwal1, { month: 10, day: 1 });
add('arafah', g(12, 9), { month: 12, day: 9 });
add('adha', g(12, 10), { month: 12, day: 10 });
add('newYear', g(1, 1), { month: 1, day: 1 });
add('ashura', g(1, 10), { month: 1, day: 10 });
add('mawlid', g(3, 11), { month: 3, day: 11 });
return out;
}

/** Празниците с първи ден в дадената година (по Григорианския календар), по ред. */
export function holidaysInYear(source: HolidaySource, year: number): Holiday[] {
    // една година по Григорианския календар засяга 2 години по Хиджра (понякога 3)
    const approx = Math.floor(((year - 622) * 33) / 32);
    const all: Holiday[] = [];
    for (let hy = approx - 1; hy <= approx + 2; hy++) all.push(...holidaysOfHijriYear(source, hy));
    return all.filter((h) => h.date.getFullYear() === year).sort((a, b) => a.date.getTime() -
b.date.getTime());
}

/** Следващият празник от днес нататък (днешният също – докато не е минал). */
export function nextHoliday(source: HolidaySource, now: Date): Holiday | null {
    const today = localEpochDay(now);
    for (const y of [now.getFullYear(), now.getFullYear() + 1]) {
        const h = holidaysInYear(source, y).find((x) => localEpochDay(x.end) >= today);
        if (h) return h;
    }
    return null;
}

/** След колко дни (0 – днес). */
export function daysUntil(h: Holiday, now: Date): number {
    return localEpochDay(h.date) - localEpochDay(now);
}

```

```

}

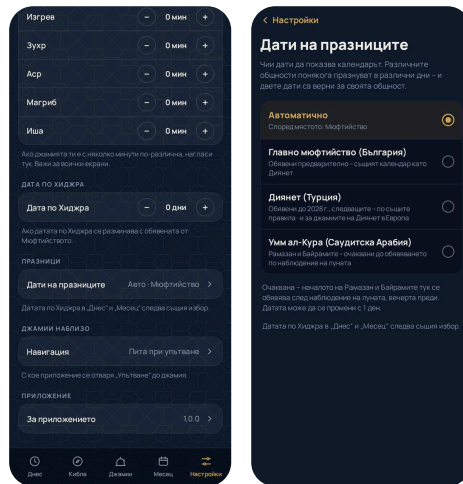
/** Денят на напомнянето: свещена нощ – същата вечер; празник – вечерта преди. */
export function reminderDayOf(h: Holiday): Date {
  return h.night ? h.date : addDays(h.date, -1);
}

```

Стъпка 4

Чии дати – Настройки

Нова група „Празници“ → „Дати на празниците“.



Настройки → Празници · изборът

- **Автоматично** (по подразбиране): България → Мюфтийството; Саудитска Арабия и Залива → Умм ал-Кура; другаде → Диянет.
- Или изрично: Главно мюфтийство, Диянет, Умм ал-Кура.
- Същият избор определя датата по Хиджра в „Днес“ и „Месец“ – така навсякъде е една и съща дата.

holiday-source.tsx src/app

- Екранът за избор.

src/app/holiday-source.tsx

112 реда

```

import { router } from 'expo-router';
import { useRef } from 'react';
import { Pressable, ScrollView, StyleSheet, Text, View } from 'react-native';
import { useSafeAreaInsets } from 'react-native-safe-area-context';

import { GeometricPattern } from '@components/GeometricPattern';
import { ChevronIcon } from '@components/icons';
import { resolveHolidaySource, type HolidaySourceChoice } from '@domain/holidays';
import { countryOf } from '@domain/resolve';
import { useI18n } from '@i18n';
import { selectLocation, useSettings } from '@store/settings';
import { colors } from '@theme/colors';
import { fonts } from '@theme/typography';

const CHOICES: readonly HolidaySourceChoice[] = ['auto', 'mufti', 'diyanet', 'ummalqura'];

```

```

/**
 * Настройки → Дати на празниците (етап 13): чии дати показва календарът на празниците.
 * Същият избор определя и датата по Хиджра в „Днес“ и „Месец“.
 */
export default function HolidaySourceScreen() {
  const insets = useSafeAreaInsets();
  const { t } = useI18n();
  const n = t.holidaySource;
  const choice = useSettings((s) => s.holidaySource ?? 'auto');
  const setChoice = useSettings((s) => s.setHolidaySource);
  const location = useSettings(selectLocation);
  const autoSource = resolveHolidaySource('auto', countryOf(location));

  const closing = useRef(false);
  const back = () => {
    if (closing.current) return;
    closing.current = true;
    if (router.canGoBack()) router.back();
    else router.replace('/settings');
  };

  const label = (c: HolidaySourceChoice) => (c === 'auto' ? n.auto : n[c]);
  const desc = (c: HolidaySourceChoice) =>
    c === 'auto' ? n.autoDesc(n.short[autoSource]) : c === 'mufti' ? n.muftiDesc : c === 'diyanet' ?
    n.diyanetDesc : n.ummAlquraDesc;

  return (
    <View style={styles.root}>
      <GeometricPattern />
      <View style={[styles.header, { paddingTop: insets.top + 8 }]}>
        <Pressable onPress={back} hitSlop={10} accessibilityRole="button" style={styles.backBtn}>
          <ChevronIcon direction="left" size={18} color={colors.gold} />
          <Text style={styles.backText}>{n.back}</Text>
        </Pressable>
      </View>
      <ScrollView contentContainerStyle={[styles.content, { paddingBottom: insets.bottom + 40 }]}
        showsVerticalScrollIndicator={false}>
        <Text style={styles.title}>{n.title}</Text>
        <Text style={styles.intro}>{n.intro}</Text>

        <View style={styles.card} accessibilityRole="radiogroup" accessibilityLabel={n.title}>
          {CHOICES.map((c, i) => {
            const on = c === choice;
            return (
              <Pressable
                key={c}
                onPress={() => setChoice(c)}
                accessibilityRole="radio"
                accessibilityState={{ checked: on }}
                style={({ pressed }) => [styles.row, i > 0 && styles.border, on && styles.rowOn, pressed &&
styles.pressed]}
              >
                <View style={styles.text}>
                  <Text style={[styles.name, on && styles.nameOn]}>{label(c)}</Text>
                  <Text style={styles.desc}>{desc(c)}</Text>
                </View>
                <View style={[styles.radio, on && styles.radioOn]}>{on && <View style={styles.dot} />}
              </View>
            );
          })}
        </View>
      </ScrollView>
    </View>
  );
}

```

```

    </View>

    <Text style={styles.note}>{t.holidays.expectedNote}</Text>
    <Text style={styles.note}>{n.note}</Text>
  </ScrollView>
</View>
);
}

const styles = StyleSheet.create({
  root: { flex: 1, backgroundColor: colors.base },
  header: { paddingHorizontal: 16, paddingBottom: 4 },
  backBtn: { flexDirection: 'row', alignItems: 'center', gap: 4, paddingVertical: 6, alignSelf: 'flex-start' },
  backText: { fontFamily: fonts.bold, fontSize: 15, color: colors.gold },
  content: { paddingHorizontal: 16, gap: 8 },
  title: { fontFamily: fonts.extrabold, fontSize: 26, color: colors.text, marginTop: 4 },
  intro: { fontFamily: fonts.regular, fontSize: 13.5, lineHeight: 19, color: colors.muted, marginBottom: 8 },
},
  card: { backgroundColor: colors.card, borderWidth: 1, borderColor: colors.cardBorder, borderRadius: 18,
overflow: 'hidden' },
  row: { flexDirection: 'row', alignItems: 'center', gap: 12, paddingHorizontal: 14, paddingVertical: 11,
minHeight: 66 },
  rowOn: { backgroundColor: 'rgba(212,168,87,0.10)' },
  border: { borderTopWidth: StyleSheet.hairlineWidth, borderTopColor: 'rgba(255,255,255,0.10)' },
  pressed: { opacity: 0.7 },
  text: { flex: 1, gap: 2 },
  name: { fontFamily: fonts.bold, fontSize: 15.5, color: colors.text },
  nameOn: { fontFamily: fonts.extrabold, color: colors.gold },
  desc: { fontFamily: fonts.regular, fontSize: 12, lineHeight: 16, color: colors.muted },
  radio: {
    width: 22,
    height: 22,
    borderRadius: 11,
    borderWidth: 2,
    borderColor: 'rgba(242,239,232,0.35)',
    alignItems: 'center',
    justifyContent: 'center',
  },
  radioOn: { borderColor: colors.gold },
  dot: { width: 10, height: 10, borderRadius: 5, backgroundColor: colors.gold },
  note: { fontFamily: fonts.regular, fontSize: 12.5, lineHeight: 18, color: colors.muted, marginTop: 6,
paddingHorizontal: 4 },
});

```

settings.ts src/store

- `holidaySource`, `holidayReminders` (за всеки празник поотделно); `selectHolidaySource`, `selectHijriCalendar`.

src/store/settings.ts

238 реда

```

import AsyncStorage from '@react-native-async-storage/async-storage';
import { create } from 'zustand';
import { createJSONStorage, persist } from 'zustand/middleware';

import {
  DEFAULT_HIGH_LAT_RULE,
  DEFAULT_LOCATION,
  DEFAULT_MADHAB,

```

```

    DEFAULT_METHOD,
} from '@config/defaults';
import {
    methodForCountry,
    type AsrMadhab,
    type CalculationMethodId,
    type HighLatRuleId,
    type MethodChoice,
} from '@domain/calc';
import type { AppLocation } from '@domain/location';
import { resolveHolidaySource, type HolidayId, type HolidaySource, type HolidaySourceChoice } from '@domain/holidays';
import type { HijriCalendar } from '@domain/hijriTable';
import type { NavAppId } from '@domain/navApps';
import { getPlace, PLACES_COUNT, placeToLocation } from '@domain/places';
import { countryOf } from '@domain/resolve';
import type { PrayerId } from '@domain/prayers';
import type { TimesOptions } from '@domain/times';

/** Граници на ръчните корекции. */
export const OFFSET_LIMIT = 15; // ±15 мин. за молитва
export const HIJRI_LIMIT = 2; // ±2 дни
/** Възможните напомняния преди молитвата (минути); 0 = без напомняне. */
export const REMINDER_CHOICES = [0, 5, 10, 15, 20, 30] as const;

const NO_OFFSETS: Record<PrayerId, number> = {
    fajr: 0,
    sunrise: 0,
    dhuhr: 0,
    asr: 0,
    maghrib: 0,
    isha: 0,
};

const clamp = (v: number, limit: number) => Math.max(-limit, Math.min(limit, v));

interface SettingsState {
    /** true - мястото идва от GPS; false - избрано ръчно. */
    autoLocation: boolean;
    /** Последното място от GPS (пази се, за да работи веднага при следващо отваряне и без GPS). */
    gpsLocation: AppLocation | null;
    /** Ръчно избраното място. */
    manualLocation: AppLocation | null;

    /** 'auto' - според държавата (виж methodForCountry). */
    method: MethodChoice;
    madhab: AsrMadhab;
    highLatRule: HighLatRuleId;
    offsets: Record<PrayerId, number>;
    hijriAdjust: number;
    /** Напомняне X минути преди молитвата; 0 = без. */
    reminderMinutes: number;
    /** Вибрация при известие (Android; на iPhone решава телефонът). */
    vibrate: boolean;
    /** С кое приложение се отваря „Упътване“ до джамия (етап 12). */
    navApp: NavAppId;
    /** Изборът е запомнен (листа „Отвори с...“ или Настройки → Навигация); false - питаме при „Упътване“. */
    navRemembered: boolean;
    /** Чии дати на празниците (етап 13): „Автоматично“ - според мястото. */
    holidaySource: HolidaySourceChoice;
    /** Напомняне за всеки празник поотделно (етап 13). */

```

```

holidayReminders: Partial<Record<HolidayId, boolean>>;

setAutoLocation: (on: boolean) => void;
setGpsLocation: (loc: AppLocation) => void;
chooseLocation: (loc: AppLocation) => void;
setMethod: (m: MethodChoice) => void;
setMadhab: (m: AsrMadhab) => void;
setHighLatRule: (r: HighLatRuleId) => void;
changeOffset: (id: PrayerId, delta: number) => void;
resetOffsets: () => void;
changeHijriAdjust: (delta: number) => void;
setReminderMinutes: (n: number) => void;
setVibrate: (on: boolean) => void;
/** Запомня приложението за навигация - остава и след затваряне на приложението и рестарт. */
setNavApp: (id: NavAppId) => void;
/** Забравя избора (избраното приложение е изтрито) - следващото „Упътване“ пак пита. */
forgetNavApp: () => void;
setHolidaySource: (choice: HolidaySourceChoice) => void;
setHolidayReminder: (id: HolidayId, on: boolean) => void;
}

export const useSettings = create<SettingsState>()(
  persist(
    (set) => ({
      autoLocation: true,
      gpsLocation: null,
      manualLocation: null,
      method: DEFAULT_METHOD,
      madhab: DEFAULT_MADHAB,
      highLatRule: DEFAULT_HIGH_LAT_RULE,
      offsets: NO_OFFSETS,
      hijriAdjust: 0,
      reminderMinutes: 0,
      vibrate: true,
      navApp: 'auto',
      navRemembered: false,
      holidaySource: 'auto',
      holidayReminders: {},

      setAutoLocation: (on) => set({ autoLocation: on }),
      setGpsLocation: (loc) => set({ gpsLocation: loc }),
      // ръчният избор изключва GPS, за да не го „презapiше“ следващото определяне
      chooseLocation: (loc) => set({ manualLocation: loc, autoLocation: false }),
      setMethod: (method) => set({ method }),
      setMadhab: (madhab) => set({ madhab }),
      setHighLatRule: (highLatRule) => set({ highLatRule }),
      changeOffset: (id, delta) =>
        set((s) => ({ offsets: { ...s.offsets, [id]: clamp(s.offsets[id] + delta, OFFSET_LIMIT) } })),
      resetOffsets: () => set({ offsets: NO_OFFSETS }),
      changeHijriAdjust: (delta) =>
        set((s) => ({ hijriAdjust: clamp(s.hijriAdjust + delta, HIJRI_LIMIT) })),
      setReminderMinutes: (reminderMinutes) => set({ reminderMinutes }),
      setVibrate: (vibrate) => set({ vibrate }),
      setNavApp: (navApp) => set({ navApp, navRemembered: true }),
      forgetNavApp: () => set({ navApp: 'auto', navRemembered: false }),
      setHolidaySource: (holidaySource) => set({ holidaySource }),
      setHolidayReminder: (id, on) => set((s) => ({ holidayReminders: { ...s.holidayReminders, [id]: on }
    })),
  ),
  {
    name: 'ezan.settings',
  }
)

```

```

// 2 (етап 5): методът по подразбиране е „Автоматично“ – старият „Диянет“ по подразбиране
// става „Автоматично“ (за Турция и Европа пак е Диянет)
// 3 (етап 10): турски език – запазените места в България получават и турското име
version: 3,
migrate: (persisted, version) => {
  const state = persisted as Partial<SettingsState>;
  if (version < 2 && state.method === 'Turkey') state.method = 'auto';
  if (version < 3) {
    state.manualLocation = withTurkishName(state.manualLocation);
    state.gpsLocation = withTurkishName(state.gpsLocation);
  }
  return state as SettingsState;
},
storage: createJSONStorage(() => AsyncStorage),
// новите полета (reminderMinutes, vibrate, navApp, navRemembered) липсват в старите записи – тогава
// важат стойностите по подразбиране
partialize: ({
  autoLocation,
  gpsLocation,
  manualLocation,
  method,
  madhab,
  highLatRule,
  offsets,
  hijriAdjust,
  reminderMinutes,
  vibrate,
  navApp,
  navRemembered,
  holidaySource,
  holidayReminders,
}) => ({
  holidaySource,
  holidayReminders,
  reminderMinutes,
  vibrate,
  navApp,
  navRemembered,
  autoLocation,
  gpsLocation,
  manualLocation,
  method,
  madhab,
  highLatRule,
  offsets,
  hijriAdjust,
}),
},
),
);

/** Мястото, за което се показват часовете в момента. */
export function selectLocation(s: SettingsState): AppLocation {
  const chosen = s.autoLocation ? s.gpsLocation : s.manualLocation;
  return chosen ?? s.gpsLocation ?? DEFAULT_LOCATION;
}

type TimesParts = Pick<SettingsState, 'method' | 'madhab' | 'highLatRule' | 'offsets'> & { location:
AppLocation };

/** Опциите за часовете и техният „отпечатък“ `key` – по него се решава кога да се преизчисли. */

```

```

export function buildTimesOptions({ location, method, madhab, highLatRule, offsets }: TimesParts): {
  options: TimesOptions;
  key: string;
} {
  const options: TimesOptions = { location, method: resolveMethod(method, location), madhab, highLatRule,
offsets };
  const key = JSON.stringify([
    location.id,
    location.latitude,
    location.longitude,
    location.source,
    location.muftiShift,
    options.method,
    madhab,
    highLatRule,
    offsets,
  ]);
  return { options, key };
}

/** „Автоматично“ → методът за държавата на мястото; иначе избраният. */
export function resolveMethod(method: MethodChoice, location: AppLocation): CalculationMethodId {
  return method === 'auto' ? methodForCountry(countryOf(location)) : method;
}

/** Същото, направо от състоянието – без React (ползва се и от известията във фонов режим). */
export function selectTimesOptions(s: SettingsState): { options: TimesOptions; key: string } {
  return buildTimesOptions({ ...s, location: selectLocation(s) });
}

/** Място в България, запазено преди етап 10: наново от списъка – с турското име. */
function withTurkishName<T extends AppLocation | null | undefined>(loc: T): T {
  if (!loc || !loc.id.startsWith('bg-')) return loc;
  const index = Number(loc.id.slice(3));
  if (!Number.isInteger(index) || index < 0 || index >= PLACES_COUNT) return loc;
  try {
    const fresh = placeToLocation(getPlace(index));
    return fresh.names.bg === loc.names.bg ? ({ ...loc, names: fresh.names, detail: fresh.detail } as T) :
loc;
  } catch {
    return loc;
  }
}

/** Чии дати на празниците важат сега (етап 13) – „Автоматично“ според държавата на мястото. */
export function selectHolidaySource(s: SettingsState): HolidaySource {
  return resolveHolidaySource(s.holidaySource ?? 'auto', countryOf(selectLocation(s)));
}

/** Календарът по Хиджра за датите в приложението – същият като за празниците. */
export function selectHijriCalendar(s: SettingsState): HijriCalendar {
  return selectHolidaySource(s) === 'ummalkura' ? 'ummalkura' : 'diyanet';
}

```

settings.tsx src/app/(tabs)

- Групата „Празници“.

src/app/(tabs)/settings.tsx

290 реда

```
import Constants from 'expo-constants';
import { router } from 'expo-router';
import { useState } from 'react';
import { ActivityIndicator, Linking, Pressable, ScrollView, StyleSheet, Text, View } from 'react-native';
import { useSafeAreaInsets } from 'react-native-safe-area-context';

import { GeometricPattern } from '@components/GeometricPattern';
import { BatteryWarning } from '@components/settings/BatteryWarning';
import { NotificationsGroup } from '@components/settings/NotificationsGroup';
import { SoundsGroup } from '@components/settings/SoundsGroup';
import { PinIcon } from '@components/icons';
import { navAppName } from '@components/mosques/navLabel';
import {
  Choice,
  Group,
  NavRow,
  Row,
  StepperRow,
  SwitchRow,
  settingsStyles,
} from '@components/settings/controls';
import { useTabBarHeight } from '@components/TabBar';
import { CALC_METHODS, HIGH_LAT_RULES, isHighLatitude, isPolar, methodForCountry, type MethodChoice } from '@domain/calc';
import { PRAYER_IDS } from '@domain/prayers';
import { countryOf } from '@domain/resolve';
import { countryName } from '@domain/worldCities';
import { useI18n } from '@i18n';
import { refreshLocation, useLocationStatus } from '@services/location';
import { selectHolidaySource, selectLocation, useSettings } from '@store/settings';
import { colors } from '@theme/colors';
import { fonts } from '@theme/typography';

const APP_VERSION = Constants.expoConfig?.version ?? '';

export default function SettingsScreen() {
  const insets = useSafeAreaInsets();
  const tabBarHeight = useTabBarHeight();
  const { lang, t, pick } = useI18n();
  const s = useSettings();
  const location = selectLocation(s);
  const status = useLocationStatus((x) => x.status);
  const [open, setOpen] = useState<'method' | 'asr' | 'highLat' | null>(null);

  const isCalc = location.source === 'calc';
  // „Автоматично“: държавата на мястото и методът за нея
  const country = isCalc ? countryOf(location) : null;
  const autoMethod = methodForCountry(country);
  const highLat = isCalc && isHighLatitude(location.latitude);
  const anyOffset = PRAYER_IDS.some((id) => s.offsets[id] !== 0);
  const toggle = (what: 'method' | 'asr' | 'highLat') => setOpen((cur) => (cur === what ? null : what));

  return (
    <View style={styles.root}>
      <GeometricPattern />
    </View>
  );
}
```

```

{/* списъкът свършва над долната навигация, за да не се застъпва с нея */}
<ScrollView
  style={{ marginBottom: tabBarHeight }}
  contentContainerStyle={[styles.content, { paddingTop: insets.top + 8, paddingBottom: 24 }]}
  showsVerticalScrollIndicator={false}
>
  <Text style={styles.title}>{t.settings.title}</Text>
  <BatteryWarning />

  {/* ---- Място ---- */}
  <Group label={t.settings.groupPlace}>
    <Pressable
      onPress={() => router.push('/place')}
      accessibilityRole="button"
      accessibilityLabel={` ${pick(location.names)}. ${t.settings.change}`}
      style={({ pressed }) => [styles.placeRow, pressed && styles.pressed]}
    >
      <PinIcon size={18} color={colors.gold} />
      <View style={styles.placeText}>
        <Text style={styles.placeName}>{pick(location.names)}</Text>
        {location.detail ? <Text style={styles.placeDetail}>{pick(location.detail)}</Text> : null}
        <Text style={styles.badge}>{isCalc ? t.settings.sourceCalc : t.settings.sourceMufti}</Text>
      </View>
      <Text style={styles.link}>{t.settings.change}</Text>
    </Pressable>
    <SwitchRow label={t.settings.auto} value={s.autoLocation} onChange={s.setAutoLocation} />
    {s.autoLocation && (
      <Row>
        <View style={styles.statusBox}>
          <View style={styles.statusLine}>
            {status === 'locating' && <ActivityIndicator size="small" color={colors.gold} />}
            <Text style={styles.statusText, status === 'locating' && styles.statusBusy}>
              {status === 'locating'
                ? t.settings.locating
                : status === 'denied'
                  ? t.settings.denied
                  : status === 'unavailable'
                    ? t.settings.unavailable
                    : t.settings.autoHint}
            </Text>
          </View>
          {status === 'denied' ? (
            <Pressable onPress={() => Linking.openSettings()} accessibilityRole="button">
              <Text style={styles.link}>{t.settings.openSettings}</Text>
            </Pressable>
          ) : status !== 'locating' ? (
            <Pressable onPress={() => refreshLocation()} accessibilityRole="button">
              <Text style={styles.link}>{t.settings.refresh}</Text>
            </Pressable>
          ) : null}
        </View>
      </Row>
    )}
  </Group>

  {/* ---- Известия ---- */}
  <NotificationsGroup />

  {/* ---- Звуци (етап 6) ---- */}
  <SoundsGroup />

```

```

{ /* ---- Чacobe ---- */ }
{ isCalc ? (
  <Group
    label={t.settings.groupTimes}
    note={
      isPolar(location.latitude)
        ? t.settings.polarNote(Math.abs(location.latitude).toFixed(0))
        : highLat
        ? t.settings.highLatNote(Math.abs(location.latitude).toFixed(0))
        : t.settings.asrNote
    }
  >
    <NavRow
      first
      label={t.settings.method}
      value={s.method === 'auto' ? t.settings.methodAutoValue(t.methods[autoMethod]) :
t.methods[s.method]}
      onPress={() => toggle('method')}
    />
    {open === 'method' && (
      <Choice<MethodChoice>
        options={[
          {
            value: 'auto',
            label: t.settings.methodAuto,
            desc: t.settings.methodAutoDesc(country ? countryName(country, lang === 'bg' ? 'bg' :
'en') : '-', t.methods[autoMethod]),
          },
          ...CALC_METHODS.map((m) => ({ value: m, label: t.methods[m] })),
        ]}
        value={s.method}
        onChange={(m) => {
          s.setMethod(m);
          setOpen(null);
        }}
      />
    )}
    <NavRow
      label={t.settings.asr}
      value={s.madhab === 'hanafi' ? t.settings.hanafi : t.settings.shafi}
      onPress={() => toggle('asr')}
    />
    {open === 'asr' && (
      <Choice
        options={[
          { value: 'shafi' as const, label: t.settings.shafi, desc: t.settings.shafiDesc },
          { value: 'hanafi' as const, label: t.settings.hanafi, desc: t.settings.hanafiDesc },
        ]}
        value={s.madhab}
        onChange={(m) => {
          s.setMadhab(m);
          setOpen(null);
        }}
      />
    )}
    {highLat && (
      <>
        <NavRow
          label={t.settings.highLat}
          value={t.highLatRules[s.highLatRule].name}
          onPress={() => toggle('highLat')}
        />
      />
    )}
  )}
}

```

```

        />
        {open === 'highLat' && (
            <Choice
                options={HIGH_LAT_RULES.map((r) => ({
                    value: r,
                    label: t.highLatRules[r].name,
                    desc: t.highLatRules[r].desc,
                }))}
                value={s.highLatRule}
                onChange={(r) => {
                    s.setHighLatRule(r);
                    setOpen(null);
                }}
            />
        )}
    </>
)}
</Group>
) : (
    <Group label={t.settings.groupTimes} note={t.settings.sourceMuftiNote}>
        <Row first>
            <Text style={settingsStyles.label}>{t.settings.sourceMufti}</Text>
        </Row>
    </Group>
)}

/* ---- Корекции ---- */
<Group label={t.settings.groupCorrections} note={t.settings.offsetsNote}>
    {PRAYER_IDS.map((id, i) => (
        <StepperRow
            key={id}
            first={i === 0}
            label={t.prayers[id]}
            valueText={t.settings.minutes(s.offsets[id])}
            highlighted={s.offsets[id] !== 0}
            onMinus={() => s.changeOffset(id, -1)}
            onPlus={() => s.changeOffset(id, 1)}
            minusLabel={t.settings.decrease(t.prayers[id])}
            plusLabel={t.settings.increase(t.prayers[id])}
        />
    ))}
    {anyOffset && (
        <Pressable onPress={s.resetOffsets} accessibilityRole="button" style={styles.resetRow}>
            <Text style={styles.link}>{t.settings.reset}</Text>
        </Pressable>
    )}
</Group>

<Group label={t.settings.hijri} note={isCalc ? t.settings.hijriNoteAbroad : t.settings.hijriNote}>
    <StepperRow
        first
        label={t.settings.hijri}
        valueText={t.settings.days(s.hijriAdjust)}
        highlighted={s.hijriAdjust !== 0}
        onMinus={() => s.changeHijriAdjust(-1)}
        onPlus={() => s.changeHijriAdjust(1)}
        minusLabel={t.settings.decrease(t.settings.hijri)}
        plusLabel={t.settings.increase(t.settings.hijri)}
    />
</Group>

```

```

    { /* ---- Празници (етап 13): чии дати - и датата по Хиджра в „Днес“ и „Месец“ ---- */
    <Group label={t.holidaySource.group} note={t.holidaySource.note}>
      <NavRow
        first
        label={t.holidaySource.row}
        value={
          s.holidaySource === 'auto' || !s.holidaySource
            ? t.holidaySource.autoValue(t.holidaySource.short[selectHolidaySource(s)])
            : t.holidaySource.short[s.holidaySource]
        }
        onPress={() => router.push('/holiday-source')}
      />
    </Group>

    { /* ---- Джамии наблизно (етап 12): с кое приложение се отвара „Упътване“ ---- */
    <Group label={t.nav.group} note={t.nav.rowNote}>
      <NavRow
        first
        label={t.nav.row}
        value={s.navRemembered ? navAppName(s.navApp, t) : t.nav.ask}
        onPress={() => router.push('/navigation')}
      />
    </Group>

    { /* ---- Приложение: източниците и поверителността са в „За приложението“ ---- */
    <Group label={t.settings.groupApp}>
      <NavRow first label={t.settings.about} value={APP_VERSION} onPress={() => router.push('/about')}
    />
    </Group>
  </ScrollView>
</View>
);
}

const styles = StyleSheet.create({
  root: { flex: 1, backgroundColor: colors.base },
  content: { paddingHorizontal: 16, gap: 20 },
  title: { fontFamily: fonts.extrabold, fontSize: 26, color: colors.text, marginLeft: 4 },
  placeRow: { flexDirection: 'row', alignItems: 'center', gap: 12, paddingHorizontal: 14, paddingVertical:
12 },
  pressed: { backgroundColor: 'rgba(255,255,255,0.04)' },
  placeText: { flex: 1, gap: 2 },
  placeName: { fontFamily: fonts.extrabold, fontSize: 17, color: colors.text },
  placeDetail: { fontFamily: fonts.medium, fontSize: 12.5, color: colors.muted },
  badge: { fontFamily: fonts.semibold, fontSize: 11.5, color: colors.gold, marginTop: 2 },
  link: { fontFamily: fonts.bold, fontSize: 13, color: colors.gold },
  statusBox: { flex: 1, gap: 6 },
  statusLine: { flexDirection: 'row', alignItems: 'center', gap: 8 },
  statusText: { flex: 1, fontFamily: fonts.regular, fontSize: 12.5, lineHeight: 17, color: colors.muted },
  statusBusy: { color: colors.text },
  resetRow: {
    paddingHorizontal: 14,
    paddingVertical: 12,
    borderTopWidth: StyleSheet.hairlineWidth,
    borderTopColor: 'rgba(255,255,255,0.10)',
  },
});

```

Стъпка 5

Напомнянията

Включва се в листа на празника – за всеки празник поотделно; важи всяка година.

ПРАЗНИКЪТ	КОГА	ТЕКСТЪТ
Свещена нощ (Регаиб, Мирадж, Бераат, Кадир, Мевлид)	същата вечер при Магриб	„Тази вечер: Нощ Регаиб“
Празник (Байрамите, Рамазан, Арефе...)	вечерта преди при Магриб	„Утре: Курбан Байрам“

Часът на Магриб е за твоето място – от същия календар като молитвите. Планират се за 30 дни напред, заедно с известията за молитвите (те се обновяват при всяко отваряне и във фонов режим).

notifications.ts src/domain

- `planHolidayReminders` – напомнянията за празниците.

src/domain/notifications.ts

212 реда

```
import { formatHM } from './format';
import { holidaysInYear, reminderDayOf, type HolidayId, type HolidaySource } from './holidays';
import { PRAYER_IDS, PRAYERS, type PrayerId } from './prayers';
import { computeDay, type TimesOptions } from './times';

/**
 * Планът на известията – чиста функция, без expo-notifications, за да се тества.
 * Службата (src/services/notifications.ts) само го изпраща към телефона.
 */

/** Режимът на камбанката (виж src/store/alertPrefs.ts). */
export type AlertMode = 'off' | 'notify' | 'adhan';

/** Кой звук: кратък сигнал или езан (в етап 4 – временен звук). */
export type SoundKind = 'chime' | 'adhan';

export type NotificationKind = 'prayer' | 'reminder' | 'refresh' | 'holiday';

export interface PlannedNotification {
  /** Стабилен идентификатор: „ezan-20261001-maghrib-prayer“. */
  id: string;
  at: Date;
  kind: NotificationKind;
  prayer: PrayerId | null;
  sound: SoundKind;
  title: string;
  body: string;
}

/** Текстовете на известията (идват от src/i18n/strings.ts). */
export interface NotificationTexts {
  prayers: Record<PrayerId, string>;
  prayerTitle: (prayer: string, time: string) => string;
  /** „Време е за Иша (العشاء) • София“ */
  prayerBody: (prayer: string, arabic: string, place: string) => string;
  sunriseBody: (place: string) => string;
  /** Вторият ред: „Следваща: Фаджр в 05:37 (утре)“ */
  nextLine: (prayer: string, time: string, tomorrow: boolean) => string;
  reminderTitle: (prayer: string, minutes: number, time: string) => string;
  reminderBody: (place: string) => string;
  refreshTitle: string;
```

```

    refreshBody: string;
}

export interface PlanInput {
    now: Date;
    options: TimesOptions;
    modes: Record<PrayerId, AlertMode>;
    /** Напомняне X минути преди молитвата; 0 = без напомняне. */
    reminderMinutes: number;
    placeName: string;
    texts: NotificationTexts;
    /**
     * Колко известия най-много. iOS пази до 64 чакащи известия на приложение;
     * оставяме място за пробното известие.
     */
    limit?: number;
    /** За колко дни напред (от днес). */
    days?: number;
}

export const NOTIFICATION_LIMIT = 62;
export const PLAN_DAYS = 7;
/** Известие по-рано от това (напр. след 2 сек.) не се планира - няма смисъл. */
const MIN_LEAD_MS = 5_000;

const pad = (n: number) => String(n).padStart(2, '0');
const ymd = (d: Date) => `${d.getFullYear()}${pad(d.getMonth() + 1)}${pad(d.getDate())}`;

/**
 * Всички известия за следващите дни, по ред на часа, до `limit`.
 * Ако не се събират всички, последното място е за напомняне „отвори приложението“ -
 * на iPhone без него известията биха спрели тихо след ~5 дни, ако приложението не се отваря.
 */
export function planNotifications(input: PlanInput): PlannedNotification[] {
    const { now, options, modes, reminderMinutes, placeName, texts } = input;
    const limit = input.limit ?? NOTIFICATION_LIMIT;
    const days = input.days ?? PLAN_DAYS;
    const earliest = now.getTime() + MIN_LEAD_MS;

    const dayAt = (d: number) => new Date(now.getFullYear(), now.getMonth(), now.getDate() + d, 12);
    const all: PlannedNotification[] = [];
    // денят след последния е нужен само за реда „Следваща: Фаджр ... (утре)“
    let next = computeDay(dayAt(0), options);
    for (let d = 0; d < days; d++) {
        const date = dayAt(d);
        const day = next;
        next = computeDay(dayAt(d + 1), options);
        PRAYER_IDS.forEach((id, i) => {
            const mode = modes[id];
            if (mode === 'off') return;
            const time = day[i].time;
            const name = texts.prayers[id];
            const hm = formatHM(time);
            const key = `ezan-${ymd(date)}-${id}`;
            const following = i + 1 < day.length ? day[i + 1] : next[0];
            const nextLine = texts.nextLine(
                texts.prayers[following.id],
                formatHM(following.time),
                i + 1 >= day.length,
            );
        });
    }
}

```

```

    if (reminderMinutes > 0) {
      all.push({
        id: `${key}-reminder`,
        at: new Date(time.getTime() - reminderMinutes * 60_000),
        kind: 'reminder',
        prayer: id,
        sound: 'chime',
        title: texts.reminderTitle(name, reminderMinutes, hm),
        body: texts.reminderBody(placeName),
      });
    }
    all.push({
      id: `${key}-prayer`,
      at: time,
      kind: 'prayer',
      prayer: id,
      // изгревът е само известие, без езан (виж PRAYERS.sunrise.canAlarm)
      sound: mode === 'adhan' && id !== 'sunrise' ? 'adhan' : 'chime',
      title: texts.prayerTitle(name, hm),
      body:
        (id === 'sunrise' ? texts.sunriseBody(placeName) : texts.prayerBody(name, PRAYERS[id].arabic,
placeName)) +
        '\n' +
        nextLine,
    });
  });
}

const upcoming = all
  .filter((n) => !isNaN(n.at.getTime()) && n.at.getTime() >= earliest)
  .sort((a, b) => a.at.getTime() - b.at.getTime());

if (upcoming.length <= limit) return upcoming;

const kept = upcoming.slice(0, limit - 1);
const last = kept[kept.length - 1];
kept.push({
  id: 'ezan-refresh',
  at: new Date(last.at.getTime() + 2 * 60_000),
  kind: 'refresh',
  prayer: null,
  sound: 'chime',
  title: texts.refreshTitle,
  body: texts.refreshBody,
});
return kept;
}

/**
 * „Отпечатък“ на плана: ако е същият като вече изпратения, не пренасрочваме
 * (иначе всяко отваряне на приложението би трило и създавало 60 известия).
 */
export function planSignature(plan: PlannedNotification[], extra: string): string {
  return (
    extra +
    '|' +
    plan.map((n) => `${n.id}@${n.at.getTime()}:${n.sound}:${n.title}:${n.body}`).join('|')
  );
}

/* ----- празници (етап 13) */

```

```

export interface HolidayReminderInput {
  now: Date;
  options: TimesOptions;
  source: HolidaySource;
  /** За кои празници е включено напомнянето (всеки поотделно). */
  enabled: Partial<Record<HolidayId, boolean>>;
  texts: {
    names: Record<HolidayId, string>;
    about: Record<HolidayId, string>;
    notifyTonight: (name: string) => string;
    notifyTomorrow: (name: string) => string;
  };
  /** За колко дни напред. */
  days?: number;
}

export const HOLIDAY_PLAN_DAYS = 30;

/**
 * Напомнянията за празниците: при Магриб – свещената нощ същата вечер („Тази вечер: Нощ Регаиб“),
 * празникът – вечерта преди („Утре: Курбан Байрам“).
 */
export function planHolidayReminders(input: HolidayReminderInput): PlannedNotification[] {
  const { now, options, source, enabled, texts } = input;
  const days = input.days ?? HOLIDAY_PLAN_DAYS;
  if (!Object.values(enabled).some(Boolean)) return [];
  const until = now.getTime() + days * 86_400_000;
  const out: PlannedNotification[] = [];
  for (const year of [now.getFullYear(), now.getFullYear() + 1]) {
    for (const h of holidaysInYear(source, year)) {
      if (!enabled[h.id]) continue;
      const day = reminderDayOf(h);
      if (day.getTime() > until + 86_400_000) continue;
      const maghrib = computeDay(day, options)[PRAYER_IDS.indexOf('maghrib')].time;
      if (isNaN(maghrib.getTime()) || maghrib.getTime() < now.getTime() + MIN_LEAD_MS || maghrib.getTime()
> until) continue;
      const name = texts.names[h.id];
      out.push({
        id: `ezan-holiday-${ymd(h.date)}-${h.id}`,
        at: maghrib,
        kind: 'holiday',
        prayer: null,
        sound: 'chime',
        title: h.night ? texts.notifyTonight(name) : texts.notifyTomorrow(name),
        body: texts.about[h.id],
      });
    }
  }
  return out.sort((a, b) => a.at.getTime() - b.at.getTime());
}

```

notifications.ts src/services

- Първо празниците, после молитвите в оставащите места (iPhone пази до 64 известия).

src/services/notifications.ts

430 реда

```

import * as Notifications from 'expo-notifications';
import { Linking, Platform } from 'react-native';

```

```

import { create } from 'zustand';

import {
  canScheduleExactAlarms,
  canUseFullScreenIntent,
  createSoundChannel,
  hasNativeAlarms,
  isIgnoringBatteryOptimizations,
  openExactAlarmSettings,
  setAlarms,
  type NativeAlarm,
} from '../modules/adhan-native';

import { NOTIFICATION_LIMIT, planHolidayReminders, planNotifications, type PlannedNotification } from '@domain/notifications';
import { PRAYERS, type PrayerId } from '@domain/prayers';
import { findSound, notificationFileName, type SoundDef } from '@domain/sounds';
import { getI18n } from '@i18n';
import { PHASE_GRADIENTS } from '@theme/gradients';
import { useAlertPrefs } from '@store/alertPrefs';
import { selectHolidaySource, selectLocation, selectTimesOptions, useSettings } from '@store/settings';
import { useSounds } from '@store/sounds';

/**
 * Известията за намаз – само локални (без интернет, без Firebase/FCM и без HMS Push).
 * Android планира всяко известие с AlarmManager, iPhone – с UNUserNotificationCenter.
 * Затова работят еднакво на телефони с Google услуги, на Huawei без тях и на iPhone.
 */

export type NotificationPermission = 'granted' | 'denied' | 'undetermined' | 'unavailable';

interface NotificationStatus {
  /** false, докато разрешението още не е проверено (първите мигове след стартиране). */
  checked: boolean;
  permission: NotificationPermission;
  /** Android 13+/iOS: може ли приложението пак да покаже системния въпрос. */
  canAskAgain: boolean;
  /** Android 12+: точни известия („Аларми и напомняния“); null – неприложимо. */
  exact: boolean | null;
  /** Колко известия са планирани и до кога (заедно с алармите). */
  count: number;
  until: Date | null;
  /** Android 14+: „Аларма на цял екран“; null – неприложимо. */
  fullScreen: boolean | null;
  /** Android: true – без ограничения на батерията; null – неприложимо. */
  battery: boolean | null;
  /** Колко аларми с езан са планирани в native частта (Android). */
  alarms: number;
}

export const useNotificationStatus = create<NotificationStatus>(() => ({
  checked: false,
  permission: Platform.OS === 'web' ? 'unavailable' : 'undetermined',
  canAskAgain: true,
  exact: null,
  count: 0,
  until: null,
  fullScreen: null,
  battery: null,
  alarms: 0,
})));

```

```

const SUPPORTED = Platform.OS === 'android' || Platform.OS === 'ios';

/* ----- звуци и канали (Android) */

/**
 * Этап 6: всяка молитва има свой звук. На Android звукът е на канала и не може да се
 * смени след създаването му - затова каналът е за двойката „вид + звук“ (+ с/без вибрация):
 * „ch4-n-chime-soft-v“ - известие/напомняне, „ch4-a-takbir_makka-v“ - езанът като известие
 * (без „Аларми и напомняния“), през потока за аларми. Ненужните канали се трият.
 */
type ChannelUse = 'notify' | 'alarm';
const CHANNEL_PREFIX = 'ch4-';
/** Каналите отпреди етап 6 („prayer-v3“, „adhan-s2“...). */
const OLD_CHANNEL = /^(prayer|adhan|reminder)-[vs]\d$/;

const channelId = (use: ChannelUse, sound: SoundDef, vibrate: boolean) =>
  `${CHANNEL_PREFIX}${use === 'alarm' ? 'a' : 'n'}-${(sound.custom ? sound.id : sound.file).replace(/[^a-z0-9_]/gi, '_')}-${vibrate ? 'v' : 's'}`;

/** Известие и напомняне: едно дълго вибриране (1 сек.). */
const VIBRATION_ONCE = [0, 1000];
/** Езанът като известие (до 30 сек.): 1 сек. вибрация / 0,7 сек. пауза. */
const VIBRATION_ALARM = [0, ...Array.from({ length: 16 }, () => [1000, 700]).flat()];
const vibrationFor = (use: ChannelUse) => (use === 'alarm' ? VIBRATION_ALARM : VIBRATION_ONCE);

/** Каналите, създадени в тази сесия (създаването е бавно - не всеки път). */
const created = new Set<string>();

async function ensureChannel(use: ChannelUse, sound: SoundDef, vibrate: boolean): Promise<string> {
  const id = channelId(use, sound, vibrate);
  if (Platform.OS !== 'android' || created.has(id)) return id;
  const { t, pick } = getI18n();
  const name = `${use === 'alarm' ? t.notifications.channelAdhan : t.notifications.channelPrayer} ·
  ${pick(sound.names)}`;
  // свой кратък звук (откъсът в Notifications/Ezan) и звукът на телефона - каналът се създава
  // в native частта, защото expo-notifications приема само звуци от res/raw
  if (sound.custom || sound.id === 'system') {
    createSoundChannel(id, name, sound.file, use === 'alarm', vibrate, vibrationFor(use));
    created.add(id);
    return id;
  }
  const { AndroidAudioUsage: Usage, AndroidAudioContentType: Content, AndroidImportance: Imp } =
  Notifications;
  await Notifications.setNotificationChannelAsync(id, {
    name,
    importance: use === 'alarm' ? Imp.MAX : Imp.HIGH,
    sound: notificationFileName(sound),
    // езанът - през потока за аларми: не зависи от силата на звука за известия и от безшумния режим
    audioAttributes: { usage: use === 'alarm' ? Usage.ALARM : Usage.NOTIFICATION, contentType:
  Content.SONIFICATION },
    enableVibrate: vibrate,
    vibrationPattern: vibrate ? vibrationFor(use) : null,
    lockscreenVisibility: Notifications.AndroidNotificationVisibility.PUBLIC,
  });
  created.add(id);
  return id;
}

/** Трие каналите, които вече не трябва (стари версии, сменени звуци, вибрация). */
async function cleanupChannels(wanted: Set<string>): Promise<void> {

```

```

if (Platform.OS !== 'android') return;
const channels = await Notifications.getNotificationChannelsAsync();
for (const c of channels) {
  if ((c.id.startsWith(CHANNEL_PREFIX) && !wanted.has(c.id)) || OLD_CHANNEL.test(c.id)) {
    await Notifications.deleteNotificationChannelAsync(c.id);
    created.delete(c.id);
  }
}
}

/** Избраните звуци - от настройките (src/store/sounds.ts). */
function soundChoice() {
  const s = useSounds.getState();
  return {
    notify: findSound(s.notify, 'short', s.custom, 'notify'),
    short: (p: PrayerId) => findSound(s.short[p], 'short', s.custom),
    full: (p: PrayerId) => findSound(s.full[p], 'full', s.custom),
  };
}

/* ----- разрешение */

function toPermission(p: Notifications.NotificationPermissionsStatus): NotificationPermission {
  if (Platform.OS === 'ios') {
    const s = p.ios?.status;
    if (
      s === Notifications.IosAuthorizationStatus.AUTHORIZED ||
      s === Notifications.IosAuthorizationStatus.PROVISIONAL ||
      s === Notifications.IosAuthorizationStatus.EPHEMERAL
    ) {
      return 'granted';
    }
    return s === Notifications.IosAuthorizationStatus.DENIED ? 'denied' : 'undetermined';
  }
  if (p.granted) return 'granted';
  return p.status === 'undetermined' ? 'undetermined' : 'denied';
}

export async function refreshPermission(): Promise<NotificationPermission> {
  if (!SUPPORTED) return 'unavailable';
  const p = await Notifications.getPermissionsAsync();
  const permission = toPermission(p);
  useNotificationStatus.setState({
    checked: true,
    permission,
    canAskAgain: p.canAskAgain,
    exact: canScheduleExactAlarms(),
    fullScreen: canUseFullScreenIntent(),
    battery: isIgnoringBatteryOptimizations(),
  });
  return permission;
}

/**
 * Пита за разрешение. Ако системата вече не показва въпроса (отказано завинаги) -
 * отваря настройките на приложението в телефона.
 */
export async function requestPermission(): Promise<NotificationPermission> {
  if (!SUPPORTED) return 'unavailable';
  const current = await Notifications.getPermissionsAsync();
  if (toPermission(current) === 'denied' && !current.canAskAgain) {

```

```

    await Linking.openSettings();
    return 'denied';
  }
  const p = await Notifications.requestPermissionsAsync({
    ios: { allowAlert: true, allowSound: true, allowBadge: false },
  });
  const permission = toPermission(p);
  useNotificationStatus.setState({ permission, canAskAgain: p.canAskAgain });
  if (permission === 'granted') rescheduleNotifications();
  return permission;
}

export { openExactAlarmSettings };

/* ----- планиране */

/** Всичко, което определя едно известие. Ако се смени, известието се пренасрочва. */
function signatureOf(n: PlannedNotification, channel: string, exact: boolean | null): string {
  return `${n.at.getTime()}|${n.sound}|${channel}|${exact}|${n.title}|${n.body}`;
}

let running: Promise<void> | null = null;
let again = false;

/**
 * Привежда планираните известия в съответствие с настройките.
 * Вика се при стартиране, при връщане в приложението, при всяка промяна на мястото,
 * часовете, камбанките, напомнянето и вибрацията, и от фоновата задача.
 * Паралелните извиквания се обединяват.
 */
export function rescheduleNotifications(): Promise<void> {
  if (running) {
    again = true;
    return running;
  }
  running = doReschedule()
    .catch((e) => console.warn('[notifications]', e))
    .finally(() => {
      running = null;
      if (again) {
        again = false;
        rescheduleNotifications();
      }
    });
  return running;
}

async function doReschedule(): Promise<void> {
  if (!SUPPORTED) return;
  const permission = await refreshPermission();
  const exact = useNotificationStatus.getState().exact;

  const s = useSettings.getState();
  const { t, pick } = getI18n();
  // етап 13: напомнянията за празниците – първо те, после молитвите в оставащите места
  const holidayPlan =
    permission === 'granted'
      ? planHolidayReminders({
          now: new Date(),
          options: selectTimesOptions(s).options,
          source: selectHolidaySource(s),

```

```

        enabled: s.holidayReminders ?? {},
        texts: t.holidays,
    })
    : [];
const plan =
    permission === 'granted'
    ? [
        ...holidayPlan,
        ...planNotifications({
            now: new Date(),
            options: selectTimesOptions(s).options,
            modes: useAlertPrefs.getState().modes,
            reminderMinutes: s.reminderMinutes,
            placeName: pick(selectLocation(s).names),
            texts: { prayers: t.prayers, ...t.notifications },
            limit: NOTIFICATION_LIMIT - holidayPlan.length,
        }),
    ].sort((a, b) => a.at.getTime() - b.at.getTime())
    : [];

const sounds = soundChoice();

// Android (етап 5): езанът е истинска аларма в native частта - пълен звук, „Спри“,
// екран „Аларма“. Без точни аларми (setAlarmClock не става) - остава звукът на известието.
let alarmCount = 0;
let notifications = plan;
if (nativeAlarmsEnabled(exact)) {
    const alarms = plan.filter(isAlarm);
    notifications = plan.filter((n) => !isAlarm(n));
    alarmCount = Math.max(
        0,
        setAlarms(alarms.map((n) => toNativeAlarm(n, s.vibrate, pick(selectLocation(s).names),
sounds.full(n.prayer!)))),
    );
} else if (hasNativeAlarms()) {
    setAlarms([]);
}

// Сравнява с вече планираните: трие само променените и липсващите, добавя само новите.
const existing = await Notifications.getAllScheduledNotificationsAsync();
const ours = new Map(
    existing
        .filter((r) => r.identifier.startsWith('ezan-'))
        .map((r) => [r.identifier, (r.content.data as { sig?: string } | null)?.sig ?? '']),
);

// кой звук за всяко известие: езанът като известие - краткият звук на молитвата; другите - звукът на
известията
const wanted = [];
for (const n of notifications) {
    const use: ChannelUse = isAlarm(n) ? 'alarm' : 'notify';
    const sound = use === 'alarm' ? sounds.short(n.prayer!) : sounds.notify;
    const channel = await ensureChannel(use, sound, s.vibrate);
    wanted.push({ n, use, sound, channel, sig: signatureOf(n, channel, exact) + '|' +
notificationFileName(sound) });
}
const wantedIds = new Set(wanted.map((w) => w.n.id));

for (const [id, sig] of ours) {
    const w = wanted.find((x) => x.n.id === id);
    if (!wantedIds.has(id) || !w || w.sig !== sig) {

```

```

        await Notifications.cancelScheduledNotificationAsync(id);
        ours.delete(id);
    }
}

for (const { n, use, sound, channel, sig } of wanted) {
    if (ours.has(n.id)) continue;
    await Notifications.scheduleNotificationAsync({
        identifier: n.id,
        content: {
            title: n.title,
            body: n.body,
            // на Android звукът е от канала, но без име на звук известието би било „тихо“;
            // на iPhone – файлът в приложението или в Library/Sounds (своите звуци)
            sound: notificationFileName(sound),
            // за Android под 8; на по-новите вибрацията е от канала
            vibrate: s.vibrate ? vibrationFor(use) : undefined,
            data: { sig, kind: n.kind, prayer: n.prayer, sound: n.sound, at: n.at.getTime() },
            ...(Platform.OS === 'android' ? { color: '#D4A857' } : {}),
        },
        trigger: {
            type: Notifications.SchedulableTriggerInputTypes.DATE,
            date: n.at,
            channelId: channel,
        },
    });
}

await cleanupChannels(new Set(wanted.map((w) => w.channel))).catch(() => {});

useNotificationStatus.setState({
    count: plan.length,
    until: plan.length ? plan[plan.length - 1].at : null,
    alarms: alarmCount,
});
}

/* ----- алармата (Android, етап 5) */

const isAlarm = (n: PlannedNotification) => n.kind === 'prayer' && n.sound === 'adhan' && n.prayer !== null;

/** Native алармата се ползва, ако я има в build-a и точните аларми не са забранени. */
function nativeAlarmsEnabled(exact: boolean | null): boolean {
    return Platform.OS === 'android' && hasNativeAlarms() && exact !== false;
}

/** Едно известие от плана → аларма за native частта, с всички текстове на езика на телефона. */
function toNativeAlarm(n: PlannedNotification, vibrate: boolean, place: string, sound: SoundDef): NativeAlarm {
    const { lang, t, pick } = getI18n();
    const prayer = (n.prayer ?? 'dhuhr') as PrayerId;
    const name = t.prayers[prayer];
    return {
        id: n.id,
        at: n.at.getTime(),
        prayer,
        title: t.alarm.title(name),
        arabic: PRAYERS[prayer].arabic,
        place,
        notifTitle: n.title,
    };
}

```

```

    notifBody: n.body,
    colors: [...PHASE_GRADIENTS[prayer]] as [string, string, string],
    vibrate,
    // вграден - името в res/raw; свой - пътят до файла
    sound: sound.file,
    lang,
    labels: {
      app: t.alarm.app,
      // „Спри езана“ - при езан; при мелодия или свой звук - „Спри алармата“
      stop: sound.category === 'adhan' || sound.category === 'takbir' ? t.alarm.stop : t.alarm.stopAlarm,
      mute: t.alarm.mute,
      muteShort: t.alarm.muteShort,
      close: t.alarm.close,
      soundName: pick(sound.names),
      channel: t.alarm.channel,
    },
  };
}

/** Трие всички известия на Езан и ги планира наново (от „Проверка на известията“). */
export async function resetAllNotifications(): Promise<void> {
  if (!SUPPORTED) return;
  await running;
  const existing = await Notifications.getAllScheduledNotificationsAsync();
  for (const r of existing) {
    if (r.identifier.startsWith('ezan-')) await
Notifications.cancelScheduledNotificationAsync(r.identifier);
  }
  created.clear();
  await rescheduleNotifications();
}

/**
 * Пробно известие след 10 сек. - със звука на известията. Връща часа, в който ще дойде.
 * (Звукът на алармата се преслушва направо в избора на звук - етап 6.)
 */
export async function sendTestNotification(): Promise<number> {
  const at = Date.now() + 10_000;
  if (!SUPPORTED) return at;
  const s = useSettings.getState();
  const sound = soundChoice().notify;
  const channel = await ensureChannel('notify', sound, s.vibrate);
  const { t } = getI18n();
  await Notifications.scheduleNotificationAsync({
    identifier: `test-${Date.now()}`,
    content: {
      title: t.notifications.testTitle,
      body: t.notifications.testBody,
      sound: notificationFileName(sound),
      vibrate: s.vibrate ? vibrationFor('notify') : undefined,
      data: { kind: 'test', at },
    },
  },
  trigger: {
    type: Notifications.SchedulableTriggerInputTypes.DATE,
    date: new Date(at),
    channelId: channel,
  },
});
  return at;
}

```

```

/** Как се показва известие, докато приложението е отворено: като обикновено, със звук. */
export function configureNotificationHandler(): void {
  if (!SUPPORTED) return;
  Notifications.setNotificationHandler({
    handleNotification: async () => ({
      shouldShowBanner: true,
      shouldShowList: true,
      shouldPlaySound: true,
      shouldSetBadge: false,
    }),
  });
}

```

diagnostics.tsx src/app

- В „Проверка на известията“ напомнянията за празници са при „напомняне“.

src/app/diagnostics.tsx

517 реда

```

import * as Notifications from 'expo-notifications';
import { router } from 'expo-router';
import { useCallback, useEffect, useRef, useState } from 'react';
import { ActivityIndicator, Platform, Pressable, ScrollView, StyleSheet, Text, View } from 'react-native';
import { useSafeAreaInsets } from 'react-native-safe-area-context';

import { getAlarmHistory, getAlarms, openFullScreenIntentSettings } from '../../modules/adhan-native';

import { GeometricPattern } from '@components/GeometricPattern';
import { AlertIcon, ChevronDownIcon, ChevronIcon } from '@components/icons';
import { formatGregorianShort, formatHM } from '@domain/format';
import type { SoundKind } from '@domain/notifications';
import { useI18n } from '@i18n';
import {
  openExactAlarmSettings,
  refreshPermission,
  requestPermission,
  resetAllNotifications,
  sendTestNotification,
  useNotificationStatus,
} from '@services/notifications';
import { colors } from '@theme/colors';
import { fonts, tabularNums } from '@theme/typography';

/* ----- данни */

type Kind = 'alarm' | 'notification' | 'reminder' | 'other';

interface Snapshot {
  /** Кога са прочетени данните – показва се под заглавието. */
  loadedAt: Date;
  channels: Notifications.NotificationChannel[];
  upcoming: { id: string; at: Date; title: string; kind: Kind }[];
  recent: { id: string; planned: Date | null; shown: Date; title: string }[];
}

const pad = (n: number) => String(n).padStart(2, '0');
const hms = (d: Date) => `${pad(d.getHours())}:${pad(d.getMinutes())}:${pad(d.getSeconds())}`;
const USAGE: Record<number, string> = { 4: 'alarm', 5: 'notification' };

const atOf = (data: unknown) => {

```

```

const at = (data as { at?: number } | null)?.at;
return typeof at === 'number' ? new Date(at) : null;
};

/** Видът на известието по канала му (Android) или по данните (iPhone). */
function kindOf(r: Notifications.NotificationRequest): Kind {
  const channel = (r.trigger as { channelId?: string } | null)?.channelId ?? '';
  const data = r.content.data as { kind?: string; sound?: string } | null;
  if (channel.startsWith('adhan') || data?.sound === 'adhan') return 'alarm';
  if (channel.startsWith('reminder') || data?.kind === 'reminder' || data?.kind === 'refresh' || data?.kind === 'holiday') return 'reminder';
  if (channel.startsWith('prayer') || data?.kind === 'prayer') return 'notification';
  return 'other';
}

async function fetchSnapshot(): Promise<Snapshot> {
  if (Platform.OS === 'web') return { loadedAt: new Date(), channels: [], upcoming: [], recent: [] };
  await refreshPermission();
  const [channels, scheduled, presented] = await Promise.all([
    Platform.OS === 'android' ? Notifications.getNotificationChannelsAsync() : Promise.resolve([]),
    Notifications.getAllScheduledNotificationsAsync(),
    Notifications.getPresentedNotificationsAsync(),
  ]);
  // Android, етап 5: алармите с езана са в native частта, не в expo-notifications
  const now = Date.now();
  const alarms = getAlarms()
    .filter((a) => a.at > now)
    .map((a) => ({ id: a.id, at: new Date(a.at), title: a.notifTitle, kind: 'alarm' as Kind }));
  const fired = getAlarmHistory().map((h) => ({
    id: `alarm-${h.id}-${h.fired}`,
    planned: new Date(h.planned),
    shown: new Date(h.fired),
    title: h.title,
  }));
  return {
    loadedAt: new Date(),
    channels: channels.filter((c) => !c.id.startsWith('expo_')),
    upcoming: scheduled
      .map((r) => ({ id: r.identifier, at: atOf(r.content.data) ?? new Date(0), title: r.content.title ?? '', kind: kindOf(r) })))
      .concat(alarms)
      .sort((a, b) => a.at.getTime() - b.at.getTime()),
    recent: presented
      .map((n) => ({
        id: n.request.identifier,
        planned: atOf(n.request.content.data),
        shown: new Date(n.date),
        title: n.request.content.title ?? '',
      })))
      .concat(fired)
      .sort((a, b) => b.shown.getTime() - a.shown.getTime()),
  };
}

/* ----- екран */

/**
 * „Проверка на известията“ (Настройки → Известия):
 * състояние с отметки, пробно известие, последните показани (със закъснението),
 * следващите планирани и – сгънати – техническите данни.
 */

```

```

export default function DiagnosticsScreen() {
  const insets = useSafeAreaInsets();
  const { t } = useI18n();
  const d = t.notifications.diag;
  const status = useNotificationStatus();
  const [snap, setSnap] = useState<Snapshot | null>(null);
  const [busy, setBusy] = useState(false);
  const [techOpen, setTechOpen] = useState(false);
  const [test, setTest] = useState<{ sound: SoundKind; at: number } | null>(null);
  const [now, setNow] = useState(() => Date.now());

  const [refreshing, setRefreshing] = useState(false);
  const [resetDone, setResetDone] = useState(false);
  const load = useCallback(() => fetchSnapshot().then(setSnap), []);

  // „Обнови“: чете наново от телефона; индикаторът стои поне 0,6 сек., за да се забележи
  const refresh = async () => {
    setRefreshing(true);
    await Promise.all([load(), new Promise((r) => setTimeout(r, 600))]);
    setRefreshing(false);
  };

  useEffect(() => {
    let alive = true;
    fetchSnapshot().then((x) => alive && setSnap(x));
    return () => {
      alive = false;
    };
  }, []);

  // обратно броене на пробното известие; след него – обновяване, за да се види в „Последни“
  useEffect(() => {
    if (!test) return;
    const timer = setInterval(() => {
      const t2 = Date.now();
      setNow(t2);
      if (t2 >= test.at + 1500) {
        setTest(null);
        load();
      }
    }, 500);
    return () => clearInterval(timer);
  }, [test, load]);

  const closing = useRef(false);
  const back = () => {
    if (closing.current) return;
    closing.current = true;
    if (router.canGoBack()) router.back();
    else router.replace('/settings');
  };

  const sendTest = async (sound: SoundKind) => {
    const at = await sendTestNotification();
    setNow(at - 10_000);
    setTest({ sound, at });
  };

  const reset = async () => {
    setBusy(true);
    setResetDone(false);
  };

```

```

    await resetAllNotifications();
    await load();
    setBusy(false);
    setResetDone(true);
};

const granted = status.permission === 'granted';
const kindLabel: Record<Kind, string> = {
    alarm: d.kindAlarm,
    notification: d.kindNotification,
    reminder: d.kindReminder,
    other: d.kindOther,
};
const until = status.until ? `${formatGregorianShort(status.until, t.date)}, ${formatHM(status.until)}` :
'';
const device =
    Platform.OS === 'android'
        ? `${Platform.constants.Manufacturer} ${Platform.constants.Model} · Android
        ${Platform.constants.Release} (API ${Platform.Version})`
        : `${Platform.OS} ${Platform.Version}`;

const testButton = (sound: SoundKind, label: string) => {
    const active = test?.sound === sound;
    const left = active ? Math.max(0, Math.ceil((test.at - now) / 1000)) : 0;
    return (
        <Pressable
            onPress={() => sendTest(sound)}
            disabled={!test || !granted}
            accessibilityRole="button"
            style={({ pressed }) => [
                styles.testBtn,
                sound === 'adhan' && styles.testBtnGold,
                (!test && !active) || !granted ? styles.disabled : null,
                pressed && styles.pressed,
            ]}
        >
            <AlertIcon mode={sound === 'adhan' ? 'adhan' : 'notify'} size={20} color={sound === 'adhan' ?
colors.goldInk : colors.text} />
            <Text style={[styles.testLabel, sound === 'adhan' && styles.testLabelGold]}>{label}</Text>
            {active && <Text style={[styles.testIn, sound === 'adhan' && styles.testLabelGold]}
{d.testIn(left)}</Text>}
        </Pressable>
    );
};

return (
    <View style={styles.root}>
        <GeometricPattern />
        <View style={[styles.header, { paddingTop: insets.top + 8 }]}>
            <Pressable onPress={back} hitSlop={10} accessibilityRole="button" style={styles.backBtn}>
                <ChevronIcon direction="left" size={18} color={colors.gold} />
                <Text style={styles.link}>{t.settings.title}</Text>
            </Pressable>
            <Pressable onPress={refresh} disabled={refreshing} hitSlop={10} accessibilityRole="button" style=
{styles.refreshBtn}>
                {refreshing ? <ActivityIndicator size="small" color={colors.gold} /> : <Text style={styles.link}>
{d.refresh}</Text>}
            </Pressable>
        </View>

        <ScrollView contentContainerStyle={[styles.content, { paddingBottom: insets.bottom + 32 }]}>

```

```

<View style={styles.titleBox}>
  <Text style={styles.title}>{d.title}</Text>
  <Text style={styles.subtitle}>{d.subtitle}</Text>
  {snap && <Text style={styles.updated}>{d.updated(hms(snap.loadedAt))}</Text>}
</View>

{/* ---- СЪСТОЯНИЕ ---- */}
<Section label={d.status}>
  <Check
    ok={granted}
    text={granted ? d.permOk : d.permBad}
    action={granted ? undefined : { label: d.allow, onPress: requestPermission }}
    first
  />
  {status.exact !== null && (
    <Check
      ok={status.exact}
      text={status.exact ? d.exactOk : d.exactBad}
      action={status.exact ? undefined : { label: d.allow, onPress: openExactAlarmSettings }}
    />
  )}
  {granted && status.fullScreen !== null && (
    <Check
      ok={status.fullScreen}
      text={status.fullScreen ? d.fullScreenOk : d.fullScreenBad}
      action={status.fullScreen ? undefined : { label: d.allow, onPress:
openFullScreenIntentSettings }}
    />
  )}
  {granted && status.battery !== null && (
    <Check
      ok={status.battery}
      text={status.battery ? d.batteryOk : d.batteryBad}
      action={status.battery ? undefined : { label: d.fix, onPress: () => router.push('/battery')}
    />
  )}
}

  />
}
  <Check ok={status.count > 0} text={status.count > 0 ? d.scheduledOk(status.count, until) :
d.scheduledNone} />
</Section>

{/* ---- Пробно известие ---- */}
<Section label={d.test}>
  <View style={styles.testBox}>
    <View style={styles.testRow}>
      {testButton('chime', d.testNotification)}
    </View>
    <Text style={styles.small}>{d.testHint}</Text>
  </View>
</Section>

{!snap ? (
  <ActivityIndicator color={colors.gold} style={{ marginTop: 12 }} />
) : (
  <>
    {/* ---- Последни ---- */}
    <Section label={d.recent}>
      {snap.recent.length === 0 ? (
        <Line first>
          <Text style={styles.small}>{d.recentNone}</Text>
        </Line>
      ) : (

```

```

    ) : (
      snap.recent.slice(0, 8).map((p, i) => {
        const delay = p.planned ? Math.round((p.shown.getTime() - p.planned.getTime()) / 1000) :
null;

        const onTime = delay !== null && delay < 60;
        return (
          <Line key={p.id + i} first={i === 0}>
            <View style={styles.item}>
              <Text style={styles.itemTitle} numberOfLines={1}>
                {p.title}
              </Text>
              <Text style={styles.small}>
                {p.planned ? d.plannedShown(hms(p.planned), hms(p.shown)) : hms(p.shown)}
              </Text>
            </View>
            {delay !== null && (
              <View style={[styles.badge, onTime ? styles.badgeOk : styles.badgeWarn]}>
                <Text style={styles.badgeText, onTime ? styles.badgeTextOk :
styles.badgeTextWarn]}>
                  {onTime ? d.onTime : d.late(delay)}
                </Text>
              </View>
            )}
          </Line>
        );
      })
    )}
  </Section>

  {/* ---- Следващи ---- */}
  {snap.upcoming.length > 0 && (
    <Section label={d.upcoming}>
      {snap.upcoming.slice(0, 8).map((r, i) => (
        <Line key={r.id} first={i === 0}>
          <View style={styles.timeCol}>
            <Text style={styles.time}>{formatHM(r.at)}</Text>
            <Text style={styles.tiny}>{t.date.weekdaysShort[r.at.getDay()]}</Text>
          </View>
          <Text style={[styles.itemTitle, styles.flex]} numberOfLines={1}>
            {r.title}
          </Text>
          <View style={[styles.tag, r.kind === 'alarm' && styles.tagGold]}>
            <Text style={styles.tagText, r.kind === 'alarm' && styles.tagTextGold}>
{kindLabel[r.kind]}</Text>
          </View>
        </Line>
      ))}
    </Section>
  )}

  {/* ---- Технически данни (сгънати) ---- */}
  <View style={styles.card}>
    <Pressable
      onPress={() => setTechOpen((o) => !o)}
      accessibilityRole="button"
      accessibilityState={{ expanded: techOpen }}
      style={styles.techHead}
    >
      <Text style={styles.itemTitle}>{d.tech}</Text>
      <View style={techOpen && styles.flip}>
        <ChevronDownIcon color={colors.muted} />
      </View>
    </Pressable>
  </View>

```

```

        </View>
      </Pressable>
      {techOpen && (
        <View style={styles.techBody}>
          <Text style={styles.small}>
            {d.device}: {device}
          </Text>
          {snap.channels.map((c) => (
            <Text key={c.id} style={styles.small}>
              <Text style={styles.mono}>{c.id}</Text> -{' '}
              {d.channelLine(
                c.importance,
                c.sound ?? '-',
                c.enableVibrate,
                USAGE[c.audioAttributes?.usage ?? -1] ?? String(c.audioAttributes?.usage ?? '-'),
              )}
            </Text>
          ))}
        </View>
      )}
    </View>

    <View style={styles.resetBox}>
      <Pressable onPress={reset} disabled={busy} accessibilityRole="button" style=
{styles.resetBtn}>
        {busy ? <ActivityIndicator color={colors.gold} /> : <Text style={styles.link}>{d.reset}
      </Text>}
      </Pressable>
      <Text style={[[styles.small, styles.center]]}>{resetDone ? d.resetDone : d.resetHint}</Text>
    </View>
  </>
)}
</ScrollView>
</View>
);
}

/* ----- части */

function Section({ label, children }: { label: string; children: React.ReactNode }) {
  return (
    <View style={styles.section}>
      <Text style={styles.sectionLabel}>{label.toUpperCase()}</Text>
      <View style={styles.card}>{children}</View>
    </View>
  );
}

function Line({ children, first }: { children: React.ReactNode; first?: boolean }) {
  return <View style={[[styles.line, !first && styles.lineBorder]]}>{children}</View>;
}

function Check({
  ok,
  text,
  action,
  first,
}: {
  ok: boolean;
  text: string;
  action?: { label: string; onPress: () => void };

```

```

    first?: boolean;
  }) {
    return (
      <Line first={first}>
        <View style={styles.dot, ok ? styles.dotOk : styles.dotWarn}>
          <Text style={styles.dotText}>{ok ? '✓' : '!'}</Text>
        </View>
        <Text style={styles.checkText, !ok && styles.checkWarn}>{text}</Text>
        {action && (
          <Pressable onPress={action.onPress} hitSlop={8} accessibilityRole="button" style=
{styles.actionBtn}>
            <Text style={styles.actionText}>{action.label}</Text>
          </Pressable>
        )}
      </Line>
    );
  }

  const GREEN = '#5FBF8F';

  const styles = StyleSheet.create({
    root: { flex: 1, backgroundColor: colors.base },
    header: {
      paddingHorizontal: 16,
      paddingBottom: 4,
      flexDirection: 'row',
      justifyContent: 'space-between',
      alignItems: 'center',
    },
    backBtn: { flexDirection: 'row', alignItems: 'center', gap: 4, paddingVertical: 6 },
    link: { fontFamily: fonts.bold, fontSize: 15, color: colors.gold },
    content: { paddingHorizontal: 16, gap: 20 },
    titleBox: { gap: 6, marginTop: 4 },
    title: { fontFamily: fonts.extrabold, fontSize: 26, color: colors.text },
    subtitle: { fontFamily: fonts.regular, fontSize: 13, lineHeight: 19, color: colors.muted },

    section: { gap: 7 },
    sectionLabel: { fontFamily: fonts.bold, fontSize: 11, letterSpacing: 1.1, color: colors.muted,
paddingLeft: 6 },
    card: {
      backgroundColor: 'rgba(255,255,255,0.05)',
      borderWidth: 1,
      borderColor: 'rgba(255,255,255,0.06)',
      borderRadius: 18,
      overflow: 'hidden',
    },
    line: { flexDirection: 'row', alignItems: 'center', gap: 12, paddingHorizontal: 14, paddingVertical: 12,
minHeight: 52 },
    lineBorder: { borderTopWidth: StyleSheet.hairlineWidth, borderTopColor: 'rgba(255,255,255,0.10)' },

    dot: { width: 24, height: 24, borderRadius: 12, alignItems: 'center', justifyContent: 'center' },
    dotOk: { backgroundColor: 'rgba(95,191,143,0.18)' },
    dotWarn: { backgroundColor: colors.warn },
    dotText: { fontFamily: fonts.extrabold, fontSize: 13, color: colors.text, includeFontPadding: false },
    checkText: { flex: 1, fontFamily: fonts.semibold, fontSize: 14, lineHeight: 19, color: colors.text },
    checkWarn: { color: colors.warn },
    actionBtn: { paddingHorizontal: 12, paddingVertical: 6, borderRadius: 999, backgroundColor: colors.gold
},
    actionText: { fontFamily: fonts.bold, fontSize: 12.5, color: colors.goldInk },

    testBox: { padding: 14, gap: 12 },
  });

```

```

testRow: { flexDirection: 'row', gap: 10 },
testBtn: {
  flex: 1,
  height: 78,
  borderRadius: 16,
  alignItems: 'center',
  justifyContent: 'center',
  gap: 4,
  backgroundColor: 'rgba(255,255,255,0.08)',
  borderWidth: 1,
  borderColor: 'rgba(255,255,255,0.10)',
},
testBtnGold: { backgroundColor: colors.gold, borderColor: colors.gold },
testLabel: { fontFamily: fonts.bold, fontSize: 14, color: colors.text },
testLabelGold: { color: colors.goldInk },
testIn: { fontFamily: fonts.semibold, fontSize: 11.5, color: colors.textDim, ...tabularNums },
disabled: { opacity: 0.4 },
pressed: { transform: [{ scale: 0.97 }] },

item: { flex: 1, gap: 2 },
flex: { flex: 1 },
itemTitle: { fontFamily: fonts.bold, fontSize: 14, color: colors.text },
small: { fontFamily: fonts.regular, fontSize: 12, lineHeight: 17, color: colors.muted },
tiny: { fontFamily: fonts.medium, fontSize: 10.5, color: colors.muted },
mono: { fontFamily: fonts.bold, color: colors.textDim },

badge: { paddingHorizontal: 9, paddingVertical: 4, borderRadius: 999 },
badgeOk: { backgroundColor: 'rgba(95,191,143,0.16)' },
badgeWarn: { backgroundColor: 'rgba(227,154,75,0.18)' },
badgeText: { fontFamily: fonts.bold, fontSize: 11.5, ...tabularNums },
badgeTextOk: { color: GREEN },
badgeTextWarn: { color: colors.warn },

timeCol: { width: 46, alignItems: 'flex-start' },
time: { fontFamily: fonts.extrabold, fontSize: 15, color: colors.text, ...tabularNums },
tag: { paddingHorizontal: 8, paddingVertical: 3, borderRadius: 999, backgroundColor:
'rgba(255,255,255,0.08)' },
tagGold: { backgroundColor: 'rgba(212,168,87,0.18)' },
tagText: { fontFamily: fonts.semibold, fontSize: 11, color: colors.textDim },
tagTextGold: { color: colors.gold },

techHead: {
  flexDirection: 'row',
  alignItems: 'center',
  justifyContent: 'space-between',
  paddingHorizontal: 14,
  paddingVertical: 14,
},
flip: { transform: [{ rotate: '180deg' }] },
techBody: {
  paddingHorizontal: 14,
  paddingBottom: 14,
  gap: 6,
  borderTopWidth: StyleSheet.hairlineWidth,
  borderTopColor: 'rgba(255,255,255,0.10)',
  paddingTop: 10,
},
resetBox: { gap: 6, alignItems: 'center' },
resetBtn: {
  alignSelf: 'stretch',
  alignItems: 'center',

```

```
paddingVertical: 13,
borderRadius: 14,
borderWidth: 1,
borderColor: 'rgba(212,168,87,0.45)',
},
center: { textAlign: 'center', paddingHorizontal: 12 },
refreshBtn: { minWidth: 70, alignItems: 'flex-end' },
updated: { fontFamily: fonts.medium, fontSize: 11.5, color: colors.muted, ...tabularNums },
});
```

Стъпка 6

Екраните и текстовете

Новите и променените файлове.

month.tsx src/app/(tabs)

- Превключвателят „Часове | Празници“; датата по Хиджра – от избрания календар.

src/app/(tabs)/month.tsx

206 реда

```
import { Fragment, useDeferredValue, useEffect, useMemo, useRef, useState } from 'react';
import { ActivityIndicator, Pressable, ScrollView, StyleSheet, Text, View } from 'react-native';
import { useSafeAreaInsets } from 'react-native-safe-area-context';

import { GeometricPattern } from '@components/GeometricPattern';
import { HolidaysView } from '@components/holidays/HolidaysView';
import {
  HIJRI_SEPARATOR_HEIGHT,
  HijriMonthSeparator,
  MONTH_ROW_HEIGHT,
  MonthColumns,
  MonthRow,
} from '@components/MonthTable';
import { MonthHeader } from '@components/MonthHeader';
import { useTabBarHeight } from '@components/TabBar';
import { toHijri } from '@domain/hijri';
import { computeMonth } from '@domain/times';
import { useNow } from '@hooks/useNow';
import { useTimesOptions } from '@hooks/useTimesOptions';
import { useI18n } from '@i18n';
import { selectHijriCalendar, useSettings } from '@store/settings';
import { colors } from '@theme/colors';
import { fonts } from '@theme/typography';
```

/**

* „Месец“: превключвател „Часове | Празници“ (етап 13) – часовете за месеца или

* ислямските празници за годината.

*/

```
export default function MonthScreen() {
  const insets = useSafeAreaInsets();
  const { t } = useI18n();
  const [view, setView] = useState<'times' | 'holidays'>('times');
  return (
    <View style={[styles.root, { paddingTop: insets.top }]}>
      <GeometricPattern />
      <View style={styles.switch} accessibilityRole="tablist">
        {(['times', 'holidays'] as const).map((v) => {
          const on = v === view;
```

```

    return (
      <Pressable
        key={v}
        onPress={() => setView(v)}
        accessibilityRole="tab"
        accessibilityState={{ selected: on }}
        style={[styles.switchBtn, on && styles.switchOn]}
      >
        <Text style={[styles.switchText, on && styles.switchTextOn]}>
          {v === 'times' ? t.holidays.tabTimes : t.holidays.tabHolidays}
        </Text>
      </Pressable>
    );
  }}
</View>
{view === 'times' ? <MonthTimes /> : <HolidaysView />}
</View>
);
}

function MonthTimes() {
  const tabBarHeight = useTabBarHeight();
  const { t, pick } = useI18n();
  const now = useNow(60_000); // тук е достатъчно веднъж в минута

  const [ym, setYm] = useState(() => ({ y: now.getFullYear(), m: now.getMonth() }));
  const shiftMonth = (delta: number) => {
    setYm(({ y, m }) => {
      const d = new Date(y, m + delta, 1);
      return { y: d.getFullYear(), m: d.getMonth() };
    });
  };

  const { options, key } = useTimesOptions();
  const hijriAdjust = useSettings((s) => s.hijriAdjust);
  const calendar = useSettings(selectHijriCalendar);

  // Заглавието и бутоните реагират веднага, а таблицата се смята „на заден план“:
  // useDeferredValue първо рисува екрана със старата таблица (или с индикатор при първо
  // отваряне), после изчислява новия месец. Така смяната на месеца не „замръзва“.
  const request = useMemo(() => ({ y: ym.y, m: ym.m, key, hijriAdjust, calendar }), [ym.y, ym.m, key,
hijriAdjust, calendar]);
  const shown = useDeferredValue(request, null);
  const pending = shown !== request;
  const table = useMemo(() => {
    if (!shown) return null;
    const days = computeMonth(shown.y, shown.m, options);
    return { days, hijri: days.map((d) => toHijri(d.date, shown.hijriAdjust, shown.calendar)) };
    // options се сменя заедно с key
    // eslint-disable-next-line react-hooks/exhaustive-deps
  }, [shown]);

  const isCurrentMonth = ym.y === now.getFullYear() && ym.m === now.getMonth();
  const shownIsCurrent = !!shown && shown.y === now.getFullYear() && shown.m === now.getMonth();
  const todayIndex = shownIsCurrent ? now.getDate() - 1 : -1;

  // „Ребиул-ахир - Джемазиел-евел 1448“ (или две години, ако месецът ги пресича) -
  // смята се само за първия и последния ден, за да е веднага в заглавието
  const first = toHijri(new Date(ym.y, ym.m, 1), hijriAdjust, calendar);
  const last = toHijri(new Date(ym.y, ym.m + 1, 0), hijriAdjust, calendar);
  const hijriRange =
    first.month === last.month

```

```

    ? `${t.hijriMonths[first.month - 1]} ${first.year}`
    : first.year === last.year
    ? `${t.hijriMonths[first.month - 1]} - ${t.hijriMonths[last.month - 1]} ${last.year}`
    : `${t.hijriMonths[first.month - 1]} ${first.year} - ${t.hijriMonths[last.month - 1]}
    ${last.year}`;

// При отваряне на текущия месец превърта така, че днешният ден да се вижда
// (с 3 реда над него). Разделителите за нов месец по Хиджра също заемат място.
const scrollRef = useRef<ScrollView>(null);
useEffect(() => {
    if (!table) return;
    const target = todayIndex - 3;
    let y = 0;
    for (let i = 0; i < target; i++) {
        y += MONTH_ROW_HEIGHT + (table.hijri[i].day === 1 ? HIJRI_SEPARATOR_HEIGHT : 0);
    }
    scrollRef.current?.scrollTo({ y, animated: false });
}, [table, todayIndex]);

const location = options.location;

return (
    <View style={styles.inner}>
        <MonthHeader
            title={` ${t.date.monthsFull[ym.m]} ${ym.y} `}
            hijriRange={hijriRange}
            city={pick(location.names)}
            showToday={!isCurrentMonth}
            onPrev={() => shiftMonth(-1)}
            onNext={() => shiftMonth(1)}
            onToday={() => setYm({ y: now.getFullYear(), m: now.getMonth() })}
        />
        { /* Картата свършва над долната навигация, за да не се застъпват редовете с нея. */ }
        <View style={ [styles.card, { marginBottom: tabBarHeight } ] }>
            <MonthColumns />
            {table ? (
                <ScrollView
                    ref={scrollRef}
                    showsVerticalScrollIndicator={false}
                    contentContainerStyle={styles.list}
                    style={pending && styles.stale}
                >
                    {table.days.map((day, i) => (
                        <Fragment key={i}>
                            {table.hijri[i].day === 1 && (
                                <HijriMonthSeparator
                                    label={`${t.hijriMonths[table.hijri[i].month - 1]} ${table.hijri[i].year}`}
                                />
                            )}
                            <MonthRow day={day} hijriDay={table.hijri[i].day} isToday={i === todayIndex} />
                        </Fragment>
                    ))}
                    <Text style={styles.source}>
                        {location.source === 'mufti' ? t.month.sourceMufti :
t.month.sourceCalc(t.methods[options.method])}
                    </Text>
                </ScrollView>
            ) : null}
            {(pending || !table) && (
                <View style={styles.busy} pointerEvents="none">
                    <ActivityIndicator size="large" color={colors.gold} accessibilityLabel={t.busy} />
                </View>
            )}
        </View>
    </View>
)

```

```

        </View>
      )}
    </View>
  </View>
);
}

const styles = StyleSheet.create({
  root: { flex: 1, backgroundColor: colors.base },
  inner: { flex: 1 },
  switch: {
    flexDirection: 'row',
    gap: 4,
    padding: 4,
    marginHorizontal: 16,
    marginTop: 8,
    marginBottom: 2,
    borderRadius: 14,
    backgroundColor: 'rgba(6,10,20,0.5)',
    borderWidth: 1,
    borderColor: colors.cardBorder,
  },
  switchBtn: { flex: 1, height: 36, borderRadius: 10, alignItems: 'center', justifyContent: 'center' },
  switchOn: { backgroundColor: colors.gold },
  switchText: { fontFamily: fonts.bold, fontSize: 14, color: colors.muted },
  switchTextOn: { fontFamily: fonts.extrabold, color: colors.goldInk },
  card: {
    flex: 1,
    marginHorizontal: 10,
    paddingHorizontal: 2,
    paddingTop: 4,
    borderRadius: 22,
    backgroundColor: colors.card,
    borderWidth: 1,
    borderColor: colors.cardBorder,
    overflow: 'hidden',
  },
  list: { paddingBottom: 14 },
  stale: { opacity: 0.4 },
  busy: { position: 'absolute', top: 0, left: 0, right: 0, bottom: 0, alignItems: 'center', justifyContent: 'center' },
  source: {
    fontFamily: fonts.regular,
    fontSize: 11,
    color: colors.muted,
    textAlign: 'center',
    marginTop: 14,
    paddingHorizontal: 16,
  },
});

```

HolidaysView.tsx src/components/holidays

- Годианата, следващият празник, празниците по месеци.

src/components/holidays/HolidaysView.tsx

244 реда

```

import { useState } from 'react';
import { Pressable, ScrollView, StyleSheet, Text, View } from 'react-native';

```

```

import { useTabBarHeight } from '@components/TabBar';
import { daysUntil, HOLIDAYS, holidaysInYear, nextHoliday, type Holiday } from '@domain/holidays';
import { localEpochDay } from '@domain/hijriTable';
import { useNow } from '@hooks/useNow';
import { useI18n } from '@i18n';
import { selectHolidaySource, useSettings } from '@store/settings';
import { colors } from '@theme/colors';
import { FONT_SCALE, fonts, tabularNums } from '@theme/typography';

import { ChevronIcon } from '../icons';

import { HolidaySheet } from './HolidaySheet';
import { dayTile, hijriLine } from './holidayFormat';

/**
 * „Месец → Празници“ (етап 13): годината по Григорианския календар (по Хиджра отдолу),
 * следващият празник и всички празници по месеци. Докосване – подробности и напомняне.
 */
export function HolidaysView() {
  const tabBarHeight = useTabBarHeight();
  const { t } = useI18n();
  const h = t.holidays;
  const now = useNow(60_000);
  const source = useSettings(selectHolidaySource);
  const [year, setYear] = useState(now.getFullYear());
  const [open, setOpen] = useState<Holiday | null>(null);

  const list = holidaysInYear(source, year);
  const next = nextHoliday(source, now);
  const today = localEpochDay(now);
  const isNext = (x: Holiday) => !!next && next.id === x.id && next.date.getTime() === x.date.getTime();

  // по месеци
  const months: { month: number; items: Holiday[] }[] = [];
  for (const x of list) {
    const m = x.date.getMonth();
    if (months.at(-1)?.month !== m) months.push({ month: m, items: [] });
    months.at(-1)!.items.push(x);
  }

  const hijriFirst = list[0]?.hijri.year;
  const hijriLast = list.at(-1)?.hijri.year;

  return (
    <View style={styles.root}>
      <View style={styles.yearRow}>
        <Pressable
          onPress={() => setYear((y) => y - 1)}
          accessibilityRole="button"
          accessibilityLabel={h.prevYear}
          hitSlop={6}
          style={({ pressed }) => [styles.arrow, pressed && styles.pressed]}
        >
          <ChevronIcon direction="left" size={18} color={colors.text} />
        </Pressable>
        <View style={styles.yearText}>
          <Text style={styles.year} accessibilityRole="header">
            {year}
          </Text>
          {hijriFirst && hijriLast ? (
            <Text maxFontSizeMultiplier={FONT_SCALE.row} style={styles.hijriYears}>

```

```

        {h.hijriYears(hijriFirst, hijriLast)}
      </Text>
    ) : null}
  </View>
  <Pressable
    onPress={() => setYear((y) => y + 1)}
    accessibilityRole="button"
    accessibilityLabel={h.nextYear}
    hitSlop={6}
    style={({ pressed }) => [styles.arrow, pressed && styles.pressed]}
  >
    <ChevronIcon direction="right" size={18} color={colors.text} />
  </Pressable>
</View>

<ScrollView
  style={{ marginBottom: tabBarHeight }}
  contentContainerStyle={styles.content}
  showsVerticalScrollIndicator={false}
>
  {next && year === now.getFullYear() && (
    <Pressable onPress={() => setOpen(next)} accessibilityRole="button" style={({ pressed }) =>
[styles.next, pressed && styles.pressed]}>
      <View style={styles.nextText}>
        <Text style={styles.nextLabel}>{h.next.toUpperCase()}</Text>
        <Text style={styles.nextName}>{h.names[next.id]}</Text>
        <Text style={styles.nextSub}>{hijriLine(next, t, true)}</Text>
      </View>
      {daysUntil(next, now) > 1 ? (
        <View style={styles.nextCount}>
          <Text style={styles.nextNum}>{daysUntil(next, now)}</Text>
          <Text style={styles.nextDays}>{h.daysWord(daysUntil(next, now))}</Text>
        </View>
      ) : (
        <Text style={styles.nextSoon}>{h.inDays(Math.max(0, daysUntil(next, now)))}</Text>
      )}
    </Pressable>
  )}

  {list.length === 0 && <Text style={styles.empty}>{h.noData}</Text>}

  {months.map(({ month, items }) => (
    <View key={month}>
      <Text style={styles.monthLabel}>{t.date.monthsFull[month].toUpperCase()}</Text>
      <View style={styles.card}>
        {items.map((x, i) => {
          const tile = dayTile(x, t);
          const past = localEpochDay(x.end) < today;
          const highlight = isNext(x);
          return (
            <Pressable
              key={`-${x.id}-${x.date.getTime()}`}
              onPress={() => setOpen(x)}
              accessibilityRole="button"
              accessibilityLabel={h.a11yRow(h.names[x.id], `-${tile.wd} ${tile.day}`, hijriLine(x, t,
false))}>
                style={({ pressed }) => [
                  styles.row,
                  i > 0 && styles.rowBorder,
                  past && styles.past,
                  highlight && styles.rowNext,

```

```

        pressed && styles.pressed,
      ]}
    >
    <View style={styles.tile}>
      <Text style={styles.tileDay, tile.day.includes('-') && styles.tileRange, x.expected
&& styles.expectedText]} numberOfLines={1}>
        {x.expected ? `≈ ${tile.day}` : tile.day}
      </Text>
      <Text style={styles.tileWd}>{tile.wd}</Text>
    </View>
    <View style={styles.rowText}>
      <View style={styles.nameLine}>
        <Text style={styles.name}>{h.names[x.id]}</Text>
        <Text style={styles.arabic} importantForAccessibility="no"
accessibilityElementsHidden>
          {HOLIDAYS[x.id].arabic}
        </Text>
      </View>
      <Text style={styles.sub}>{hijriLine(x, t, false)}</Text>
    </View>
    {x.expected ? (
      <Text style={styles.badge}>{h.expected}</Text>
    ) : highlight ? (
      <Text style={styles.badgeNext}>{h.inDays(Math.max(0, daysUntil(x, now)))}</Text>
    ) : null}
  </Pressable>
);
  })}
</View>
</View>
))}

  <Text style={styles.source}>{h.source[source]}</Text>
</ScrollView>

  {open && <HolidaySheet holiday={open} source={source} onClose={() => setOpen(null)} />}
</View>
);
}

const styles = StyleSheet.create({
  root: { flex: 1 },
  pressed: { opacity: 0.7 },
  yearRow: {
    flexDirection: 'row',
    alignItems: 'center',
    justifyContent: 'space-between',
    paddingHorizontal: 16,
    paddingTop: 10,
    paddingBottom: 4,
  },
  arrow: {
    width: 40,
    height: 40,
    borderRadius: 20,
    alignItems: 'center',
    justifyContent: 'center',
    backgroundColor: 'rgba(255,255,255,0.07)',
  },
  yearText: { alignItems: 'center' },
  year: { fontFamily: fonts.extrabold, fontSize: 26, color: colors.text, ...tabularNums },

```

```

hijriYears: { fontFamily: fonts.semibold, fontSize: 12.5, color: colors.gold, ...tabularNums },
content: { paddingHorizontal: 16, paddingBottom: 18 },
next: {
  marginTop: 10,
  flexDirection: 'row',
  alignItems: 'center',
  gap: 14,
  paddingVertical: 14,
  paddingHorizontal: 16,
  borderRadius: 18,
  backgroundColor: 'rgba(212,168,87,0.16)',
  borderWidth: 1,
  borderColor: 'rgba(212,168,87,0.55)',
},
nextText: { flex: 1, minWidth: 0 },
nextLabel: { fontFamily: fonts.extrabold, fontSize: 11, letterSpacing: 1.1, color: colors.gold },
nextName: { fontFamily: fonts.extrabold, fontSize: 18, color: colors.text, marginTop: 2 },
nextSub: { fontFamily: fonts.medium, fontSize: 12.5, color: colors.textDim, marginTop: 2, ...tabularNums },
},
nextCount: { alignItems: 'flex-end' },
nextNum: { fontFamily: fonts.extrabold, fontSize: 26, lineHeight: 30, color: colors.gold, ...tabularNums },
},
nextDays: { fontFamily: fonts.bold, fontSize: 11.5, color: colors.gold },
nextSoon: { fontFamily: fonts.extrabold, fontSize: 15, color: colors.gold },
empty: { fontFamily: fonts.medium, fontSize: 13.5, color: colors.muted, textAlign: 'center', marginTop: 30 },
monthLabel: { fontFamily: fonts.extrabold, fontSize: 11, letterSpacing: 1.1, color: colors.muted,
paddingTop: 14, paddingBottom: 6, paddingHorizontal: 4 },
card: { borderRadius: 16, backgroundColor: colors.card, borderWidth: 1, borderColor: colors.cardBorder,
overflow: 'hidden' },
row: { flexDirection: 'row', alignItems: 'center', gap: 10, minHeight: 56, paddingVertical: 8,
paddingLeft: 6, paddingRight: 12 },
rowBorder: { borderTopWidth: StyleSheet.hairlineWidth, borderTopColor: 'rgba(255,255,255,0.10)' },
rowNext: { backgroundColor: 'rgba(212,168,87,0.14)' },
past: { opacity: 0.5 },
tile: { width: 58, alignItems: 'center' },
tileDay: { fontFamily: fonts.extrabold, fontSize: 17, lineHeight: 21, color: colors.text, ...tabularNums },
},
tileRange: { fontSize: 14 },
tileWd: { fontFamily: fonts.bold, fontSize: 10.5, color: colors.muted },
expectedText: { color: colors.warn, fontSize: 13.5 },
rowText: { flex: 1, minWidth: 0 },
nameline: { flexDirection: 'row', alignItems: 'baseline', flexWrap: 'wrap', columnGap: 8 },
name: { fontFamily: fonts.bold, fontSize: 14.5, color: colors.text },
arabic: { fontFamily: fonts.arabic, fontSize: 15, color: colors.gold },
sub: { fontFamily: fonts.regular, fontSize: 11.5, color: colors.muted, marginTop: 1, ...tabularNums },
badge: {
  fontFamily: fonts.extrabold,
  fontSize: 10.5,
  color: colors.warn,
  borderWidth: 1,
  borderColor: 'rgba(227,154,75,0.6)',
  borderRadius: 999,
  paddingHorizontal: 8,
  paddingVertical: 3,
  overflow: 'hidden',
},
badgeNext: {
  fontFamily: fonts.extrabold,
  fontSize: 11.5,
  color: colors.goldInk,

```

```

    backgroundColor: colors.gold,
    borderRadius: 999,
    paddingHorizontal: 9,
    paddingVertical: 4,
    overflow: 'hidden',
  },
  source: { fontFamily: fonts.regular, fontSize: 11, color: colors.muted, textAlign: 'center', marginTop:
16, paddingHorizontal: 12 },
});

```

HolidaySheet.tsx src/components/holidays

- Празникът: дати, обяснение, напомняне.

src/components/holidays/HolidaySheet.tsx

149 реда

```

import { Modal, Pressable, ScrollView, StyleSheet, Switch, Text, View } from 'react-native';
import { useSafeAreaInsets } from 'react-native-safe-area-context';

import { DIYANET_OFFICIAL_UNTIL } from '@/domain/hijriTable';
import { HOLIDAYS, reminderDayOf, type Holiday, type HolidaySource } from '@/domain/holidays';
import { useI18n } from '@/i18n';
import { useSettings } from '@/store/settings';
import { colors } from '@/theme/colors';
import { fonts, tabularNums } from '@/theme/typography';

import { fullDate, hijriText, shortDate } from './holidayFormat';

const dayBefore = (d: Date) => new Date(d.getFullYear(), d.getMonth(), d.getDate() - 1, 12);

/** Празникът - докосване в списъка (етап 13): датите, кратко обяснение, напомняне. */
export function HolidaySheet({ holiday: x, source, onClose }: { holiday: Holiday; source: HolidaySource;
onClose: () => void }) {
  const insets = useSafeAreaInsets();
  const { t } = useI18n();
  const h = t.holidays;
  const meta = HOLIDAYS[x.id];
  const on = useSettings((s) => !!s.holidayReminders[x.id]);
  const setReminder = useSettings((s) => s.setHolidayReminder);
  const computed =
    source !== 'ummAlqura' &&
    (x.hijri.year > DIYANET_OFFICIAL_UNTIL.year ||
    (x.hijri.year === DIYANET_OFFICIAL_UNTIL.year && x.hijri.month > DIYANET_OFFICIAL_UNTIL.month));
  const eve = x.id === 'fitr' || x.id === 'adha' ? dayBefore(x.date) : null;

  return (
    <Modal visible transparent animationType="slide" onRequestClose={onClose} statusBarTranslucent>
      <Pressable style={styles.backdrop} onPress={onClose} accessibilityRole="button" accessibilityLabel=
{h.close} />
      <View style={[styles.sheet, { paddingBottom: insets.bottom + 24 }]} accessibilityViewIsModal>
        <View style={styles.handle} />
        <ScrollView showsVerticalScrollIndicator={false} bounces={false}>
          <View style={styles.head}>
            <View style={styles.headText}>
              <Text style={styles.title} accessibilityRole="header">
                {h.names[x.id]}
              </Text>
              <Text style={styles.arabic} importantForAccessibility="no" accessibilityElementsHidden>
                {meta.arabic}
              </Text>
            </View>
          </View>
        </ScrollView>
      </View>
    </Modal>
  );
}

```

```

    </View>
    {meta.days > 1 && <Text style={styles.chip}>{h.daysCount(meta.days)}</Text>}
  </View>

  <View style={styles.box}>
    <View style={styles.boxRow}>
      <Text style={styles.boxLabel}>{meta.days > 1 ? h.dates : h.date}</Text>
      <Text style={[styles.boxValue, styles.bold, x.expected && styles.warn]}>
        {x.expected ? '≈ ' : ''}
        {fullDate(x, t)}
        {x.night ? ` · ${h.evening}` : ''}
      </Text>
    </View>
    <View style={[styles.boxRow, styles.border]}>
      <Text style={styles.boxLabel}>{h.hijri}</Text>
      <Text style={styles.boxValue}>{hijriText(x, t)}</Text>
    </View>
    {eve && (
      <View style={[styles.boxRow, styles.border]}>
        <Text style={styles.boxLabel}>{h.arefe}</Text>
        <Text style={styles.boxValue}>{shortDate(eve, t)}</Text>
      </View>
    )}
  </View>

  {x.expected && <Text style={styles.noteWarn}>{h.expectedNote}</Text>}
  {computed && <Text style={styles.note}>{h.computedNote}</Text>}

  <Text style={styles.about}>{h.about[x.id]}</Text>

  <View style={styles.remind}>
    <View style={styles.remindText}>
      <Text style={styles.remindLabel}>{x.night ? h.remindNight : h.remindDay}</Text>
      <Text style={styles.remindSub}>{h.remindAt(shortDate(reminderDayOf(x), t))}</Text>
    </View>
    <Switch
      value={on}
      onValueChange={({v} => setReminder(x.id, v))}
      trackColor={{ false: 'rgba(255,255,255,0.18)', true: colors.gold }}
      thumbColor="#FFFFFF"
      accessibilityLabel={x.night ? h.remindNight : h.remindDay}
    />
  </View>

  <Text style={styles.source}>{h.source[source]}</Text>
</ScrollView>
</View>
</Modal>
);
}

const styles = StyleSheet.create({
  backdrop: { position: 'absolute', top: 0, left: 0, right: 0, bottom: 0, backgroundColor:
'rgba(3,6,12,0.55)' },
  sheet: {
    position: 'absolute',
    left: 0,
    right: 0,
    bottom: 0,
    maxHeight: '88%',
    backgroundColor: '#16243A',

```

```

borderTopLeftRadius: 24,
borderTopRightRadius: 24,
paddingTop: 10,
paddingHorizontal: 20,
},
handle: { width: 38, height: 4, borderRadius: 2, backgroundColor: 'rgba(255,255,255,0.2)', alignSelf:
'center', marginBottom: 16 },
head: { flexDirection: 'row', alignItems: 'flex-start', gap: 12 },
headText: { flex: 1 },
title: { fontFamily: fonts.extrabold, fontSize: 24, lineHeight: 30, color: colors.text },
arabic: { fontFamily: fonts.arabic, fontSize: 22, lineHeight: 32, color: colors.gold, textAlign: 'left'
},
chip: {
fontFamily: fonts.extrabold,
fontSize: 11.5,
color: colors.muted,
backgroundColor: 'rgba(255,255,255,0.07)',
borderRadius: 999,
paddingHorizontal: 10,
paddingVertical: 5,
overflow: 'hidden',
},
box: { marginTop: 14, borderRadius: 14, backgroundColor: colors.card, borderWidth: 1, borderColor:
colors.cardBorder },
boxRow: { flexDirection: 'row', justifyContent: 'space-between', alignItems: 'center', gap: 12,
paddingHorizontal: 14, paddingVertical: 11 },
border: { borderTopWidth: StyleSheet.hairlineWidth, borderTopColor: 'rgba(255,255,255,0.10)' },
boxLabel: { fontFamily: fonts.medium, fontSize: 13, color: colors.muted },
boxValue: { flexShrink: 1, textAlign: 'right', fontFamily: fonts.bold, fontSize: 14, color: colors.text,
...tabularNums },
bold: { fontFamily: fonts.extrabold },
warn: { color: colors.warn },
noteWarn: { fontFamily: fonts.medium, fontSize: 12.5, lineHeight: 18, color: colors.warn, marginTop: 10
},
note: { fontFamily: fonts.regular, fontSize: 12, lineHeight: 17, color: colors.muted, marginTop: 10 },
about: { fontFamily: fonts.regular, fontSize: 13.5, lineHeight: 20, color: colors.textDim, marginTop: 14
},
remind: {
marginTop: 14,
flexDirection: 'row',
alignItems: 'center',
gap: 12,
paddingHorizontal: 14,
paddingVertical: 12,
borderRadius: 14,
backgroundColor: 'rgba(255,255,255,0.05)',
borderWidth: 1,
borderColor: 'rgba(255,255,255,0.08)',
},
remindText: { flex: 1 },
remindLabel: { fontFamily: fonts.semibold, fontSize: 14, color: colors.text },
remindSub: { fontFamily: fonts.medium, fontSize: 12, color: colors.muted, ...tabularNums },
source: { fontFamily: fonts.regular, fontSize: 11.5, color: colors.muted, textAlign: 'center', marginTop:
12 },
});

```

holidayFormat.ts src/components/holidays

- „Чт, 10 дек.“, „20-22 / Пт-Нд“, „1-3 Шеввал 1447“.

src/components/holidays/holidayFormat.ts

36 реда

```
import type { Holiday } from '@domain/holidays';
import type { Strings } from '@i18n';

/** Плочката отляво: „10“ / „Чт“ или „20-22“ / „Пт-Нд“. */
export function dayTile(h: Holiday, t: Strings): { day: string; wd: string } {
  const wd = t.date.weekdaysShort;
  if (h.end.getTime() === h.date.getTime()) return { day: String(h.date.getDate()), wd: wd[h.date.getDay()] };
  return { day: `${h.date.getDate()}-${h.end.getDate()}`, wd: `${wd[h.date.getDay()]}-${wd[h.end.getDay()]}` };
}

/** „Чт, 10 дек.“ */
export function shortDate(d: Date, t: Strings): string {
  return `${t.date.weekdaysShort[d.getDay()]}, ${d.getDate()} ${t.date.monthsShort[d.getMonth()]}`;
}

/** „Ср, 27 - Сб, 30 май 2026“ / „Чт, 10 дек. 2026“ */
export function fullDate(h: Holiday, t: Strings): string {
  const y = h.end.getFullYear();
  if (h.end.getTime() === h.date.getTime()) return `${shortDate(h.date, t)} ${y}`;
  const wd = t.date.weekdaysShort;
  const sameMonth = h.date.getMonth() === h.end.getMonth();
  const from = sameMonth ? `${wd[h.date.getDay()]}, ${h.date.getDate()}` : shortDate(h.date, t);
  return `${from} - ${shortDate(h.end, t)} ${y}`;
}

/** „26 Реджеб 1447“ / „1-3 Шеввал 1447“ */
export function hijriText(h: Holiday, t: Strings): string {
  const days = Math.round((h.end.getTime() - h.date.getTime()) / 86400000);
  const day = days > 0 ? `${h.hijri.day}-${h.hijri.day + days}` : String(h.hijri.day);
  return `${day} ${t.hijriMonths[h.hijri.month - 1]} ${h.hijri.year}`;
}

/** Редът под името: „вечерта · 26 Реджеб 1447“; с датата - „Чт, 10 дек. · вечерта · 1 Реджеб 1448“. */
export function hijriLine(h: Holiday, t: Strings, withDate: boolean): string {
  return [withDate ? shortDate(h.date, t) : null, h.night ? t.holidays.evening : null, hijriText(h, t)].filter(Boolean).join(' · ');
}
```

index.tsx src/app/(tabs)

- „Днес“ – датата по Хиджра от избрания календар.

src/app/(tabs)/index.tsx

94 реда

```
import { router } from 'expo-router';
import { useCallback, useState } from 'react';
import { ScrollView, StyleSheet, View } from 'react-native';
import { useSafeAreaInsets } from 'react-native-safe-area-context';

import { NextPrayerHero } from '@components/NextPrayerHero';
import { NoticeBanner } from '@components/NoticeBanner';
import { PhaseBackground } from '@components/PhaseBackground';
```

```

import { PrayerList } from '@components/PrayerList';
import { useTabBarHeight } from '@components/TabBar';
import { Toast } from '@components/Toast';
import { TopBar } from '@components/TopBar';
import { formatGregorianShort } from '@domain/format';
import { formatHijri } from '@domain/hijri';
import type { PrayerId } from '@domain/prayers';
import { useNow } from '@hooks/useNow';
import { usePrayerSchedule } from '@hooks/usePrayerSchedule';
import { useI18n } from '@i18n';
import { useLocationStatus } from '@services/location';
import { requestPermission, useNotificationStatus } from '@services/notifications';
import { useAlertPrefs } from '@store/alertPrefs';
import { selectHijriCalendar, selectLocation, useSettings } from '@store/settings';

export default function TodayScreen() {
  const insets = useSafeAreaInsets();
  const { t, pick } = useI18n();
  const tabBarHeight = useTabBarHeight();

  const now = useNow();
  const schedule = usePrayerSchedule(now);
  const location = useSettings(selectLocation);
  const hijriAdjust = useSettings((s) => s.hijriAdjust);
  const hijriCalendar = useSettings(selectHijriCalendar);
  const autoLocation = useSettings((s) => s.autoLocation);
  const hasPlace = useSettings((s) => s.gpsLocation !== null || s.manualLocation !== null);
  // „idle“ - само в първия миг преди първото търсене; броим го като търсене, за да не мигне „Избери място“
  const locating = useLocationStatus((s) => s.status === 'locating' || s.status === 'idle') &&
  autoLocation;

  const cycle = useAlertPrefs((s) => s.cycle);
  const notificationsOff = useNotificationStatus(
    (s) => s.checked && (s.permission === 'denied' || s.permission === 'undetermined'),
  );
  const [toast, setToast] = useState<{ id: number; text: string } | null>(null);

  const onBellPress = useCallback(
    (id: PrayerId) => {
      // Без разрешение камбанката не се сменя: първо питаме за разрешение (или отваряме
      // настройките на телефона). Съобщението се показва СЛЕД въпроса – иначе прозорецът
      // на системата го скрива и потребителят не успява да го прочете.
      if (notificationsOff) {
        requestPermission().then((p) => {
          setToast({
            id: Date.now(),
            text: p === 'granted' ? t.notifications.allowedToast : t.notifications.bellDenied,
          });
        });
        return;
      }
      const mode = cycle(id);
      const text = mode === 'adhan' ? t.alarmToast(t.prayers[id]) : t.toast(t.prayers[id], t.alert[mode]);
      setToast({ id: Date.now(), text });
    },
    [cycle, t, notificationsOff],
  );

  return (
    <View style={styles.root}>
      <PhaseBackground phase={schedule.current.id} />
    </View>
  );
}

```

```

<ScrollView
  contentContainerStyle={{ paddingTop: insets.top, paddingBottom: tabBarHeight + 16 }}
  showsVerticalScrollIndicator={false}
>
  <TopBar
    city={pick(location.names)}
    gregorian={formatGregorianShort(now, t.date)}
    hijri={formatHijri(now, t.hijriMonths, hijriAdjust, hijriCalendar)}
    onCityPress={() => router.push('/place')}
    locating={locating}
    locatingFirstTime={!hasPlace}
  />
  <NoticeBanner />
  <NextPrayerHero schedule={schedule} />
  <PrayerList schedule={schedule} onBellPress={onBellPress} />
</ScrollView>

  <Toast message={toast} bottom={tabBarHeight + 14} />
</View>
);
}

const styles = StyleSheet.create({
  root: { flex: 1 },
});

```

`_layout.tsx` src/app

- Новият екран.

src/app/_layout.tsx

99 реда

```

import { Amiri_400Regular, Amiri_700Bold } from '@expo-google-fonts/amiri';
import {
  Manrope_400Regular,
  Manrope_500Medium,
  Manrope_600SemiBold,
  Manrope_700Bold,
  Manrope_800ExtraBold,
} from '@expo-google-fonts/manrope';
import { useFonts } from 'expo-font';
import { Stack } from 'expo-router';
import * as SplashScreen from 'expo-splash-screen';
import { StatusBar } from 'expo-status-bar';
import { useEffect, useState } from 'react';

import { useLocationUpdater } from '@/hooks/useLocationUpdater';
import { useNotificationScheduler } from '@/hooks/useNotificationScheduler';
// дефинира фоновата задача при зареждане на кода (виж файла)
import '@/services/backgroundTask';
import { configureNotificationHandler } from '@/services/notifications';
import { useAlertPrefs } from '@/store/alertPrefs';
import { useSettings } from '@/store/settings';
import { useSounds } from '@/store/sounds';
import { colors } from '@/theme/colors';

// Splash екранът стои, докато шрифтовете и запазените настройки се заредят -
// иначе за миг би се показвала София по подразбиране вместо твоето място.
SplashScreen.preventAutoHideAsync();

```

```

// известие, дошло докато приложението е отворено, се показва като обикновено – със звук
configureNotificationHandler();

function useStoresHydrated(): boolean {
  const all = () =>
    useSettings.persist.hasHydrated() && useAlertPrefs.persist.hasHydrated() &&
    useSounds.persist.hasHydrated();
  const [done, setDone] = useState(all);
  useEffect(() => {
    const check = () => setDone(all());
    const a = useSettings.persist.onFinishHydration(check);
    const b = useAlertPrefs.persist.onFinishHydration(check);
    const c = useSounds.persist.onFinishHydration(check);
    check();
    // предпазител: ако четенето от паметта на телефона се провали, zustand не съобщава
    // „готово“ – след 2 сек. приложението се показва с настройките по подразбиране
    const timer = setTimeout(() => setDone(true), 2000);
    return () => {
      a();
      b();
      c();
      clearTimeout(timer);
    };
  }, []);
  return done;
}

export default function RootLayout() {
  const [loaded, error] = useFonts([
    Manrope_400Regular,
    Manrope_500Medium,
    Manrope_600SemiBold,
    Manrope_700Bold,
    Manrope_800ExtraBold,
    Amiri_400Regular,
    Amiri_700Bold,
  ]);

  const hydrated = useStoresHydrated();
  const ready = (loaded || !error) && hydrated;
  useNotificationScheduler(hydrated);

  useEffect(() => {
    if (ready) SplashScreen.hideAsync();
  }, [ready]);

  // GPS при стартиране и при връщане в приложението (ако мястото е автоматично)
  useLocationUpdater();

  if (!ready) return null;

  return (
    <>
      <StatusBar style="light" />
      <Stack
        screenOptions={{
          headerShown: false,
          contentStyle: { backgroundColor: colors.base },
        }}
      >
        <Stack.Screen name="(tabs)" />
      </Stack>
    </>
  );
}

```

```

    { /* изборът на място се отваря отдолу нагоре като модален екран */ }
    <Stack.Screen name="place" options={{ presentation: 'modal' }} />
    <Stack.Screen name="about" />
    <Stack.Screen name="diagnostics" />
    <Stack.Screen name="battery" />
    <Stack.Screen name="sound" />
    <Stack.Screen name="navigation" />
    <Stack.Screen name="holiday-source" />
  </Stack>
</>
);
}

```

strings.ts src/i18n

- Имената и обясненията на празниците на български и английски.

src/i18n/strings.ts

1192 реда

```

import type { CalculationMethodId, HighLatRuleId } from '@domain/calc';
import type { PrayerId } from '@domain/prayers';
import type { AlertMode } from '@store/alertPrefs';
import type { BatteryBrand } from '@domain/device';
import type { HolidayId } from '@domain/holidays';

/**
 * Всички текстове на приложението на двата езика.
 * `en` е типизиран като `Strings` (формата на `bg`), затова TypeScript
 * дава грешка, ако някой ключ липсва в единия от езиците.
 */
export const bg = {
  prayers: {
    fajr: 'Фаджр',
    sunrise: 'Изгрев',
    dhuhhr: 'Зухр',
    asr: 'Асп',
    maghrib: 'Магриб',
    isha: 'Иша',
  } as Record<PrayerId, string>,

  alert: {
    off: 'без известие',
    notify: 'известие',
    adhan: 'аларма',
  } as Record<AlertMode, string>,

  tabs: {
    today: 'Днес',
    qibla: 'Кибла',
    mosques: 'Джамии',
    month: 'Месец',
    settings: 'Настройки',
  },

  hero: {
    next: 'Следваща',
    at: (time: string) => `в ${time}`,
    tomorrowAt: (time: string) => `утре в ${time}`,
    remaining: (time: string) => `Остават ${time}`,
  },

```

```

nowPill: 'СЕГА',

busy: 'Зарежда...',
locating: 'Определям мястото...',
choosePlace: 'Избери място',
noPlaceBanner: 'Мястото не е определено – часовете са за София. Докосни, за да избереш място.',

ally: {
  city: (city: string) => `Град: ${city}. Смени града`,
  locating: 'Определям местоположението',
  bell: (prayer: string, mode: string) => `${prayer}: ${mode}. Докосни за смяна`,
},

toast: (prayer: string, mode: string) => `${prayer}: ${mode}`,
alarmToast: (prayer: string) => `${prayer}: аларма – звучи и при тих режим`,

placeholder: {
  qibla: 'Компас към Кябе. Идва в етап 7 от плана.',
},

month: {
  dayColumn: 'Ден',
  hijriColumn: 'Хиджра',
  today: 'Днес',
  prev: 'Предишен месец',
  next: 'Следващ месец',
  friday: 'петък',
  sourceMufti: 'Източник: Главно мюфтийство на Република България',
  sourceCalc: (method: string) => `Изчислено по метод ${method}`,
},

settings: {
  title: 'Настройки',
  groupPlace: 'Място',
  groupTimes: 'Часове',
  groupCorrections: 'Корекции',
  groupApp: 'Приложение',
  about: 'За приложението',
  change: 'Смени мястото',
  auto: 'Автоматично (GPS)',
  autoHint: 'Мястото се определя при отваряне на приложението.',
  locating: 'Определям местоположението...',
  denied: 'Няма разрешение за местоположение.',
  openSettings: 'Отвори настройките на телефона',
  unavailable: 'Местоположението не може да се определи. Показва се последното известно място.',
  refresh: 'Обнови местоположението',
  sourceMufti: 'Календар на Главно мюфтийство',
  sourceMuftiNote: 'В България часовете са по официалния календар. Методът и Аср се избират само извън
България.',
  sourceCalc: 'Изчисление за мястото',
  method: 'Метод',
  methodAuto: 'Автоматично',
  methodAutoValue: (method: string) => `Авто · ${method}`,
  methodAutoDesc: (country: string, method: string) => `според държавата: ${country} → ${method}`,
  asr: 'Аср',
  shafi: 'Шафии, Малики, Ханбали',
  hanafi: 'Ханафи',
  shafiDesc: 'Сянката е равна на предмета – „първият“ Аср, като в календара на Диянет',
  hanafiDesc: 'Сянката е два пъти по-дълга – около час по-късно',
  asrNote: 'Шафии, Малики и Ханбали смятат Аср еднакво; само при Ханафи е по-късно.',
}

```

```

polarNote: (lat: string) =>
    `На ${lat}° ширина има полярен ден и полярна нощ. В дните без изгрев или залез часовете са изчислени
    за 65° ширина – най-близките места с нормален ден и нощ. Свърете с местната джамия.` ,
    highLat: 'Северни ширини',
    highLatNote: (lat: string) =>
        `На ${lat}° ширина през лятото няма истински здрач и правилото определя Фаджр и Иша. Свърете с
        местната джамия.` ,
    offsetsNote: 'Ако джамията ти е с няколко минути по-различна, нагласи тук. Важи за всички екрани.',
    minutes: (n: number) => (n === 0 ? '0 мин' : `${n > 0 ? '+' : ''}${n} мин`),
    hijri: 'Дата по Хиджра',
    days: (n: number) => (n === 0 ? '0 дни' : `${n > 0 ? '+' : ''}${n} ${Math.abs(n) === 1 ? 'ден' :
    'дни'}`),
    hijriNote: 'Ако датата по Хиджра се разминава с обявената от Мюфтийството.',
    hijriNoteAbroad: 'Ако датата по Хиджра се разминава с обявената от местната джамия.',
    reset: 'Нулирай корекциите',
    decrease: (label: string) => `Намали: ${label}`,
    increase: (label: string) => `Увеличи: ${label}`,
},

widget: {
    next: 'Следваща',
    tomorrowShort: (time: string) => `утре ${time}`,
    until: (name: string) => `остават до ${name}`,
    left: 'остават',
    empty: 'Отвори Езан, за да се покажат часовете.',
},

notifications: {
    prayerTitle: (prayer: string, time: string) => `${prayer} – ${time}`,
    prayerBody: (prayer: string, arabic: string, place: string) => `Време е за намаза ${prayer} (${arabic})
    • ${place}`,
    sunriseBody: (place: string) => `Изгрев – край на времето за Фаджр • ${place}`,
    nextLine: (prayer: string, time: string, tomorrow: boolean) =>
        `Следваща: ${prayer} в ${time}${tomorrow ? ' (утре)' : ''}`,
    reminderTitle: (prayer: string, minutes: number, time: string) => `${prayer} след ${minutes} мин. –
    ${time}`,
    reminderBody: (place: string) => `Време е за абдест и подготовка за намаза • ${place}`,
    refreshTitle: 'Отвори Езан',
    refreshBody: 'Отвори приложението, за да продължат известията за намаз.',
    channelPrayer: 'Молитви',
    channelAdhan: 'Аларма',
    channelReminder: 'Напомняния',
    group: 'Известия',
    status: 'Разрешение',
    granted: 'Дадено',
    denied: 'Отказано',
    undetermined: 'Не е дадено',
    allow: 'Разреша известията',
    deniedNote: 'Известията са изключени в настройките на телефона – без тях известията и алармите за намаз
    няма да работят.',
    webNote: 'Известията работят само в приложението на телефона.',
    reminder: 'Напомняне преди молитвата',
    reminderOff: 'Без напомняне',
    reminderValue: (n: number) => `${n} мин. преди`,
    vibrate: 'Вибрация',
    exact: 'Точни известия',
    exactOff: 'Изключени',
    exactNote:
        `Без разрешението „Аларми и напомняния“ Android може да забави известията с няколко минути.`,
    exactAllow: 'Разреша „Аларми и напомняния“',
    scheduled: (n: number, until: string) => `Планирани ${n} известия – до ${until}`.

```

```

scheduledNone: 'Няма планирани известия – всички камбанки на „Днес“ са изключени.',
testTitle: 'Пробно известие',
testBody: 'Така ще изглеждат известията за намаз.',
bellDenied: 'Известията не са разрешени – известията и алармите няма да работят.',
allowedToast: '✓ Известията са разрешени',
exactBanner: 'Известията може да закъснеят с няколко минути. Докосни, за да разрешиш точни известия.',
deniedBanner: 'Известията не са разрешени – известията и алармите няма да работят. Докосни, за да ги разрешиш.',
diagnostics: 'Проверка на известията',
diag: {
  title: 'Проверка на известията',
  subtitle: 'Дали телефонът показва известията на Езан навреме – и какво да оправиш, ако не.',
  refresh: 'Обнови',
  updated: (time: string) => `Обновено в ${time}`,
  status: 'Състояние',
  permOk: 'Известията са разрешени',
  permBad: 'Известията не са разрешени',
  exactOk: 'Точни известия („Аларми и напомняния“)',
  exactBad: 'Точните известия са изключени – може да закъсняват',
  allow: 'Разреши',
  scheduledOk: (n: number, until: string) => `Планирани ${n} известия · до ${until}`,
  scheduledNone: 'Няма планирани известия – камбанките на „Днес“ са изключени',
  test: 'Пробно известие',
  testNotification: 'Известие',
  testIn: (s: number) => `след ${s} сек.`,
  testHint: 'Идва след 10 сек. Заклучи телефона, за да го видиш и чуеш както ще идва. Звукът на алармата се преслушва в Настройки → Звуци.',
  recent: 'Последни известия',
  recentNone: 'Още няма показани известия от Езан.',
  onTime: 'навреме',
  late: (s: number) => (s < 60 ? `+${s} сек.` : `+${Math.round(s / 6) / 10} мин.`),
  plannedShown: (planned: string, shown: string) => `планирано ${planned} · показано ${shown}`,
  upcoming: 'Следващи известия',
  kindAlarm: 'Аларма',
  kindNotification: 'Известие',
  kindReminder: 'Напомняне',
  kindOther: 'Друго',
  tech: 'Технически данни',
  device: 'Телефон',
  channelLine: (importance: number, sound: string, vibrate: boolean, usage: string) =>
    `важност ${importance} · звук ${sound} · вибрация ${vibrate ? 'да' : 'не'} · поток ${usage}`,
  reset: 'Пренасрочи всички известия',
  resetHint: 'Трие всички известия на Езан и ги планира наново по текущите настройки. Ползвай го, ако нещо изглежда объркано.',
  resetDone: '✓ Всички известия са планирани наново.',
  fullScreenOk: 'Аларма на цял екран',
  fullScreenBad: 'Аларма на цял екран е изключена',
  batteryOk: 'Работа на заден план без ограничения',
  batteryBad: 'Телефонът ограничава работата на заден план',
  fix: 'Поправи',
},
fullScreen: 'Аларма на цял екран',
fullScreenOn: 'Разрешена',
fullScreenOff: 'Изключена',
fullScreenNote: 'Без нея при заключен телефон езанът пак звучи, но се вижда само известие – не целият екран.',
fullScreenAllow: 'Разреши „Аларма на цял екран“',
exactOn: 'Разрешени',
background: 'Работа на заден план',
backgroundOk: 'Без ограничения',
backgroundBad: 'Ограничена',

```

```

batteryTitle: 'Алармите може да закъснеят',
batteryText: 'Телефонът може да спре приложението, за да пести батерия.',
batteryFix: 'Поправи',
},

qibla: {
  title: 'Кибла',
  fromNorth: 'от север',
  toMecca: 'до Мека',
  km: 'км',
  aligned: '✓ Обърнат си към Кибла',
  turnLabel: (direction: 'left' | 'right') => `Завърти се ${direction === 'right' ? 'надясно' :
'наляво'}`,
  starting: 'Компасът се включва...',
  quality: { high: 'висока', medium: 'средна', low: 'ниска' } as Record<'high' | 'medium' | 'low',
string>,
  accuracy: (q: string) => `Компас: ${q} точност`,
  tip: 'Дръж телефона хоризонтално, с екрана нагоре. Метал и магнити наблизко объркват компаса.',
  calibrateTitle: 'Калибрирай компаса',
  calibrateText: 'Движи телефона като осмица във въздуха 2-3 пъти. Дръж го далеч от метал, магнити и
магнитни каълфи.',
  noCompassTitle: 'Няма компас',
  noCompassText: (deg: number) =>
    `Този телефон няма магнитен сензор. Киблата е на ${deg}° от север: застани с лице на север и се
завърти надясно на ${deg}°`,
  permissionTitle: 'Нужно е разрешение',
  permissionText: 'На iPhone компасът работи само с разрешение за местоположение.',
  allowLocation: 'Разреши местоположението',
  letters: ['C', 'И', 'Ю', 'З'] as readonly string[],
  allyDial: (bearing: number) => `Компас. Киблата е на ${bearing}° от север`,
  // {t} е часът - показва се удебелен и златен
  sunToday: 'Днес в {t} слънцето е точно в посоката на Киблата',
  sunTomorrow: 'Утре в {t} слънцето ще е точно в посоката на Киблата',
  sunNow: 'Сега слънцето е точно в посоката на Киблата',
  sunrise: (azimuth: number, degrees: number, side: 'left' | 'right' | 'same') =>
    side === 'same'
      ? `Изгрев на ${azimuth}° - точно в посоката на Киблата`
      : `Изгрев на ${azimuth}° - Киблата е ${degrees}° ${side === 'right' ? 'вдясно' : 'вляво'} от него`,
},

sounds: {
  group: 'Звуци',
  notifyGroup: 'Напомняне и известие',
  notifyRow: 'Звук на известията',
  noteAndroid: 'Звукът свири, когато камбанката на молитвата е „аларма“. Без „Аларми и напомняния“ -
краткият звук (до 30 сек.)',
  noteIos: 'Звукът свири, когато камбанката на молитвата е „аларма“. На iPhone звукът е до 30 сек.',
  fullOffTitle: 'Пълният езан е изключен',
  fullOffText: 'Без „Аларми и напомняния“ свири само краткият звук (до 30 сек.). За пълния езан разреши
„Аларми и напомняния“.',
  allow: 'Разреши',
  title: (prayer: string) => `Звук за ${prayer}`,
  notifyTitle: 'Звук на известията',
  fullSection: 'Пълен звук - аларма',
  fullHint: 'Свири докрай; „Спри“ го прекъсва.',
  shortSection: 'Кратък звук - до 30 сек.',
  shortHintAndroid: 'Когато „Аларми и напомняния“ е изключено. Своят звук се скъсява до 30 сек. - на
пауза, с плавно заглъхване.',
  iosSection: 'Звук - до 30 сек.',
  iosHint: 'На iPhone звукът на известие е най-много 30 сек. По-дългите записи се скъсяват на пауза, с
плавно заглъхване.',

```

```

notifyHint: 'За известията и напомнянията преди молитвата.',
byDefault: 'по подразбиране',
mine: 'МОЙ',
add: 'Добави свой звук',
addFullHint: 'mp3, m4a, mp4, wav, ogg, flac · от телефона',
addShortHint: 'mp3, m4a, mp4, wav... · по-дълъг – скъсява се до 30 сек.',
addNotifyHint: 'до 10 сек. · mp3, m4a, mp4, wav...',
tooLongNotify: (name: string, dur: string) =>
  `„${name}“ е ${dur} – звукът за известия може да е до 10 сек. Избери по-кратък.`,
adding: 'Добавяне...',
workingShort: 'Изрязване на откъса...',
workingFull: 'Добавяне на звука...',
workingHint: 'Търси се паузата до 30-ата секунда. Може да отнеме няколко секунди.',
applyAll: 'Същият звук за всички молитви',
apply: 'Приложи',
applied: '✓ Приложено за всички молитви',
iosFuture: 'Пълният езан на iPhone ще дойде с AlarmKit (iOS 26) – етап 9.',
added: (name: string) => `✓ Добавен: ${name}`,
addedTrimmed: (name: string, dur: string) =>
  `✓ Добавен: ${name}. Краткият звук е до 30 сек. – ще свирят първите ${dur}, до паузата, с плавно
заглъхване.`,
trimmed: (dur: string, of: string) => `свирят първите ${dur} от ${of}`,
tooLongFull: (name: string) => `„${name}“ е по-дълъг от 15 мин.`,
tooBig: 'Файлът е по-голям от 60 MB.',
unreadable: 'Този файл не може да се прочете като звук.',
unsupported: 'Своите звуци се добавят само в приложението на телефона.',
deleteTitle: (name: string) => `Да махна ли „${name}“?`,
deleteText: 'Молитвите с този звук се връщат към звука по подразбиране.',
delete: 'Махни',
cancel: 'Отказ',
play: (name: string) => `Пусни ${name}`,
stop: (name: string) => `Спри ${name}`,
},

alarm: {
  app: 'Езан',
  title: (prayer: string) => `Време е за ${prayer}`,
  stop: 'Спри езана',
  stopAlarm: 'Спри алармата',
  mute: 'Заглуши звука',
  muteShort: 'Заглуши',
  close: 'Затвори',
  soundName: 'Езан',
  channel: 'Аларма за молитва',
},

battery: {
  title: 'Работа на заден план',
  lede: 'За да пести батерия, телефонът може да „приспи“ Езан и алармата да закъснее или да не звънне.
Разрешавай на приложението да работи без ограничения.',
  yourPhone: 'Твоят телефон',
  open: 'Отвори настройките на Езан',
  openList: 'Списък „Оптимизация на батерията“',
  now: 'Сега',
  ok: 'Без ограничения ✓',
  bad: 'ограничено',
  unknown: 'не може да се провери',
  otherPhone: 'Друг телефон?',
  more: 'Още съвети за всяка марка: dontkillmyapp.com',
  brands: {
    samsung: 'Samsung',
  }
}

```

```

xiaomi: 'Xiaomi / Redmi / POCO',
huawei: 'Huawei / Honor',
oppo: 'OPPO / Realme / OnePlus',
other: 'Друг',
} as Record<BatteryBrand, string>,
// [текст, път (по желание), бележка (по желание)]
steps: {
  samsung: [
    ['Натисни бутона долу – отварят се настройките на Езан.'],
    ['Отвори', 'Батерия'],
    ['Избери', 'Без ограничения', 'Не „Оптимизирана“ и не „Ограничена“'],
    ['По желание:', 'Настройки → Батерия → Ограничения на фона', 'Езан да не е в „Приложения в дълбок
сън“ и „Приложения в сън“'],
  ],
  xiaomi: [
    ['Натисни бутона долу – отварят се настройките на Езан.'],
    ['Отвори', 'Пестене на батерия', 'Избери „Без ограничения“'],
    ['Включи', 'Автоматично стартиране'],
    ['По желание: в последните приложения задръж картата на Езан и я заключи (катиначе).'],
  ],
  huawei: [
    ['Отвори', 'Настройки → Батерия → Стартиране на приложения'],
    ['Намери Езан и изключи', 'Управлявай автоматично'],
    ['В прозореца включи и трите:', 'Автоматично стартиране · Вторично стартиране · Работа на заден
план'],
  ],
  oppo: [
    ['Натисни бутона долу – отварят се настройките на Езан.'],
    ['Отвори', 'Използване на батерията'],
    ['Включи', 'Разреши работа на заден план', 'и „Автоматично стартиране“, ако го има.'],
  ],
  other: [
    ['Натисни бутона долу – отварят се настройките на Езан.'],
    ['Отвори', 'Батерия'],
    ['Избери', 'Без ограничения', 'На някои телефони се казва „Не оптимизирай“'],
  ],
} as Record<BatteryBrand, readonly (readonly string[])[]>,
},

about: {
  title: 'За приложението',
  name: 'Езан',
  back: 'Настройки',
  version: (v: string) => `Версия ${v}`,
  tagline: 'Часове за намаз, напомняния и езан',
  groupTimes: 'Източници на часовете',
  groupPlaces: 'Данни за местата',
  groupPrivacy: 'Поверителност',
  groupMap: 'Картата и джамиите',
  sources: {
    mufti: { name: 'Главно мюфтийство на Република България', desc: 'Календар за намаз – часовете в
България' },
    adhan: { name: 'adhan', desc: 'Изчисление извън България · лиценз MIT' },
    osm: { name: 'OpenStreetMap', desc: 'Населени места в България · © OpenStreetMap contributors, ODbL'
  },
  ne: { name: 'Natural Earth', desc: 'Граница на България · public domain' },
  geonames: { name: 'GeoNames', desc: 'Градове по света – името и държавата без интернет · CC BY 4.0'
  },
  praytimes: { name: 'PrayTimes.org', desc: 'Записите на езан: Мека, Медина, Ал-Акса' },
  overpass: { name: 'OpenStreetMap · Overpass API', desc: 'Джамиите наблизко · © OpenStreetMap
contributors, ODbL' },

```

```

    openfreemap: { name: 'OpenFreeMap', desc: 'Картата · © OpenMapTiles, данни © OpenStreetMap' },
  },
  privacy:
    'Местоположението се ползва на телефона – за да се изберат населеното място и часовете. Извън България телефонът пита своята картова услуга за името на града. За „Джамии наблизо“ до OpenStreetMap се изпраща само приблизителен район (около 1 км), а картата се зарежда от OpenFreeMap.',
  },

  mosques: {
    title: 'Джамии наблизо',
    count: (n: number, dist: string) => `${n} · до ${dist}`,
    next: 'Следваща:',
    nextAt: (prayer: string, time: string) => `${prayer} в ${time}`,
    nextIn: (left: string) => `след ${left}`,
    walk: (min: number) => `${min} мин пеша`,
    drive: (min: number) => `с кола ~${min} мин`,
    unnamed: 'Джамия',
    unnamedSub: 'без име в OpenStreetMap',
    locating: 'Търся къде си...',
    loading: 'Търся джамиите наоколо...',
    noneNear: (dist: string) => `Няма джамии до ${dist}`,
    noneFar: (dist: string) => `Няма джамии и до ${dist}`,
    noneHint: 'Картата е на OpenStreetMap – в малките места не всяка джамия е отбелязана.',
    searchFar: (dist: string) => `Търси до ${dist}`,
    within: (dist: string) => `до ${dist}`,
    offlineTitle: 'Няма интернет.',
    offlineText: (when: string) => `Показвам последното търсене тук – от ${when}. Упътването ще работи, щом има връзка.`,
    today: 'днес',
    yesterday: 'вчера',
    errorOffline: 'Няма интернет',
    errorOfflineText: 'Джамиите се търсят в OpenStreetMap – нужна е връзка.',
    errorServer: 'Сървърът не отговаря',
    errorServerText: 'OpenStreetMap (Overpass) е претоварен. Опитай пак след минута.',
    retry: 'Опитай пак',
    noGps: (place: string) => `Без достъп до местоположението – търся около ${place}.`,
    allow: 'Разреши',
    myLocation: 'Моето място',
    locate: 'Към моето място',
    back: 'Назад към списъка',
    reach: (min: number) => `ще стигнеш за ${min} мин`,
    directions: (app: string) => `Упътване с ${app}`,
    directionsAuto: 'Упътване',
    withApp: (app: string) => `С ${app} · смени в Настройки → Навигация`,
    withAuto: 'С картите на телефона · смени в Настройки → Навигация',
    share: 'Сподели',
    copyAddress: 'Копирай адреса',
    copyCoords: 'Копирай координатите',
    copied: 'Копирано',
    noAddress: 'Няма адрес в OpenStreetMap',
    allyRow: (name: string, dist: string, time: string) => `${name}, ${dist}, ${time}`,
    allyMap: (n: number) => `Карта с ${n} джамии`,
    attribution: 'OpenFreeMap © OpenMapTiles © OpenStreetMap',
  },

  nav: {
    title: 'Навигация',
    back: 'Настройки',
    intro: 'С кое приложение да се отваря „Упътване“ до джамията.',
    auto: 'Автоматично',
    autoAndroid: 'Картите по подразбиране на телефона',
  }
}

```

```

autoIos: 'Apple Maps',
walkDrive: 'Пеша и с кола',
driveOnly: 'Само с кола',
notInstalled: 'не е инсталирано',
organic: 'OpenStreetMap · и без интернет',
huawei: 'на Huawei',
download: 'Изтегли',
note: 'Waze води само с кола – и за джамия на 300 м ще те упъти с колата. Ако избраното приложение
липсва, „Упътване“ пита с какво да отвори.',
sheetTitle: (name: string) => `Упътване до ${name}`,
sheetSub: (dist: string) => `${dist} · с кое приложение?`,
walk: 'пеша',
drive: 'с кола',
remember: 'Запомни избора',
rememberHint: 'Смяна: Настройки → Навигация',
missing: (app: string) => `${app} вече не е на телефона – избери друго приложение.`,
group: 'Джамии наблизно',
row: 'Навигация',
rowNote: 'С кое приложение се отваря „Упътване“ до джамия.',
ask: 'Пита при упътване',
close: 'Затвори',
},

holidays: {
  tabTimes: 'Часове',
  tabHolidays: 'Празници',
  hijriYears: (a: number, b: number) => (a === b ? `${a} по Хиджра` : `${a} – ${b} по Хиджра`),
  next: 'Следващ',
  today: 'днес',
  tomorrow: 'утре',
  daysWord: (n: number): string => (n === 1 ? 'ден' : 'дни'),
  inDays: (n: number) => (n === 0 ? 'днес' : n === 1 ? 'утре' : `след ${n} дни`),
  evening: 'вечерта',
  daysCount: (n: number) => `${n} дни`,
  expected: 'очаквана',
  expectedNote:
    'Очаквана – началото на Рамазан и Байрамите тук се обявява след наблюдение на луната, вечерта преди.
    Датата може да се промени с 1 ден.',
  computedNote: 'Диянет е обявил датите до началото на 2028 г.; тази е по същите астрономически
  правила.',
  noData: 'Няма данни за тази година.',
  dates: 'Дати',
  date: 'Дата',
  hijri: 'По Хиджра',
  arefe: 'Арефе',
  remindNight: 'Напомни ми същата вечер',
  remindDay: 'Напомни ми вечерта преди',
  remindAt: (date: string) => `${date} · при Магриб`,
  source: {
    mufti: 'Източник: Главно мюфтийство (календарът на Диянет)',
    diyanet: 'Източник: Диянет (Турция)',
    ummalqura: 'Източник: Умм ал-Кура (Саудитска Арабия)',
  } as Record<'mufti' | 'diyanet' | 'ummalqura', string>,
  names: {
    threeMonths: 'Трите свещени месеца',
    regaib: 'Нощ Регаиб',
    miraj: 'Нощ Мирадж',
    berat: 'Нощ Бераат',
    ramadan: 'Начало на Рамазан',
    qadr: 'Кадир гедже',
    fitrEve: 'Арефе',
  }
}

```

```

    fitr: 'Рамазан Байрам',
    arafah: 'Арефе',
    adha: 'Курбан Байрам',
    newYear: 'Нова хиджри година',
    ashura: 'Ашура',
    mawlid: 'Мевлид',
  } as Record<HolidayId, string>,
  about: {
    threeMonths: 'Започват свещените месеци Реджеб, Шабан и Рамазан.',
    regaib: 'Нощта срещу първия петък на месец Реджеб – първата от свещените нощи на трите месеца.',
    miraj: 'Нощта на Ибра и Мирадж – нощното пътуване на Пророка ﷺ от Мека до Йерусалим и възнесението му.',
    berat: 'Нощта срещу 15 Шабан – нощ на опрощение и молитва.',
    ramadan: 'Първият ден на говеенето. Първият теравих намаз е вечерта преди.',
    qadr: 'Нощта на предопределението, „по-добра от хиляда месеца“. Отбелязва се в нощта срещу 27 Рамазан.',
    fitrEve: 'Последният ден на Рамазан – денят преди Рамазан Байрам.',
    fitr: 'Празникът след края на Рамазан. Започва с байрам намаза сутринта на първия ден.',
    arafah: 'Денят, в който поклонниците стоят на Арафат – денят преди Курбан Байрам.',
    adha: 'Празникът на жертвоприношението. Започва с байрам намаза сутринта на първия ден.',
    newYear: '1 Мухаррем – началото на годината по Хиджра.',
    ashura: '10 Мухаррем – ден на говеене и възпоменание.',
    mawlid: 'Нощта на рождението на Пророка Мухаммед ﷺ – срещу 12 Ребиул-евел.',
  } as Record<HolidayId, string>,
  notifyTonight: (name: string) => `Тази вечер: ${name}`,
  notifyTomorrow: (name: string) => `Утре: ${name}`,
  allyRow: (name: string, date: string, hijri: string) => `${name}, ${date}, ${hijri}`,
  prevYear: 'Предишна година',
  nextYear: 'Следваща година',
  close: 'Затвори',
},

holidaySource: {
  title: 'Дати на празниците',
  back: 'Настройки',
  intro:
    'Чии дати да показва календарът. Различните общности понякога празнуват в различни дни – и двете дати са верни за своята общност.',
  auto: 'Автоматично',
  autoDesc: (source: string) => `Според мястото: ${source}`,
  mufti: 'Главно мюфтийство (България)',
  muftiDesc: 'Обявени предварително – същият календар като Диянет',
  diyanet: 'Диянет (Турция)',
  diyanetDesc: 'Обявени до 2028 г., следващите – по същите правила · и за джамиите на Диянет в Европа',
  ummalqura: 'Умм ал-Кура (Саудитска Арабия)',
  ummalquraDesc: 'Рамазан и Байрамите – очаквани до обявяването по наблюдение на луната',
  short: { mufti: 'Мюфтийство', diyanet: 'Диянет', ummalqura: 'Умм ал-Кура' } as Record<'mufti' | 'diyanet' | 'ummalqura', string>,
  autoValue: (short: string) => `Авто · ${short}`,
  note: 'Датата по Хиджра в „Днес“ и „Месец“ следва същия избор.',
  group: 'Празници',
  row: 'Дати на празниците',
},

methods: {
  Turkey: 'Диянет (Турция)',
  MuslimWorldLeague: 'Muslim World League',
  MoonsightingCommittee: 'Moonsighting Committee',
  Egyptian: 'Египетски генерален орган',
  Karachi: 'Университет в Карачи',
  UmmAlQura: 'Umm al-Qura (Мека)',
}

```

```

    NorthAmerica: 'ISNA (Северна Америка)',
    Dubai: 'Дубай (OAE)',
    Kuwait: 'Кувейт',
    Qatar: 'Катар',
    Singapore: 'Сингапур (MUIS)',
    Tehran: 'Техеран (Институт по геофизика)',
    France: 'Франция (UOIF, 12°)',
    Russia: 'Русия (ДУМ)',
    Malaysia: 'Малайзия (JAKIM)',
    Indonesia: 'Индонезия (Kemenag)',
} as Record<CalculationMethodId, string>,

highLatRules: {
    seventhofthenight: { name: 'Една седма от нощта', desc: 'Най-често ползваното в Европа' },
    middleofthenight: { name: 'Средата на нощта', desc: 'Фаджр и Иша не минават средата на нощта' },
    twilightangle: { name: 'По ъгъла на здрача', desc: 'Част от нощта според ъгъла на метода' },
} as Record<HighLatRuleId, { name: string; desc: string }>,

place: {
    title: 'Избери място',
    search: 'Търси населено място в България',
    useGps: 'Използвай местоположението ми (GPS)',
    cities: 'Градове',
    noResults: 'Няма такова населено място',
    abroadNote: 'Извън България мястото се определя с GPS.',
    close: 'Затвори',
},

date: {
    weekdaysShort: ['Нд', 'Пн', 'Вт', 'Ср', 'Чт', 'Пт', 'Сб'],
    // съкращенията са с точка; „март“, „май“, „юни“, „юли“ са пълни думи
    monthsShort: ['яну.', 'фев.', 'март', 'апр.', 'май', 'юни', 'юли', 'авг.', 'септ.', 'окт.', 'ное.',
'дек.'],
    monthsFull: ['Януари', 'Февруари', 'Март', 'Април', 'Май', 'Юни', 'Юли', 'Август', 'Септември',
'Октомври', 'Ноември', 'Декември'],
},

hijriMonths: [
    'Мухаррем',
    'Сафер',
    'Ребиул-евел',
    'Ребиул-ахир',
    'Джемазиел-евел',
    'Джемазиел-ахир',
    'Реджеб',
    'Шабан',
    'Рамазан',
    'Шеввал',
    'Зилкаде',
    'Зилхидже',
],
};

export type Strings = typeof bg;

export const en: Strings = {
    prayers: {
        fajr: 'Fajr',
        sunrise: 'Sunrise',
        dhuhr: 'Dhuhr',
        asr: 'Asr',
    },

```

```

    maghrib: 'Maghrib',
    isha: 'Isha',
  },

  alert: {
    off: 'no notification',
    notify: 'notification',
    adhan: 'alarm',
  },

  tabs: {
    today: 'Today',
    qibla: 'Qibla',
    mosques: 'Mosques',
    month: 'Month',
    settings: 'Settings',
  },

  hero: {
    next: 'Next',
    at: (time) => `at ${time}`,
    tomorrowAt: (time) => `tomorrow at ${time}`,
    remaining: (time) => `${time} left`,
  },

  nowPill: 'NOW',

  busy: 'Loading...',
  locating: 'Finding your location...',
  choosePlace: 'Choose place',
  noPlaceBanner: 'Location not found - the times shown are for Sofia. Tap to choose your place.',

  ally: {
    city: (city) => `City: ${city}. Change city`,
    locating: 'Finding your location',
    bell: (prayer, mode) => `${prayer}: ${mode}. Tap to change`,
  },

  toast: (prayer, mode) => `${prayer}: ${mode}`,
  alarmToast: (prayer) => `${prayer}: alarm - sounds even in silent mode`,

  placeholder: {
    qibla: 'Compass pointing to the Kaaba. Coming in stage 7.',
  },

  month: {
    dayColumn: 'Day',
    hijriColumn: 'Hijri',
    today: 'Today',
    prev: 'Previous month',
    next: 'Next month',
    friday: 'Friday',
    sourceMufti: 'Source: Grand Mufti of Bulgaria',
    sourceCalc: (method) => `Calculated with the ${method} method`,
  },

  settings: {
    title: 'Settings',
    groupPlace: 'Location',
    groupTimes: 'Prayer times',
    groupCorrections: 'Adjustments',
  },

```

```

groupApp: 'App',
about: 'About',
change: 'Change location',
auto: 'Automatic (GPS)',
autoHint: 'Your location is detected when the app opens.',
locating: 'Detecting your location...',
denied: 'Location permission is off.',
openSettings: 'Open phone settings',
unavailable: 'Your location could not be detected. Showing the last known place.',
refresh: 'Update location',
sourceMufti: 'Grand Mufti of Bulgaria calendar',
sourceMuftiNote: 'In Bulgaria the official calendar is used. Method and Asr apply only outside
Bulgaria.',
sourceCalc: 'Calculated for this place',
method: 'Method',
methodAuto: 'Automatic',
methodAutoValue: (method) => `Auto · ${method}`,
methodAutoDesc: (country, method) => `by country: ${country} → ${method}`,
asr: 'Asr',
shafi: "Shafi'i, Maliki, Hanbali",
hanafi: 'Hanafi',
shafiDesc: 'Shadow equal to the object – the "first" Asr, as in the Diyanet calendar',
hanafiDesc: 'Shadow twice the object – about an hour later',
asrNote: "Shafi'i, Maliki and Hanbali calculate Asr the same way; only Hanafi is later.",
polarNote: (lat) =>
  `At ${lat}° latitude there are polar days and nights. On days without sunrise or sunset, times are
  calculated for 65° latitude – the nearest places with a normal day and night. Check with your local
  mosque.`,
  highLat: 'High latitudes',
  highLatNote: (lat) =>
    `At ${lat}° latitude there is no true twilight in summer, so this rule sets Fajr and Isha. Check with
    your local mosque.`,
  offsetsNote: 'If your mosque differs by a few minutes, adjust it here. Applies to all screens.',
  minutes: (n) => (n === 0 ? '0 min' : `${n > 0 ? '+' : ''}${n} min`),
  hijri: 'Hijri date',
  days: (n) => (n === 0 ? '0 days' : `${n > 0 ? '+' : ''}${n} ${Math.abs(n) === 1 ? 'day' : 'days'}`),
  hijriNote: 'If the Hijri date differs from the one announced by the Grand Mufti of Bulgaria.',
  hijriNoteAbroad: 'If the Hijri date differs from the one announced by your local mosque.',
  reset: 'Reset adjustments',
  decrease: (label) => `Decrease: ${label}`,
  increase: (label) => `Increase: ${label}`,
},

widget: {
  next: 'Next',
  tomorrowShort: (time) => `tomorrow ${time}`,
  until: (name) => `until ${name}`,
  left: 'left',
  empty: 'Open Adhan to see the prayer times.',
},

notifications: {
  prayerTitle: (prayer, time) => `${prayer} – ${time}`,
  prayerBody: (prayer, arabic, place) => `It's time for the ${prayer} prayer (${arabic}) · ${place}`,
  sunriseBody: (place) => `Sunrise – the time for Fajr has ended · ${place}`,
  nextLine: (prayer, time, tomorrow) => `Next: ${prayer} at ${time}${tomorrow ? ' (tomorrow)' : ''}`,
  reminderTitle: (prayer, minutes, time) => `${prayer} in ${minutes} min – ${time}`,
  reminderBody: (place) => `Time for wudu and to get ready for prayer · ${place}`,
  refreshTitle: 'Open Adhan',
  refreshBody: 'Open the app to keep prayer notifications coming.',
  channelPrayer: 'Prayers',
}

```

```

channelAdhan: 'Alarm',
channelReminder: 'Reminders',
group: 'Notifications',
status: 'Permission',
granted: 'Allowed',
denied: 'Denied',
undetermined: 'Not asked yet',
allow: 'Allow notifications',
deniedNote: "Notifications are turned off in the phone settings - without them prayer notifications and
alarms won't work.",
webNote: 'Notifications work only in the phone app.',
reminder: 'Reminder before prayer',
reminderOff: 'No reminder',
reminderValue: (n) => `${n} min before`,
vibrate: 'Vibration',
exact: 'Exact notifications',
exactOff: 'Off',
exactNote: 'Without the "Alarms & reminders" permission Android may delay notifications by a few
minutes.',
exactAllow: 'Allow "Alarms & reminders"',
scheduled: (n, until) => `${n} notifications scheduled - until ${until}.`,
scheduledNone: 'No notifications scheduled - all bells on "Today" are off.',
testTitle: 'Test notification',
testBody: 'This is how prayer notifications will look.',
bellDenied: "Notifications are not allowed - notifications and alarms won't work.",
allowedToast: '✓ Notifications are allowed',
exactBanner: 'Notifications may be a few minutes late. Tap to allow exact notifications.',
deniedBanner: "Notifications are not allowed - notifications and alarms won't work. Tap to allow
them.",
diagnostics: 'Notification check',
diag: {
  title: 'Notification check',
  subtitle: 'Whether the phone shows Adhan notifications on time - and what to fix if not.',
  refresh: 'Refresh',
  updated: (time) => `Updated at ${time}`,
  status: 'Status',
  permOk: 'Notifications are allowed',
  permBad: 'Notifications are not allowed',
  exactOk: 'Exact notifications ("Alarms & reminders")',
  exactBad: 'Exact notifications are off - they may be late',
  allow: 'Allow',
  scheduledOk: (n, until) => `${n} notifications scheduled - until ${until}`,
  scheduledNone: 'No notifications scheduled - the bells on "Today" are off',
  test: 'Test notification',
  testNotification: 'Notification',
  testIn: (s) => `in ${s} s`,
  testHint: 'Arrives in 10 s. Lock the phone to see and hear it as it will arrive. The alarm sound can
be previewed in Settings → Sounds.',
  recent: 'Recent notifications',
  recentNone: 'No Adhan notifications shown yet.',
  onTime: 'on time',
  late: (s) => (s < 60 ? `+${s} s` : `+${Math.round(s / 6) / 10} min`),
  plannedShown: (planned, shown) => `planned ${planned} · shown ${shown}`,
  upcoming: 'Upcoming notifications',
  kindAlarm: 'Alarm',
  kindNotification: 'Notification',
  kindReminder: 'Reminder',
  kindOther: 'Other',
  tech: 'Technical details',
  device: 'Phone',
  channelLine: (importance, sound, vibrate, usage) =>

```

```

        `importance ${importance} · sound ${sound} · vibration ${vibrate ? 'yes' : 'no'} · stream
        ${usage}` ,
        reset: 'Reschedule all notifications',
        resetHint: 'Deletes all Adhan notifications and schedules them again from the current settings. Use
it if something looks wrong.',
        resetDone: '✓ All notifications have been scheduled again.',
        fullScreenOk: 'Full-screen alarm',
        fullScreenBad: 'Full-screen alarm is off',
        batteryOk: 'Runs in the background without limits',
        batteryBad: 'The phone limits background activity',
        fix: 'Fix',
    },
    fullScreen: 'Full-screen alarm',
    fullScreenOn: 'Allowed',
    fullScreenOff: 'Off',
    fullScreenNote: 'Without it the adhan still plays on a locked phone, but you only see a notification -
not the full screen.',
    fullScreenAllow: 'Allow "Full-screen alarm"',
    exactOn: 'Allowed',
    background: 'Background activity',
    backgroundOk: 'Unrestricted',
    backgroundBad: 'Restricted',
    batteryTitle: 'Alarms may be late',
    batteryText: 'The phone may stop the app to save battery.',
    batteryFix: 'Fix',
},

qibla: {
    title: 'Qibla',
    fromNorth: 'from north',
    toMecca: 'to Mecca',
    km: 'km',
    aligned: '✓ You are facing the Qibla',
    turnLabel: (direction) => `Turn ${direction}`,
    starting: 'Starting the compass...',
    quality: { high: 'high', medium: 'medium', low: 'low' },
    accuracy: (q) => `Compass: ${q} accuracy`,
    tip: 'Hold the phone flat, screen up. Metal and magnets nearby confuse the compass.',
    calibrateTitle: 'Calibrate the compass',
    calibrateText: 'Move the phone in a figure-eight in the air 2-3 times. Keep it away from metal, magnets
and magnetic cases.',
    noCompassTitle: 'No compass',
    noCompassText: (deg) =>
        `This phone has no magnetic sensor. The Qibla is ${deg}° from north: face north and turn right by
        ${deg}°`,
    permissionTitle: 'Permission needed',
    permissionText: 'On iPhone the compass works only with location permission.',
    allowLocation: 'Allow location',
    letters: ['N', 'E', 'S', 'W'],
    allyDial: (bearing) => `Compass. The Qibla is ${bearing}° from north.`,
    sunToday: 'Today at {t} the sun is exactly in the Qibla direction',
    sunTomorrow: 'Tomorrow at {t} the sun will be exactly in the Qibla direction',
    sunNow: 'Right now the sun is exactly in the Qibla direction',
    sunrise: (azimuth, degrees, side) =>
        side === 'same'
        ? `Sunrise at ${azimuth}° - exactly in the Qibla direction`
        : `Sunrise at ${azimuth}° - the Qibla is ${degrees}° to its ${side}`,
},

sounds: {
    group: 'Sounds',

```

```

    notifyGroup: 'Reminder and notification',
    notifyRow: 'Notification sound',
    noteAndroid: 'The sound plays when the prayer bell is set to "alarm". Without "Alarms & reminders" -
the short sound (up to 30 s).',
    noteIos: 'The sound plays when the prayer bell is set to "alarm". On iPhone the sound is up to 30 s.',
    fullOffTitle: 'The full adhan is off',
    fullOffText: 'Without "Alarms & reminders" only the short sound plays (up to 30 s). To get the full
adhan, allow "Alarms & reminders".',
    allow: 'Allow',
    title: (prayer) => `Sound for ${prayer}`,
    notifyTitle: 'Notification sound',
    fullSection: 'Full sound - alarm',
    fullHint: 'Plays to the end; "Stop" ends it.',
    shortSection: 'Short sound - up to 30 s',
    shortHintAndroid: 'When "Alarms & reminders" is off. Your own sound is trimmed to 30 s - at a pause,
with a gentle fade-out.',
    iosSection: 'Sound - up to 30 s',
    iosHint: 'On iPhone a notification sound can be 30 s at most. Longer recordings are trimmed at a pause,
with a gentle fade-out.',
    notifyHint: 'For notifications and reminders before prayer.',
    byDefault: 'default',
    mine: 'MINE',
    add: 'Add your own sound',
    addFullHint: 'mp3, m4a, mp4, wav, ogg, flac · from the phone',
    addShortHint: 'mp3, m4a, mp4, wav... · longer - trimmed to 30 s',
    addNotifyHint: 'up to 10 s · mp3, m4a, mp4, wav...',
    tooLongNotify: (name, dur) => `${name} is ${dur} - a notification sound can be up to 10 s. Pick a
shorter one.`,
    adding: 'Adding...',
    workingShort: 'Trimming the excerpt...',
    workingFull: 'Adding the sound...',
    workingHint: 'Looking for the pause before the 30th second. This may take a few seconds.',
    applyAll: 'Same sound for all prayers',
    apply: 'Apply',
    applied: '✓ Applied to all prayers',
    iosFuture: 'The full adhan on iPhone will come with AlarmKit (iOS 26) - stage 9.',
    added: (name) => `✓ Added: ${name}`,
    addedTrimmed: (name, dur) =>
        `✓ Added: ${name}. The short sound is up to 30 s - the first ${dur} will play, up to a pause, with a
gentle fade-out.`,
    trimmed: (dur, of) => `plays the first ${dur} of ${of}`,
    tooLongFull: (name) => `${name} is longer than 15 min.`,
    tooBig: 'The file is larger than 60 MB.',
    unreadable: 'This file can't be read as a sound.',
    unsupported: 'Your own sounds can only be added in the phone app.',
    deleteTitle: (name) => `Remove "${name}"?`,
    deleteText: 'Prayers using this sound go back to the default sound.',
    delete: 'Remove',
    cancel: 'Cancel',
    play: (name) => `Play ${name}`,
    stop: (name) => `Stop ${name}`,
},

alarm: {
    app: 'Adhan',
    title: (prayer) => `It's time for ${prayer}`,
    stop: 'Stop adhan',
    stopAlarm: 'Stop alarm',
    mute: 'Mute sound',
    muteShort: 'Mute',
    close: 'Close',

```

```

    soundName: 'Adhan',
    channel: 'Prayer alarm',
  },

  battery: {
    title: 'Background activity',
    lede: 'To save battery, the phone may put Adhan to sleep, and the alarm may be late or not ring at all.
Let the app run without restrictions.',
    yourPhone: 'Your phone',
    open: 'Open Adhan settings',
    openList: '"Battery optimization" list',
    now: 'Now',
    ok: 'unrestricted ✓',
    bad: 'restricted',
    unknown: "can't be checked",
    otherPhone: 'Another phone?',
    more: 'More tips for every brand: dontkillmyapp.com',
    brands: {
      samsung: 'Samsung',
      xiaomi: 'Xiaomi / Redmi / POCO',
      huawei: 'Huawei / Honor',
      oppo: 'OPPO / Realme / OnePlus',
      other: 'Other',
    },
  },
  steps: {
    samsung: [
      ['Tap the button below - the Adhan settings open.'],
      ['Open', 'Battery'],
      ['Choose', 'Unrestricted', 'Not "Optimized" and not "Restricted"'],
      ['Optional:', 'Settings → Battery → Background usage limits', 'Adhan should not be in "Deep
sleeping apps" or "Sleeping apps"'],
    ],
    xiaomi: [
      ['Tap the button below - the Adhan settings open.'],
      ['Open', 'Battery saver', 'Choose "No restrictions"'],
      ['Turn on', 'Autostart'],
      ['Optional: in recent apps, hold the Adhan card and lock it (padlock)'],
    ],
    huawei: [
      ['Open', 'Settings → Battery → App launch'],
      ['Find Adhan and turn off', 'Manage automatically'],
      ['In the window, turn on all three:', 'Auto-launch · Secondary launch · Run in background'],
    ],
    oppo: [
      ['Tap the button below - the Adhan settings open.'],
      ['Open', 'Battery usage'],
      ['Turn on', 'Allow background activity', 'and "Auto launch", if it is there.'],
    ],
    other: [
      ['Tap the button below - the Adhan settings open.'],
      ['Open', 'Battery'],
      ['Choose', 'Unrestricted', 'On some phones it is called "Don\'t optimize"'],
    ],
  },
},

  about: {
    title: 'About',
    name: 'Adhan',
    back: 'Settings',
    version: (v: string) => `Version ${v}`,
  },
}

```

```

tagline: 'Prayer times, reminders and the adhan',
groupTimes: 'Prayer time sources',
groupPlaces: 'Place data',
groupPrivacy: 'Privacy',
groupMap: 'The map and mosques',
sources: {
  mufti: { name: 'Grand Mufti of Bulgaria', desc: 'Prayer calendar - times in Bulgaria' },
  adhan: { name: 'adhan', desc: 'Calculation outside Bulgaria · MIT license' },
  osm: { name: 'OpenStreetMap', desc: 'Places in Bulgaria · © OpenStreetMap contributors, ODbL' },
  ne: { name: 'Natural Earth', desc: 'Border of Bulgaria · public domain' },
  geonames: { name: 'GeoNames', desc: 'World cities - name and country without internet · CC BY 4.0' },
  praytimes: { name: 'PrayTimes.org', desc: 'Adhan recordings: Makkah, Madinah, Al-Aqsa' },
  overpass: { name: 'OpenStreetMap · Overpass API', desc: 'Mosques nearby · © OpenStreetMap contributors, ODbL' },
  openfreemap: { name: 'OpenFreeMap', desc: 'The map · © OpenMapTiles, data © OpenStreetMap' },
},
privacy:
  'Your location is used on the phone to pick the place and the prayer times. Outside Bulgaria the phone asks its map service for the city name. For "Mosques nearby" only an approximate area (about 1 km) is sent to OpenStreetMap, and the map is loaded from OpenFreeMap.',
},

mosques: {
  title: 'Mosques nearby',
  count: (n: number, dist: string) => `${n} · within ${dist}`,
  next: 'Next:',
  nextAt: (prayer: string, time: string) => `${prayer} at ${time}`,
  nextIn: (left: string) => `in ${left}`,
  walk: (min: number) => `${min} min walk`,
  drive: (min: number) => `~${min} min by car`,
  unnamed: 'Mosque',
  unnamedSub: 'no name in OpenStreetMap',
  locating: 'Finding where you are...',
  loading: 'Looking for mosques nearby...',
  noneNear: (dist: string) => `No mosques within ${dist}`,
  noneFar: (dist: string) => `No mosques even within ${dist}`,
  noneHint: 'The map is OpenStreetMap - in small places not every mosque is on it.',
  searchFar: (dist: string) => `Search within ${dist}`,
  within: (dist: string) => `within ${dist}`,
  offlineTitle: 'No internet.',
  offlineText: (when: string) => `Showing the last search here - from ${when}. Directions will work once you are online.`,
  today: 'today',
  yesterday: 'yesterday',
  errorOffline: 'No internet',
  errorOfflineText: 'Mosques are looked up in OpenStreetMap - a connection is needed.',
  errorServer: 'The server is not responding',
  errorServerText: 'OpenStreetMap (Overpass) is busy. Try again in a minute.',
  retry: 'Try again',
  noGps: (place: string) => `No access to your location - searching around ${place}.`,
  allow: 'Allow',
  myLocation: 'My location',
  locate: 'Go to my location',
  back: 'Back to the list',
  reach: (min: number) => `you'll be there in ${min} min`,
  directions: (app: string) => `Directions with ${app}`,
  directionsAuto: 'Directions',
  withApp: (app: string) => `With ${app} · change in Settings → Navigation`,
  withAuto: 'With the phone's maps · change in Settings → Navigation',
  share: 'Share',
  copyAddress: 'Copy address',
}

```

```

copyCoords: 'Copy coordinates',
copied: 'Copied',
noAddress: 'No address in OpenStreetMap',
allyRow: (name: string, dist: string, time: string) => `${name}, ${dist}, ${time}`,
allyMap: (n: number) => `Map with ${n} mosques`,
attribution: 'OpenFreeMap © OpenMapTiles © OpenStreetMap',
},

nav: {
  title: 'Navigation',
  back: 'Settings',
  intro: 'Which app opens "Directions" to the mosque.',
  auto: 'Automatic',
  autoAndroid: 'The phone's default maps app',
  autoIos: 'Apple Maps',
  walkDrive: 'Walking and driving',
  driveOnly: 'Driving only',
  notInstalled: 'not installed',
  organic: 'OpenStreetMap · works offline',
  huawei: 'on Huawei',
  download: 'Get',
  note: 'Waze only does driving - even for a mosque 300 m away it will route you by car. If the chosen
app is missing, "Directions" asks which one to use.',
  sheetTitle: (name: string) => `Directions to ${name}`,
  sheetSub: (dist: string) => `${dist} · which app?`,
  walk: 'walking',
  drive: 'driving',
  remember: 'Remember my choice',
  rememberHint: 'Change: Settings → Navigation',
  missing: (app: string) => `${app} is no longer on the phone - pick another app.`,
  group: 'Mosques nearby',
  row: 'Navigation',
  rowNote: 'Which app opens "Directions" to a mosque.',
  ask: 'Ask each time',
  close: 'Close',
},

holidays: {
  tabTimes: 'Times',
  tabHolidays: 'Holidays',
  hijriYears: (a: number, b: number) => (a === b ? `${a} AH` : `${a} - ${b} AH`),
  next: 'Next',
  today: 'today',
  tomorrow: 'tomorrow',
  daysWord: (n: number) => (n === 1 ? 'day' : 'days'),
  inDays: (n: number) => (n === 0 ? 'today' : n === 1 ? 'tomorrow' : `in ${n} days`),
  evening: 'evening',
  daysCount: (n: number) => `${n} days`,
  expected: 'expected',
  expectedNote:
    'Expected - here the start of Ramadan and the Eids is announced after the moon is sighted, the
evening before. The date may move by 1 day.',
  computedNote: 'Diyanet has announced the dates until early 2028; this one follows the same astronomical
rules.',
  noData: 'No data for this year.',
  dates: 'Dates',
  date: 'Date',
  hijri: 'Hijri',
  arefe: 'Arafah / eve',
  remindNight: 'Remind me that evening',
  remindDay: 'Remind me the evening before',
}

```

```

remindAt: (date: string) => `${date} · at Maghrib`,
source: {
  mufti: 'Source: Chief Mufti of Bulgaria (the Diyanet calendar)',
  diyanet: 'Source: Diyanet (Türkiye)',
  ummalqura: 'Source: Umm al-Qura (Saudi Arabia)',
} as Record<'mufti' | 'diyanet' | 'ummalqura', string>,
names: {
  threeMonths: 'The three holy months',
  regaib: 'Laylat al-Raghaib',
  miraj: 'Laylat al-Mi'raj',
  berat: 'Laylat al-Bara'ah',
  ramadan: 'Start of Ramadan',
  qadr: 'Laylat al-Qadr',
  fitrEve: 'Eve of Eid al-Fitr',
  fitr: 'Eid al-Fitr',
  arafah: 'Day of Arafah',
  adha: 'Eid al-Adha',
  newYear: 'Islamic New Year',
  ashura: 'Ashura',
  mawlid: 'Mawlid an-Nabi',
} as Record<HolidayId, string>,
about: {
  threeMonths: 'The holy months of Rajab, Sha'ban and Ramadan begin.',
  regaib: 'The night before the first Friday of Rajab – the first of the holy nights of the three months.',
  miraj: 'The night of al-Isra and al-Mi'raj – the Prophet's ﷺ night journey from Makkah to Jerusalem and his ascension.',
  berat: 'The night before 15 Sha'ban – a night of forgiveness and prayer.',
  ramadan: 'The first day of fasting. The first tarawih prayer is the evening before.',
  qadr: 'The Night of Decree, “better than a thousand months”. Observed on the night before 27 Ramadan.',
  fitrEve: 'The last day of Ramadan – the day before Eid al-Fitr.',
  fitr: 'The festival after Ramadan. It begins with the Eid prayer on the morning of the first day.',
  arafah: 'The day the pilgrims stand at Arafah – the day before Eid al-Adha.',
  adha: 'The festival of sacrifice. It begins with the Eid prayer on the morning of the first day.',
  newYear: '1 Muharram – the start of the Hijri year.',
  ashura: '10 Muharram – a day of fasting and remembrance.',
  mawlid: 'The night of the birth of the Prophet Muhammad ﷺ – before 12 Rabi al-Awwal.',
} as Record<HolidayId, string>,
notifyTonight: (name: string) => `Tonight: ${name}`,
notifyTomorrow: (name: string) => `Tomorrow: ${name}`,
allyRow: (name: string, date: string, hijri: string) => `${name}, ${date}, ${hijri}`,
prevYear: 'Previous year',
nextYear: 'Next year',
close: 'Close',
},

holidaySource: {
  title: 'Holiday dates',
  back: 'Settings',
  intro:
    'Whose dates the calendar shows. Different communities sometimes celebrate on different days – both dates are right for their community.',
  auto: 'Automatic',
  autoDesc: (source: string) => `By your place: ${source}`,
  mufti: 'Chief Mufti (Bulgaria)',
  muftiDesc: 'Announced in advance – the same calendar as Diyanet',
  diyanet: 'Diyanet (Türkiye)',
  diyanetDesc: 'Announced until 2028, later ones by the same rules · also for Diyanet mosques in Europe',
  ummalqura: 'Umm al-Qura (Saudi Arabia)',
  ummalquraDesc: 'Ramadan and the Eids – expected until announced after moon sighting',
}

```

```

    short: { mufti: 'Chief Mufti', diyanet: 'Diyanet', ummalqura: 'Umm al-Qura' } as Record<'mufti' |
'diyanet' | 'ummalqura', string>,
    autoValue: (short: string) => `Auto · ${short}`,
    note: 'The Hijri date on "Today" and "Month" follows the same choice.',
    group: 'Holidays',
    row: 'Holiday dates',
  },

  methods: {
    Turkey: 'Diyanet (Turkey)',
    MuslimWorldLeague: 'Muslim World League',
    MoonsightingCommittee: 'Moonsighting Committee',
    Egyptian: 'Egyptian General Authority',
    Karachi: 'University of Karachi',
    UmmAlQura: 'Umm al-Qura (Makkah)',
    NorthAmerica: 'ISNA (North America)',
    Dubai: 'Dubai (UAE)',
    Kuwait: 'Kuwait',
    Qatar: 'Qatar',
    Singapore: 'Singapore (MUIS)',
    Tehran: 'Tehran (Institute of Geophysics)',
    France: 'France (UOIF, 12°)',
    Russia: 'Russia (DUM)',
    Malaysia: 'Malaysia (JAKIM)',
    Indonesia: 'Indonesia (Kemenag)',
  },

  highLatRules: {
    seventhofthenight: { name: 'Seventh of the night', desc: 'Most common in Europe' },
    middleofthenight: { name: 'Middle of the night', desc: 'Fajr and Isha never pass midnight' },
    twilightangle: { name: 'Twilight angle', desc: 'Share of the night based on the method angle' },
  },

  place: {
    title: 'Choose location',
    search: 'Search a place in Bulgaria',
    useGps: 'Use my location (GPS)',
    cities: 'Cities',
    noResults: 'No such place',
    abroadNote: 'Outside Bulgaria your location comes from GPS.',
    close: 'Close',
  },

  date: {
    weekdaysShort: ['Sun', 'Mon', 'Tue', 'Wed', 'Thu', 'Fri', 'Sat'],
    monthsShort: ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun', 'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec'],
    monthsFull: ['January', 'February', 'March', 'April', 'May', 'June', 'July', 'August', 'September',
'October', 'November', 'December'],
  },

  hijriMonths: [
    'Muharram',
    'Safar',
    'Rabi al-Awwal',
    'Rabi al-Thani',
    'Jumada al-Ula',
    'Jumada al-Akhirah',
    'Rajab',
    'Shaban',
    'Ramadan',
    'Shawwal',
  ],

```

```

    'Dhu al-Qadah',
    'Dhu al-Hijjah',
  ],
};

```

tr.ts src/i18n

- И на турски – както при Диянет: Regaip Kandili, Kadir Gecesi, Kurban Bayramı...

src/i18n/tr.ts

595 реда

```

import type { HolidayId } from '@domain/holidays';
import type { Strings } from './strings';

/**
 * Türkçe (етап 10). Имената на молитвите и месеците – както в календара на Диянет
 * (İmsak, Güneş, Öğle, İkindi, Akşam, Yatsı). Където името на молитвата влиза в изречение,
 * след него стои „vakti/vaktine“ – така не трябва турски падежни наставки.
 * Настройките на телефона (Samsung, Xiaomi...) – с имената им в турския интерфейс.
 */
export const tr: Strings = {
  prayers: {
    fajr: 'İmsak',
    sunrise: 'Güneş',
    dhuhr: 'Öğle',
    asr: 'İkindi',
    maghrib: 'Akşam',
    isha: 'Yatsı',
  },

  alert: {
    off: 'bildirim yok',
    notify: 'bildirim',
    adhan: 'alarm',
  },

  tabs: {
    today: 'Bugün',
    qibla: 'Kıble',
    mosques: 'Camiler',
    month: 'Ay',
    settings: 'Ayarlar',
  },

  hero: {
    next: 'Sıradaki',
    at: (time) => `saat ${time}`,
    tomorrowAt: (time) => `yarın ${time}`,
    remaining: (time) => `${time} kaldı`,
  },

  nowPill: 'ŞİMDİ',

  busy: 'Yükleniyor...',
  locating: 'Konumunuz bulunuyor...',
  choosePlace: 'Yer seçin',
  noPlaceBanner: 'Konum bulunamadı – gösterilen vakitler Sofya içindir. Yerinizi seçmek için dokununuz.',

  all: {
    city: (city) => `Şehir: ${city}. Şehri değiştir`,
  },
};

```

```

    locating: 'Konumunuz bulunuyor',
    bell: (prayer, mode) => `${prayer}: ${mode}. Değiştirmek için dokunun`,
  },

  toast: (prayer, mode) => `${prayer}: ${mode}`,
  alarmToast: (prayer) => `${prayer}: alarm - sessiz modda da çalar`,

  placeholder: {
    qibla: 'Kâbe'yi gösteren pusula. 7. aşamada gelecek.',
  },

  month: {
    dayColumn: 'Gün',
    hijriColumn: 'Hicri',
    today: 'Bugün',
    prev: 'Önceki ay',
    next: 'Sonraki ay',
    friday: 'Cuma',
    sourceMufti: 'Kaynak: Bulgaristan Başmüftülüğü',
    sourceCalc: (method) => `${method} yöntemiyle hesaplandı`,
  },

  settings: {
    title: 'Ayarlar',
    groupPlace: 'Konum',
    groupTimes: 'Namaz vakitleri',
    groupCorrections: 'Düzeltilmeler',
    groupApp: 'Uygulama',
    about: 'Hakkında',
    change: 'Konumu değiştir',
    auto: 'Otomatik (GPS)',
    autoHint: 'Konumunuz uygulama açıldığında belirlenir.',
    locating: 'Konumunuz belirleniyor...',
    denied: 'Konum izni kapalı.',
    openSettings: 'Telefon ayarlarını aç',
    unavailable: 'Konumunuz belirlenemedi. Bilinen son yer gösteriliyor.',
    refresh: 'Konumu güncelle',
    sourceMufti: 'Bulgaristan Başmüftülüğü takvimi',
    sourceMuftiNote: 'Bulgaristan'da resmî takvim kullanılır. Yöntem ve İkinci ayarı yalnızca Bulgaristan dışında geçerlidir.',
    sourceCalc: 'Bu yer için hesaplandı',
    method: 'Yöntem',
    methodAuto: 'Otomatik',
    methodAutoValue: (method) => `Otomatik · ${method}`,
    methodAutoDesc: (country, method) => `Ülkeye göre: ${country} → ${method}`,
    asr: 'İkinci',
    shafi: 'Şâfiî, Mâlikî, Hanbelî',
    hanafi: 'Hanefî',
    shafiDesc: 'Gölge cismin boyu kadar - Diyanet takvimindeki gibi "birinci" İkinci',
    hanafiDesc: 'Gölge cismin iki katı - yaklaşık bir saat sonra',
    asrNote: 'Şâfiî, Mâlikî ve Hanbelî İkinci'yi aynı hesaplar; yalnızca Hanefî daha geçtir.',
    polarNote: (lat) =>
      `${lat}° enlemde kutup gündüzleri ve geceleri vardır. Güneşin doğmadığı veya batmadığı günlerde vakitler 65° enleme göre - normal gece ve gündüzün olduğu en yakın yerlere göre - hesaplanır. Bölgenizdeki camiye danışın.`,
    highLat: 'Yüksek enlemler',
    highLatNote: (lat) =>
      `${lat}° enlemde yazın gerçek alacakaranlık olmaz; İmsak ve Yatsı bu kurala göre belirlenir. Bölgenizdeki camiye danışın.`,
    offsetsNote: 'Caminiz birkaç dakika farklıysa buradan düzeltin. Tüm ekranlarda geçerlidir.',
    minutes: (n) => (n === 0 ? '0 dk' : `${n > 0 ? '+' : ''}${n} dk`),
  },

```

```

hijri: 'Hicri tarih',
days: (n) => (n === 0 ? '0 gün' : `${n} > 0 ? '+' : ''}${n} gün`),
hijriNote: 'Hicri tarih Bulgaristan Başmüftülüğünün ilan ettiğinden farklıysa.',
hijriNoteAbroad: 'Hicri tarih bölgenizdeki caminin ilan ettiğinden farklıysa.',
reset: 'Düzeltilmeleri sıfırla',
decrease: (label) => `Azalt: ${label}`,
increase: (label) => `Artır: ${label}`,
},

widget: {
  next: 'Sıradaki',
  tomorrowShort: (time) => `yarın ${time}`,
  until: (name) => `${name} vaktine kalan`,
  left: 'kaldı',
  empty: 'Vakitleri görmek için Ezan'ı açın.',
},

notifications: {
  prayerTitle: (prayer, time) => `${prayer} - ${time}`,
  prayerBody: (prayer, arabic, place) => `${prayer} vakti girdi (${arabic}) • ${place}`,
  sunriseBody: (place) => `Güneş doğdu - İmsak vakti sona erdi • ${place}`,
  nextLine: (prayer, time, tomorrow) => `Sıradaki: ${prayer}, ${tomorrow ? 'yarın ' : ''}saat ${time}`,
  reminderTitle: (prayer, minutes, time) => `${prayer} vaktine ${minutes} dk - ${time}`,
  reminderBody: (place) => `Abdest alıp namaza hazırlanma vakti • ${place}`,
  refreshTitle: 'Ezan'ı açın',
  refreshBody: 'Namaz bildirimlerinin gelmeye devam etmesi için uygulamayı açın.',
  channelPrayer: 'Namazlar',
  channelAdhan: 'Alarm',
  channelReminder: 'Hatırlatmalar',
  group: 'Bildirimler',
  status: 'İzin',
  granted: 'Verildi',
  denied: 'Reddedildi',
  undetermined: 'Henüz sorulmadı',
  allow: 'Bildirimlere izin ver',
  deniedNote: 'Bildirimler telefon ayarlarında kapalı - bunlar olmadan namaz bildirimleri ve alarmlar çalışmaz.',
  webNote: 'Bildirimler yalnızca telefon uygulamasında çalışır.',
  reminder: 'Namazdan önce hatırlatma',
  reminderOff: 'Hatırlatma yok',
  reminderValue: (n) => `${n} dk önce`,
  vibrate: 'Titreşim',
  exact: 'Tam zamanında bildirimler',
  exactOff: 'Kapalı',
  exactNote: '"Alarmlar ve hatırlatıcılar" izni olmadan Android bildirimleri birkaç dakika geciktirebilir.',
  exactAllow: '"Alarmlar ve hatırlatıcılar"a izin ver',
  scheduled: (n, until) => `${n} bildirim planlandı - ${until} tarihine kadar.`,
  scheduledNone: 'Planlanmış bildirim yok - "Bugün" ekranındaki tüm zil simgeleri kapalı.',
  testTitle: 'Deneme bildirimi',
  testBody: 'Namaz bildirimleri böyle görünecek.',
  bellDenied: 'Bildirimlere izin verilmedi - bildirimler ve alarmlar çalışmaz.',
  allowedToast: '✓ Bildirimlere izin verildi',
  exactBanner: 'Bildirimler birkaç dakika gecikebilir. Tam zamanında bildirimlere izin vermek için dokununuz.',
  deniedBanner: 'Bildirimlere izin verilmedi - bildirimler ve alarmlar çalışmaz. İzin vermek için dokununuz.',
  diagnostics: 'Bildirim kontrolü',
  diag: {
    title: 'Bildirim kontrolü',
    subtitle: 'Telefon Ezan bildirimlerini zamanında gösteriyor mu - göstermiyorsa neyin düzeltilmesi

```

```

gerekir.',
  refresh: 'Yenile',
  updated: (time) => `Güncellendi: ${time}`,
  status: 'Durum',
  permOk: 'Bildirimlere izin verildi',
  permBad: 'Bildirimlere izin verilmedi',
  exactOk: 'Tam zamanında bildirimler ("Alarmlar ve hatırlatıcılar")',
  exactBad: 'Tam zamanında bildirimler kapalı - gecikebilir',
  allow: 'İzin ver',
  scheduledOk: (n, until) => `${n} bildirim planlandı · ${until} tarihine kadar`,
  scheduledNone: 'Planlanmış bildirim yok - "Bugün" ekranındaki zil simgeleri kapalı',
  test: 'Deneme bildirimi',
  testNotification: 'Bildirim',
  testIn: (s) => `${s} sn sonra`,
  testHint: '10 sn sonra gelir. Gerçekte nasıl geleceğini görmek ve duymak için telefonu kilitleyin.
Alarm sesi Ayarlar → Sesler bölümünden dinlenebilir.',
  recent: 'Son bildirimler',
  recentNone: 'Henüz Ezan bildirimi gösterilmedi.',
  onTime: 'zamanında',
  late: (s) => (s < 60 ? `+${s} sn` : `+${Math.round(s / 6) / 10} dk`),
  plannedShown: (planned, shown) => `planlanan ${planned} · gösterilen ${shown}`,
  upcoming: 'Yaklaşan bildirimler',
  kindAlarm: 'Alarm',
  kindNotification: 'Bildirim',
  kindReminder: 'Hatırlatma',
  kindOther: 'Diğer',
  tech: 'Teknik ayrıntılar',
  device: 'Telefon',
  channelLine: (importance, sound, vibrate, usage) =>
    `Önem ${importance} · ses ${sound} · titreşim ${vibrate ? 'evet' : 'hayır'} · akış ${usage}`,
  reset: 'Tüm bildirimleri yeniden planla',
  resetHint: 'Tüm Ezan bildirimlerini siler ve geçerli ayarlarla yeniden planlar. Bir şey yanlış
görünüyorsa kullanın.',
  resetDone: '✓ Tüm bildirimler yeniden planlandı.',
  fullScreenOk: 'Tam ekran alarm',
  fullScreenBad: 'Tam ekran alarm kapalı',
  batteryOk: 'Arka planda kısıtlamasız çalışıyor',
  batteryBad: 'Telefon arka plan etkinliğini kısıtlıyor',
  fix: 'Düzelt',
},
fullScreen: 'Tam ekran alarm',
fullScreenOn: 'İzin verildi',
fullScreenOff: 'Kapalı',
fullScreenNote: 'Bu olmadan ezan kilitli telefonda yine çalar, ancak tam ekran yerine yalnızca bir
bildirim görürsünüz.',
fullScreenAllow: '"Tam ekran alarm"a izin ver',
exactOn: 'İzin verildi',
background: 'Arka plan etkinliği',
backgroundOk: 'Kısıtlamasız',
backgroundBad: 'Kısıtlı',
batteryTitle: 'Alarmlar gecikebilir',
batteryText: 'Telefon pil tasarrufu için uygulamayı durdurabilir.',
batteryFix: 'Düzelt',
},

qibla: {
  title: 'Kible',
  fromNorth: 'kuzeyden',
  toMecca: 'Mekke'ye',
  km: 'km',
  aligned: '✓ Kibleye dönüyorsunuz',

```

```

turnLabel: (direction) => (direction === 'right' ? 'Sağa dönün' : 'Sola dönün'),
starting: 'Pusulâ başlatılıyor...',
quality: { high: 'yüksek', medium: 'orta', low: 'düşük' },
accuracy: (q) => `Pusulâ: ${q} doğruluk`,
tip: 'Telefonu ekranı yukarı bakacak şekilde düz tutun. Yakındaki metal ve mıknatıslar pusulayı
şaşırtır.',
calibrateTitle: 'Pusulayı kalibre edin',
calibrateText: 'Telefonu havada 2-3 kez sekiz çizerek hareket ettirin. Metalden, mıknatıslardan ve
mıknatıslı kılıflardan uzak tutun.',
noCompassTitle: 'Pusulâ yok',
noCompassText: (deg) =>
  `Bu telefonda manyetik sensör yok. Kible kuzeyden ${deg}°: kuzeye dönün ve ${deg}° sağa dönün.`,
permissionTitle: 'İzin gerekli',
permissionText: 'iPhone’da pusulâ yalnızca konum izniyle çalışır.',
allowLocation: 'Konuma izin ver',
letters: ['K', 'D', 'G', 'B'],
allyDial: (bearing) => `Pusulâ. Kible kuzeyden ${bearing}°`,
sunToday: 'Güneş bugün tam Kible yönünde: saat {t}',
sunTomorrow: 'Güneş yarın tam Kible yönünde olacak: saat {t}',
sunNow: 'Şu anda güneş tam Kible yönünde',
sunrise: (azimuth, degrees, side) =>
  side === 'same'
    ? `Gün doğumu ${azimuth}° - tam Kible yönünde`
    : `Gün doğumu ${azimuth}° - Kible, gün doğumunun ${degrees}° ${side === 'right' ? 'sağında' :
'solunda'}`,
},

sounds: {
  group: 'Sesler',
  notifyGroup: 'Hatırlatma ve bildirim',
  notifyRow: 'Bildirim sesi',
  noteAndroid: 'Ses, namazın zili “alarm” olduğunda çalar. “Alarmlar ve hatırlatıcılar” olmadan - kısa
ses (en fazla 30 sn).',
  noteIos: 'Ses, namazın zili “alarm” olduğunda çalar. iPhone’da ses en fazla 30 sn’dir.',
  fullOffTitle: 'Tam ezan kapalı',
  fullOffText: “Alarmlar ve hatırlatıcılar” olmadan yalnızca kısa ses çalar (en fazla 30 sn). Tam ezan
için “Alarmlar ve hatırlatıcılar”a izin verin.',
  allow: 'İzin ver',
  title: (prayer) => `${prayer} sesi`,
  notifyTitle: 'Bildirim sesi',
  fullSection: 'Tam ses - alarm',
  fullHint: 'Sonuna kadar çalar; “Durdur” keser.',
  shortSection: 'Kısa ses - en fazla 30 sn',
  shortHintAndroid: “Alarmlar ve hatırlatıcılar” kapalıyken. Kendi sesiniz bir duraklamada, yumuşak bir
kısılmayla 30 sn’ye kısaltılır.',
  iosSection: 'Ses - en fazla 30 sn',
  iosHint: 'iPhone’da bildirim sesi en fazla 30 sn olabilir. Daha uzun kayıtlar bir duraklamada, yumuşak
bir kısılmayla kısaltılır.',
  notifyHint: 'Bildirimler ve namazdan önceki hatırlatmalar için.',
  byDefault: 'varsayılan',
  mine: 'BENİM',
  add: 'Kendi sesini ekle',
  addFullHint: 'mp3, m4a, mp4, wav, ogg, flac · telefondan',
  addShortHint: 'mp3, m4a, mp4, wav... · uzunsa 30 sn’ye kısaltılır',
  addNotifyHint: 'en fazla 10 sn · mp3, m4a, mp4, wav...',
  tooLongNotify: (name, dur) => `${name} ${dur} uzunluğunda - bildirim sesi en fazla 10 sn olabilir.
Daha kısa bir ses seçin.`,
  adding: 'Ekleniyor...',
  workingShort: 'Kesit kesiliyor...',
  workingFull: 'Ses ekleniyor...',
  workingHint: '30. saniyeden önceki duraklama aranıyor. Birkaç saniye sürebilir.',

```

```

    applyAll: 'Tüm namazlar için aynı ses',
    apply: 'Uygula',
    applied: '✓ Tüm namazlara uygulandı',
    iosFuture: 'iPhone'da tam ezan AlarmKit (iOS 26) ile gelecek - 9. aşama.',
    added: (name) => `✓ Eklendi: ${name}`,
    addedTrimmed: (name, dur) =>
      `✓ Eklendi: ${name}. Kısa ses en fazla 30 sn'dir - ilk ${dur} bir duraklamaya kadar, yumuşak bir
      kısılmayla çalar.`,
    trimmed: (dur, of) => `${of} içinden ilk ${dur} çalar`,
    tooLongFull: (name) => `"${name}" 15 dakikadan uzun.`,
    tooBig: 'Dosya 60 MB'tan büyük.',
    unreadable: 'Bu dosya ses olarak okunamıyor.',
    unsupported: 'Kendi seslerinizi yalnızca telefon uygulamasında ekleyebilirsiniz.',
    deleteTitle: (name) => `"${name}" kaldırılсын mı?`,
    deleteText: 'Bu sesi kullanan namazlar varsayılan sese döner.',
    delete: 'Kaldır',
    cancel: 'Vazgeç',
    play: (name) => `${name} - çal`,
    stop: (name) => `${name} - durdur`,
  },

  alarm: {
    app: 'Ezan',
    title: (prayer) => `${prayer} vakti`,
    stop: 'Ezanı durdur',
    stopAlarm: 'Alarmı durdur',
    mute: 'Sesi kapat',
    muteShort: 'Sessiz',
    close: 'Kapat',
    soundName: 'Ezan',
    channel: 'Namaz alarmı',
  },

  battery: {
    title: 'Arka plan etkinliği',
    lede: 'Telefon pil tasarrufu için Ezan'ı uyutabilir; alarm gecikebilir ya da hiç çalmayabilir.
    Uygulamanın kısıtlamasız çalışmasına izin verin.',
    yourPhone: 'Telefonunuz',
    open: 'Ezan ayarlarını aç',
    openList: '"Pil optimizasyonu" listesi',
    now: 'Şu an',
    ok: 'Kısıtlamasız ✓',
    bad: 'Kısıtlı',
    unknown: 'kontrol edilemiyor',
    otherPhone: 'Başka bir telefon mu?',
    more: 'Her marka için daha fazla ipucu: dontkillmyapp.com',
    brands: {
      samsung: 'Samsung',
      xiaomi: 'Xiaomi / Redmi / POCO',
      huawei: 'Huawei / Honor',
      oppo: 'OPPO / Realme / OnePlus',
      other: 'Diğer',
    },
  },

  steps: {
    samsung: [
      ['Aşağıdaki düğmeye dokunun - Ezan ayarları açılır.'],
      ['Açın:', 'Pil'],
      ['Seçin:', 'Kısıtlanmamış', '"Optimize edildi" ya da "Kısıtlanmış" değil.'],
      ['İsteğe bağlı:', 'Ayarlar → Pil → Arka plan kullanım sınırları', 'Ezan "Derin uykudaki
      uygulamalar" veya "Uykudaki uygulamalar" içinde olmamalı.'],
    ],
  },

```

```

xiaomi: [
  ['Aşağıdaki düğmeye dokunun - Ezan ayarları açılır.'],
  ['Açın:', 'Pil tasarrufu', '"Kısıtlama yok" seçeneğini seçin.'],
  ['Açın:', 'Otomatik başlatma'],
  ['İsteğe bağlı: son uygulamalarda Ezan kartını basılı tutun ve kilitleyin (asma kilit).'],
],
huawei: [
  ['Açın:', 'Ayarlar → Pil → Uygulama başlatma'],
  ['Ezan'ı bulun ve kapatın:', 'Otomatik olarak yönet'],
  ['Açılan pencerede üçünü de açın:', 'Otomatik başlatma · İkincil başlatma · Arka planda çalıştır'],
],
oppo: [
  ['Aşağıdaki düğmeye dokunun - Ezan ayarları açılır.'],
  ['Açın:', 'Pil kullanımı'],
  ['Açın:', 'Arka plan etkinliğine izin ver', 've varsa "Otomatik başlatma"yı.'],
],
other: [
  ['Aşağıdaki düğmeye dokunun - Ezan ayarları açılır.'],
  ['Açın:', 'Pil'],
  ['Seçin:', 'Kısıtlanmamış', 'Bazı telefonlarda adı "Optimize etme"dir.'],
],
},
},

about: {
  title: 'Hakkında',
  name: 'Ezan',
  back: 'Ayarlar',
  version: (v: string) => `Sürüm ${v}`,
  tagline: 'Namaz vakitleri, hatırlatmalar ve ezan',
  groupTimes: 'Vakitlerin kaynakları',
  groupPlaces: 'Yer verileri',
  groupPrivacy: 'Gizlilik',
  groupMap: 'Harita ve camiler',
  sources: {
    mufti: { name: 'Bulgaristan Başmüftülüğü', desc: 'Namaz takvimi - Bulgaristan'daki vakitler' },
    adhan: { name: 'adhan', desc: 'Bulgaristan dışında hesaplama · MIT lisansı' },
    osm: { name: 'OpenStreetMap', desc: 'Bulgaristan'daki yerler · © OpenStreetMap katkıcıları, ODbL' },
    ne: { name: 'Natural Earth', desc: 'Bulgaristan sınırı · kamu malı' },
    geonames: { name: 'GeoNames', desc: 'Dünya şehirleri - internetsiz ad ve ülke · CC BY 4.0' },
    praytimes: { name: 'PrayTimes.org', desc: 'Ezan kayıtları: Mekke, Medine, Mescid-i Aksa' },
    overpass: { name: 'OpenStreetMap · Overpass API', desc: 'Yakındaki camiler · © OpenStreetMap katkıcıları, ODbL' },
    openfreemap: { name: 'OpenFreeMap', desc: 'Harita · © OpenMapTiles, veriler © OpenStreetMap' },
  },
  privacy:
    'Konumunuz telefonda, yeri ve vakitleri seçmek için kullanılır. Bulgaristan dışında telefon şehrin adını kendi harita hizmetine sorar. „Yakındaki camiler“ için OpenStreetMap'e yalnızca yaklaşık bölge (yaklaşık 1 km) gönderilir; harita OpenFreeMap'ten yüklenir.',
},

mosques: {
  title: 'Yakındaki camiler',
  count: (n: number, dist: string) => `${n} · ${dist} içinde`,
  next: 'Sıradaki:',
  nextAt: (prayer: string, time: string) => `${prayer} ${time}`,
  nextIn: (left: string) => `${left} kaldı`,
  walk: (min: number) => `yürüyerek ${min} dk`,
  drive: (min: number) => `arabayla ~${min} dk`,
  unnamed: 'Cami',
  unnamedSub: 'OpenStreetMap'te adı yok',
}

```

```

    locating: 'Konumunuz bulunuyor...',
    loading: 'Yakındaki camiler aranıyor...',
    noneNear: (dist: string) => `${dist} içinde cami yok`,
    noneFar: (dist: string) => `${dist} içinde de cami yok`,
    noneHint: 'Harita OpenStreetMap\'tir - küçük yerlerde her cami işaretli değildir.',
    searchFar: (dist: string) => `${dist} içinde ara`,
    within: (dist: string) => `${dist} içinde`,
    offlineTitle: 'İnternet yok.',
    offlineText: (when: string) => `Buradaki son arama gösteriliyor (${when}). Yol tarifi bağlantı gelince çalışır.`,
    today: 'bugün',
    yesterday: 'dün',
    errorOffline: 'İnternet yok',
    errorOfflineText: 'Camiler OpenStreetMap\'te aranır - bağlantı gerekir.',
    errorServer: 'Sunucu yanıt vermiyor',
    errorServerText: 'OpenStreetMap (Overpass) yoğun. Bir dakika sonra tekrar deneyin.',
    retry: 'Tekrar dene',
    noGps: (place: string) => `Konum izni yok - ${place} çevresinde aranıyor.`,
    allow: 'İzin ver',
    myLocation: 'Konumum',
    locate: 'Konumuma git',
    back: 'Listeye dön',
    reach: (min: number) => `${min} dakikada varırsınız`,
    directions: (app: string) => `${app} ile yol tarifi`,
    directionsAuto: 'Yol tarifi',
    withApp: (app: string) => `${app} ile · Ayarlar → Navigasyon bölümünden değiştirin`,
    withAuto: 'Telefonun haritalarıyla · Ayarlar → Navigasyon bölümünden değiştirin',
    share: 'Paylaş',
    copyAddress: 'Adresi kopyala',
    copyCoords: 'Koordinatları kopyala',
    copied: 'Kopyalandı',
    noAddress: 'OpenStreetMap\'te adres yok',
    allyRow: (name: string, dist: string, time: string) => `${name}, ${dist}, ${time}`,
    allyMap: (n: number) => `Harita: ${n} cami`,
    attribution: 'OpenFreeMap © OpenMapTiles © OpenStreetMap',
  },

  nav: {
    title: 'Navigasyon',
    back: 'Ayarlar',
    intro: 'Camiye „Yol tarifi“ hangi uygulamayla açılsın.',
    auto: 'Otomatik',
    autoAndroid: 'Telefonun varsayılan harita uygulaması',
    autoIos: 'Apple Haritalar',
    walkDrive: 'Yürüyerek ve arabayla',
    driveOnly: 'Yalnızca arabayla',
    notInstalled: 'yükü değil',
    organic: 'OpenStreetMap · internetsiz de çalışır',
    huawei: 'Huawei telefonlarda',
    download: 'İndir',
    note: 'Waze yalnızca araba için yol tarifi verir - 300 m uzaktaki camiye bile arabayla götürür. Seçilen uygulama yoksa „Yol tarifi“ hangisiyle açılacağını sorar.',
    sheetTitle: (name: string) => `Yol tarifi: ${name}`,
    sheetSub: (dist: string) => `${dist} · hangi uygulamayla?`,
    walk: 'Yürüyerek',
    drive: 'arabayla',
    remember: 'Seçimimi hatırla',
    rememberHint: 'Değiştirmek için: Ayarlar → Navigasyon',
    missing: (app: string) => `${app} artık telefonda yok - başka bir uygulama seçin.`,
    group: 'Yakındaki camiler',
    row: 'Navigasyon',
  },

```

```

    rowNote: 'Camiye „Yol tarifi“ hangi uygulamayla açılır.',
    ask: 'Her seferinde sor',
    close: 'Kapat',
  },

  holidays: {
    tabTimes: 'Vakitler',
    tabHolidays: 'Dini günler',
    hijriYears: (a: number, b: number) => (a === b ? `Hicri ${a}` : `Hicri ${a} - ${b}`),
    next: 'Sıradaki',
    today: 'bugün',
    tomorrow: 'yarın',
    daysWord: () => 'gün',
    inDays: (n: number) => (n === 0 ? 'bugün' : n === 1 ? 'yarın' : `${n} gün sonra`),
    evening: 'akşam',
    daysCount: (n: number) => `${n} gün`,
    expected: 'tahmini',
    expectedNote:
      'Tahmini - burada Ramazan'ın ve bayramların başlangıcı, bir önceki akşam hilal gözlemiyle ilan edilir. Tarih 1 gün değişebilir.',
    computedNote: 'Diyanet tarihleri 2028 başına kadar ilan etti; bu tarih aynı astronomik kurallara göredir.',
    noData: 'Bu yıl için veri yok.',
    dates: 'Tarihler',
    date: 'Tarih',
    hijri: 'Hicri',
    arefe: 'Arife',
    remindNight: '0 akşam hatırlat',
    remindDay: 'Bir önceki akşam hatırlat',
    remindAt: (date: string) => `${date} · akşam namazı vakti`,
    source: {
      mufti: 'Kaynak: Bulgaristan Başmüftülüğü (Diyanet takvimi)',
      diyanet: 'Kaynak: Diyanet İşleri Başkanlığı',
      ummalqura: 'Kaynak: Ümmü'l-Kurâ (Suudi Arabistan)',
    } as Record<'mufti' | 'diyanet' | 'ummalqura', string>,
    names: {
      threeMonths: 'Üç Ayların Başlangıcı',
      regaib: 'Regaip Kandili',
      miraj: 'Miraç Kandili',
      berat: 'Berat Kandili',
      ramadan: 'Ramazan Başlangıcı',
      qadr: 'Kadir Gecesi',
      fitrEve: 'Ramazan Bayramı Arifesi',
      fitr: 'Ramazan Bayramı',
      arafah: 'Kurban Bayramı Arifesi',
      adha: 'Kurban Bayramı',
      newYear: 'Hicri Yılbaşı',
      ashura: 'Aşure Günü',
      mawlid: 'Mevlid Kandili',
    } as Record<HolidayId, string>,
    about: {
      threeMonths: 'Mübarek Recep, Şaban ve Ramazan ayları başlıyor.',
      regaib: 'Recep ayının ilk cumasına bağlanan gece - Üç ayların mübarek gecelerinin ilki.',
      miraj: 'İsra ve Miraç gecesi - Peygamberimizin ﷺ Mekke'den Kudüs'e gece yolculuğu ve göğe yükselişi.',
      berat: 'Şaban ayının 15. gecesi - af ve dua gecesi.',
      ramadan: 'Orucun ilk günü. İlk teravih namazı bir önceki akşam kılınır.',
      qadr: '„Bin aydan daha hayırlı“ olan gece. Ramazan'ın 27. gecesinde idrak edilir.',
      fitrEve: 'Ramazan'ın son günü - Ramazan Bayramı'ndan önceki gün.',
      fitr: 'Ramazan'ın ardından gelen bayram. İlk gün sabah bayram namazıyla başlar.',
      arafah: 'Hacıların Arafat'ta vakfe yaptığı gün - Kurban Bayramı'ndan önceki gün.',
    }
  }

```

```

    adha: 'Kurban bayramı. İlk gün sabah bayram namazıyla başlar.',
    newYear: '1 Muharrem - hicri yılın başlangıcı.',
    ashura: '10 Muharrem - oruç ve anma günü.',
    mawlid: 'Peygamberimiz Hz. Muhammed'in ﷺ doğum gecesi - Rebiülevvel'in 12. gecesi.',
  } as Record<HolidayId, string>,
  notifyTonight: (name: string) => `Bu akşam: ${name}`,
  notifyTomorrow: (name: string) => `Yarın: ${name}`,
  allyRow: (name: string, date: string, hijri: string) => `${name}, ${date}, ${hijri}`,
  prevYear: 'Önceki yıl',
  nextYear: 'Sonraki yıl',
  close: 'Kapat',
},

holidaySource: {
  title: 'Dini gün tarihleri',
  back: 'Ayarlar',
  intro:
    'Takvim kimin tarihlerini gösterebilir. Farklı topluluklar bazen farklı günlerde bayram yapar - her iki tarih de kendi topluluğu için doğrudur.',
  auto: 'Otomatik',
  autoDesc: (source: string) => `Konuma göre: ${source}`,
  mufti: 'Başmüftülük (Bulgaristan)',
  muftiDesc: 'Önceden ilan edilir - Diyanet ile aynı takvim',
  diyanet: 'Diyanet (Türkiye)',
  diyanetDesc: '2028'e kadar ilan edildi, sonrakilere aynı kurallarla · Avrupa'daki Diyanet camileri için de',
  ummalqura: 'Ümmü'l-Kurâ (Suudi Arabistan)',
  ummalquraDesc: 'Ramazan ve bayramlar - hilal gözlemiyle ilan edilene kadar tahmini',
  short: { mufti: 'Başmüftülük', diyanet: 'Diyanet', ummalqura: 'Ümmü'l-Kurâ' } as Record<'mufti' | 'diyanet' | 'ummalqura', string>,
  autoValue: (short: string) => `Otomatik · ${short}`,
  note: '„Bugün“ ve „Ay“ ekranlarındaki hicri tarih de aynı seçime uyar.',
  group: 'Dini günler',
  row: 'Dini gün tarihleri',
},

methods: {
  Turkey: 'Diyanet (Türkiye)',
  MuslimWorldLeague: 'Müslüman Dünya Birliği',
  MoonsightingCommittee: 'Moonsighting Committee',
  Egyptian: 'Mısır Genel Araştırma Kurumu',
  Karachi: 'Karaçi Üniversitesi',
  UmmAlQura: 'Ümmü'l-Kurâ (Mekke)',
  NorthAmerica: 'ISNA (Kuzey Amerika)',
  Dubai: 'Dubai (BAE)',
  Kuwait: 'Kuveyt',
  Qatar: 'Katar',
  Singapore: 'Singapur (MUIS)',
  Tehran: 'Tahran (Jeofizik Enstitüsü)',
  France: 'Fransa (UOIF, 12°)',
  Russia: 'Rusya (DUM)',
  Malaysia: 'Malezya (JAKIM)',
  Indonesia: 'Endonezya (Kemenag)',
},

highLatRules: {
  seventhofthenight: { name: 'Gecenin yedide biri', desc: 'Avrupa'da en yaygın olanı' },
  middleofthenight: { name: 'Gecenin yarısı', desc: 'İmsak ve Yatsı gece yarısını geçmez' },
  twilightangle: { name: 'Alacakaranlık açısı', desc: 'Yöntemin açısına göre gecenin bir payı' },
},

```

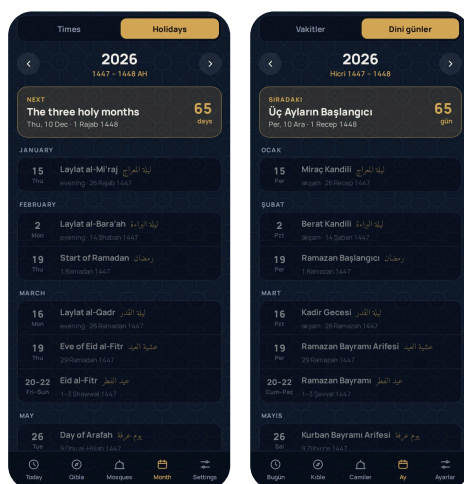
```

place: {
  title: 'Konum seçin',
  search: 'Bulgaristan'da yer arayın',
  useGps: 'Konumumu kullan (GPS)',
  cities: 'Şehirler',
  noResults: 'Böyle bir yer yok',
  abroadNote: 'Bulgaristan dışında konumunuz GPS'ten alınır.',
  close: 'Kapat',
},

date: {
  weekdaysShort: ['Paz', 'Pzt', 'Sal', 'Çar', 'Per', 'Cum', 'Cmt'],
  monthsShort: ['Oca', 'Şub', 'Mar', 'Nis', 'May', 'Haz', 'Tem', 'Ağu', 'Eyl', 'Eki', 'Kas', 'Ara'],
  monthsFull: ['Ocak', 'Şubat', 'Mart', 'Nisan', 'Mayıs', 'Haziran', 'Temmuz', 'Ağustos', 'Eylül',
'Ekim', 'Kasım', 'Aralık'],
},

hijriMonths: [
  'Muharrem',
  'Safer',
  'Rebiülevvel',
  'Rebiülahir',
  'Cemaziyelevvel',
  'Cemaziyelahir',
  'Recep',
  'Şaban',
  'Ramazan',
  'Şevval',
  'Zilkade',
  'Zilhicce',
],
};

```



Английски и турски

Преди публикуване: кратките обяснения на празниците да ги прегледа имам или богослов – на трите езика.

Стъпка 7

Тестове

266 теста общо, 8 нови.

holidays.test.ts src/domain/__tests__

- Всички празници на 2026 г. и Байрамите на 2027 г. – точно като обявените от Диянет / Мюфтийството.
- Свещените нощи 2021–2023 и Регаиб 2025 – срещу обявените дати.
- Умм ал-Кура: Рамазан и Байрамите – очаквани; „Автоматично“ по държавата; следващият празник; датата по Хиджра днес.
- Напомнянията: Нощ Регаиб – същата вечер, Курбан Байрам – вечерта преди, при Магриб.

src/domain/__tests__/holidays.test.ts

137 реда

```
import { bg } from '@i18n/strings';

import { toHijri } from '../hijri';
import { hijriFromTable } from '../hijriTable';
import { holidaysInYear, nextHoliday, resolveHolidaySource, type HolidayId } from '../holidays';
import { planHolidayReminders } from '../notifications';
import type { AppLocation } from '../location';
import type { TimesOptions } from '../times';

const ymd = (d: Date) => `${d.getFullYear()}-${String(d.getMonth() + 1).padStart(2, '0')}-${String(d.getDate()).padStart(2, '0')}`;
const dates = (year: number, source: 'mufti' | 'diyanet' | 'ummalqura' = 'mufti') => {
  const out: Partial<Record<HolidayId, string>> = {};
  for (const h of holidaysInYear(source, year)) out[h.id] = h.end.getTime() === h.date.getTime() ?
  ymd(h.date) : `${ymd(h.date)}/${ymd(h.end)}`;
  return out;
};

describe('празници – точно като обявените от Диянет / Мюфтийството', () => {
  it('2026 (утвърдени и в България: Рамазан Байрам 20 март, Курбан Байрам 27 май)', () => {
    expect(dates(2026)).toEqual({
      miraj: '2026-01-15',
      berat: '2026-02-02',
      ramadan: '2026-02-19',
      qadr: '2026-03-16',
      fitrEve: '2026-03-19',
      fitr: '2026-03-20/2026-03-22',
      arafah: '2026-05-26',
      adha: '2026-05-27/2026-05-30',
      newYear: '2026-06-16',
      ashura: '2026-06-25',
      mawlid: '2026-08-24',
      threeMonths: '2026-12-10',
      regaib: '2026-12-10',
    });
  });

  it('2027 – Байрамите', () => {
    const d = dates(2027);
    expect([d.fitrEve, d.fitr, d.arafah, d.adha]).toEqual([
      '2027-03-08',
      '2027-03-09/2027-03-11',
      '2027-05-15',
      '2027-05-16/2027-05-19',
    ]);
  });

  it('2021–2023 – свещените нощи', () => {
```

```

const a = dates(2021);
expect([a.regaib, a.miraj, a.berat, a.qadr, a.mawlid]).toEqual(['2021-02-18', '2021-03-10', '2021-03-27', '2021-05-08', '2021-10-17']);
const b = dates(2022);
expect([b.regaib, b.miraj, b.berat, b.ramadan, b.qadr, b.fitr?.slice(0, 10), b.adha?.slice(0, 10)]).toEqual([
  '2022-02-03',
  '2022-02-27',
  '2022-03-17',
  '2022-04-02',
  '2022-04-27',
  '2022-05-02',
  '2022-07-09',
]);
const c = dates(2023);
expect([c.threeMonths, c.regaib, c.miraj, c.berat, c.ramadan, c.qadr, c.mawlid]).toEqual([
  '2023-01-23',
  '2023-01-26',
  '2023-02-17',
  '2023-03-06',
  '2023-03-23',
  '2023-04-17',
  '2023-09-26',
]);
expect(dates(2025).regaib).toBe('2025-12-25');
});

it('Умм ал-Кура: Рамазан и Байрамите – очаквани, Новата година – не', () => {
  const list = holidaysInYear('ummalqura', 2027);
  const get = (id: HolidayId) => list.find((h) => h.id === id!);
  expect(ymd(get('ramadan').date)).toBe('2027-02-08');
  expect(get('ramadan').expected).toBe(true);
  expect(get('adha').expected).toBe(true);
  expect(get('newYear').expected).toBe(false);
  expect(holidaysInYear('mufti', 2027).some((h) => h.expected)).toBe(false);
});

it('„Автоматично“ – според държавата', () => {
  expect(resolveHolidaySource('auto', 'BG')).toBe('mufti');
  expect(resolveHolidaySource('auto', 'TR')).toBe('diyanet');
  expect(resolveHolidaySource('auto', 'DE')).toBe('diyanet');
  expect(resolveHolidaySource('auto', 'SA')).toBe('ummalqura');
  expect(resolveHolidaySource('auto', null)).toBe('diyanet');
  expect(resolveHolidaySource('ummalqura', 'BG')).toBe('ummalqura');
});

it('следващият празник и датата по Хиджра', () => {
  const now = new Date(2026, 9, 6, 10);
  expect(ymd(nextHoliday('mufti', now)!.date)).toBe('2026-12-10');
  expect(toHijri(now, 0, 'diyanet')).toEqual({ day: 25, month: 4, year: 1448 });
  expect(hijriFromTable(new Date(2026, 2, 20, 12), 'diyanet')).toEqual({ day: 1, month: 10, year: 1447 });
});
});

describe('празници – напомнянията', () => {
  const location: AppLocation = {
    id: 'kardzhali',
    names: { bg: 'Кърджали', en: 'Kardzhali' },
    latitude: 41.64,
    longitude: 25.37,
  };

```

```

    source: 'calc',
    country: 'BG',
  };
  const options = { location, method: 'Turkey', madhab: 'shafi', highLatRule: 'middleofthenight', offsets:
{ fajr: 0, sunrise: 0, dhuhr: 0, asr: 0, maghrib: 0, isha: 0 } } as unknown as TimesOptions;

  it('свещена нощ – същата вечер при Магриб; празник – вечерта преди', () => {
    const plan = planHolidayReminders({
      now: new Date(2026, 11, 1, 9),
      options,
      source: 'mufti',
      enabled: { regaib: true },
      texts: bg.holidays,
    });
    expect(plan).toHaveLength(1);
    expect(ymd(plan[0].at)).toBe('2026-12-10');
    expect(plan[0].at.getHours()).toBeGreaterThanOrEqual(16);
    expect(plan[0].title).toBe('Тази вечер: Нощ Регаиб');

    const adha = planHolidayReminders({
      now: new Date(2026, 4, 20, 9),
      options,
      source: 'mufti',
      enabled: { adha: true },
      texts: bg.holidays,
    });
    expect(ymd(adha[0].at)).toBe('2026-05-26');
    expect(adha[0].title).toBe('Утре: Курбан Байрам');
  });

  it('без включени напомняния – нищо', () => {
    expect(planHolidayReminders({ now: new Date(2026, 4, 20), options, source: 'mufti', enabled: {}, texts:
bg.holidays })).toEqual([]);
  });
});

```

Трябва да видиш: Tests: 266 passed, 266 total

npm test

Стъпка 8

Проверка на телефона

Samsung и Huawei:

- „Месец“ → „Празници“: 2026 г., следващият е 10 дек. (Трите свещени месеца и Нощ Регаиб), след 65 дни (на 6 окт.).
- Стрелките: 2027 и 2025 г.; Рамазан Байрам 2027 – 9–11 март, Курбан Байрам – 16–19 май.
- „Днес“: датата по Хиджра е същата като в „Месец“.
- Докосни Нощ Регаиб → „Напомни ми същата вечер“. Настройки → Проверка на известията: напомнянето е там, ако празникът е до 30 дни.
- Настройки → Празници → Дати на празниците → Умм ал-Кура: Рамазан и Байрамите стават „≈ ... очаквана“. Върни „Автоматично“.
- Езикът на телефона на турски и английски – имената на празниците.

Терминал

```
git add -A  
git status  
git commit -m "Stage 13: Islamic holidays"  
git tag stage-13  
git push  
git push origin stage-13
```